(Not) Understanding Neural Networks

DD2412, Josephine Sullivan

October, 2021

Deep learning & neural networks

Generalization not well understood

The Mystery of Generalization in Deep Learning

- GoogLeNet, VGGNet,.... have many millions of parameters.
- Networks trained with (ignoring data augmentations)
  # training points $\ll$ # of parameters.
- But these networks still generalize well.
- What's going on??

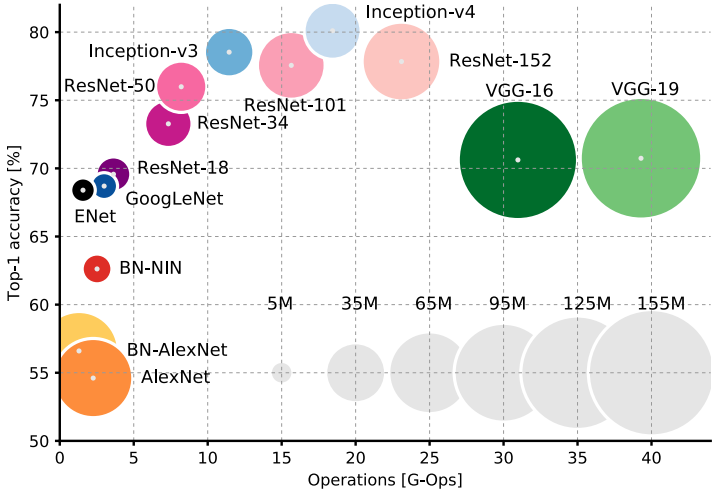
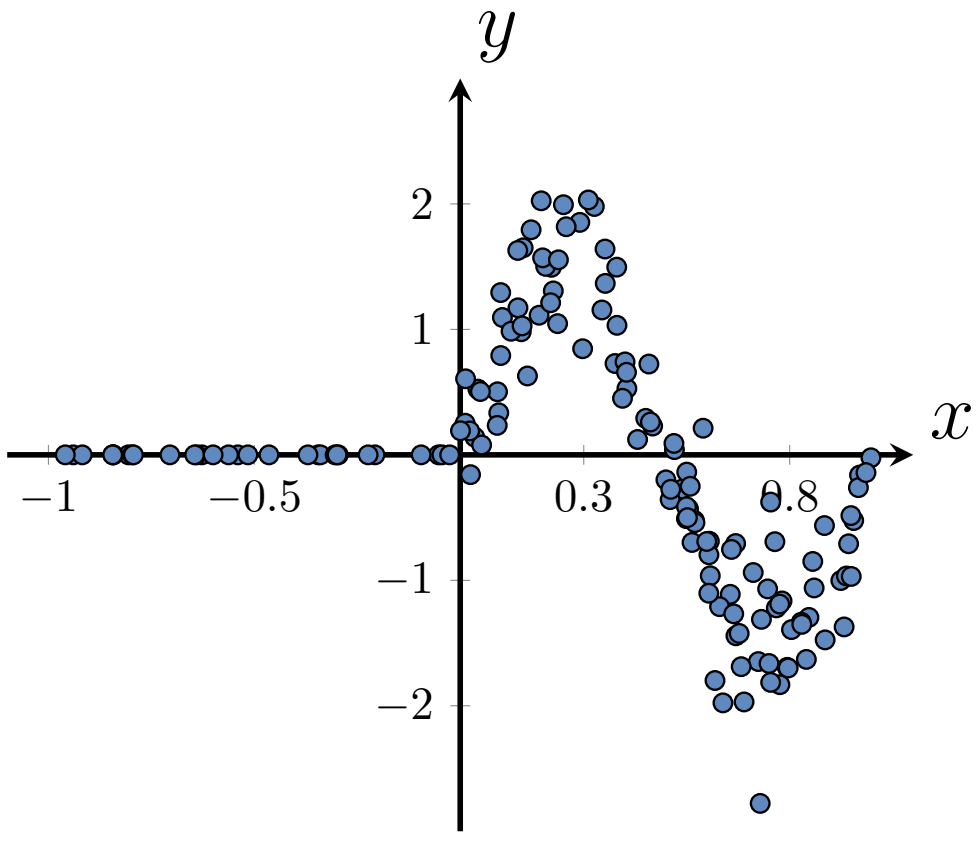


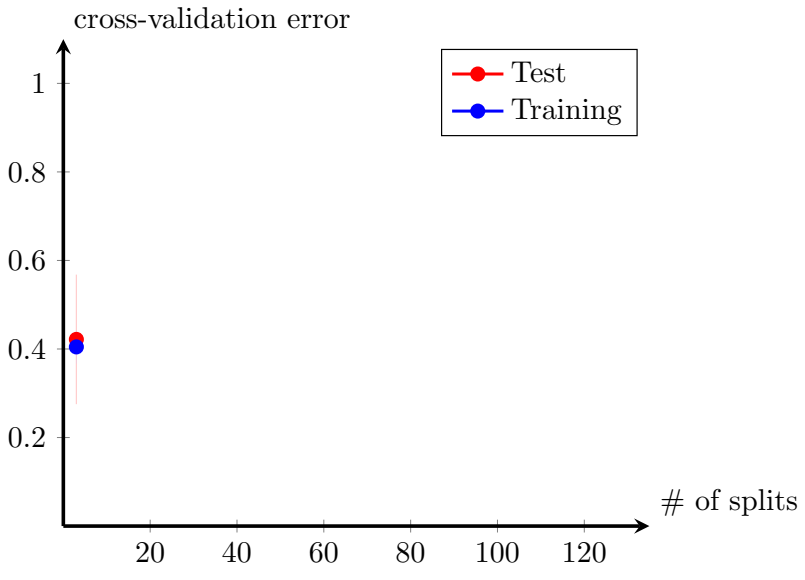
Fig credit: **An Analysis of Deep Neural Network Models for Practical Applications** by Canziani, Culurciello & Paszke.

Consider this 1D regression problem

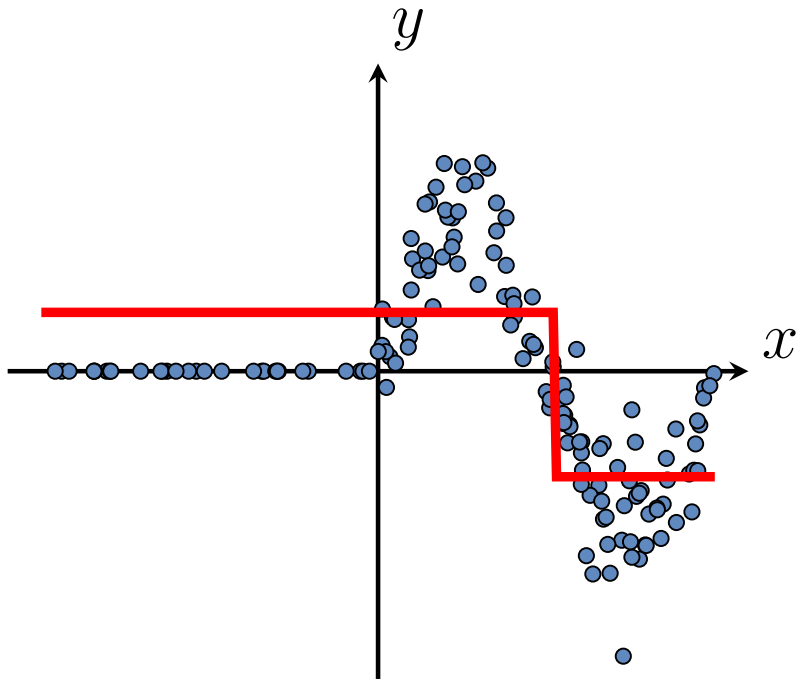


Have 120 data points.

Single Decision Tree with no regularization

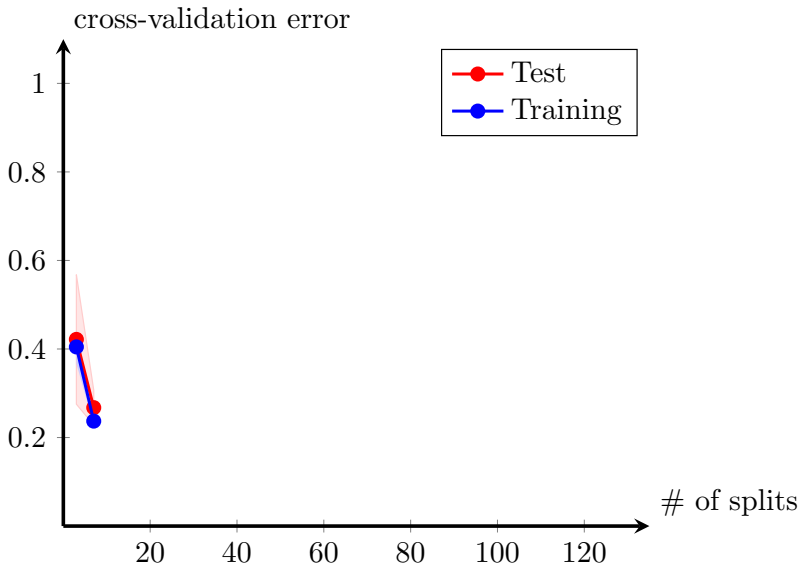


MSE scores

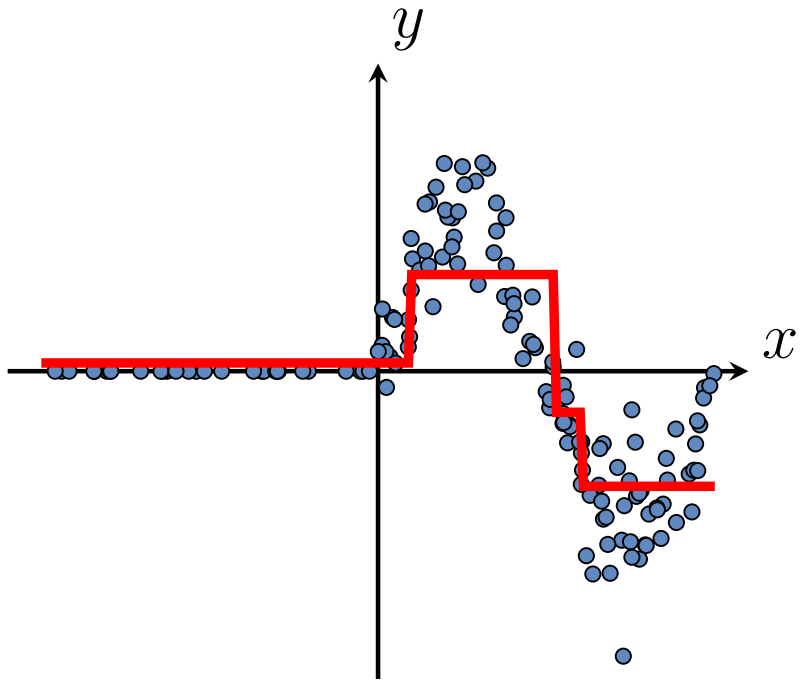


Predicted regression curve

Single Decision Tree with no regularization

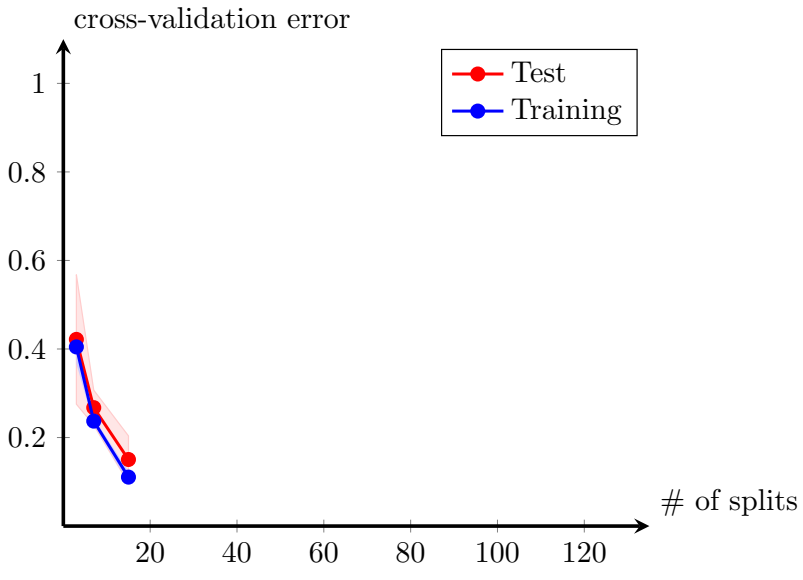


MSE scores

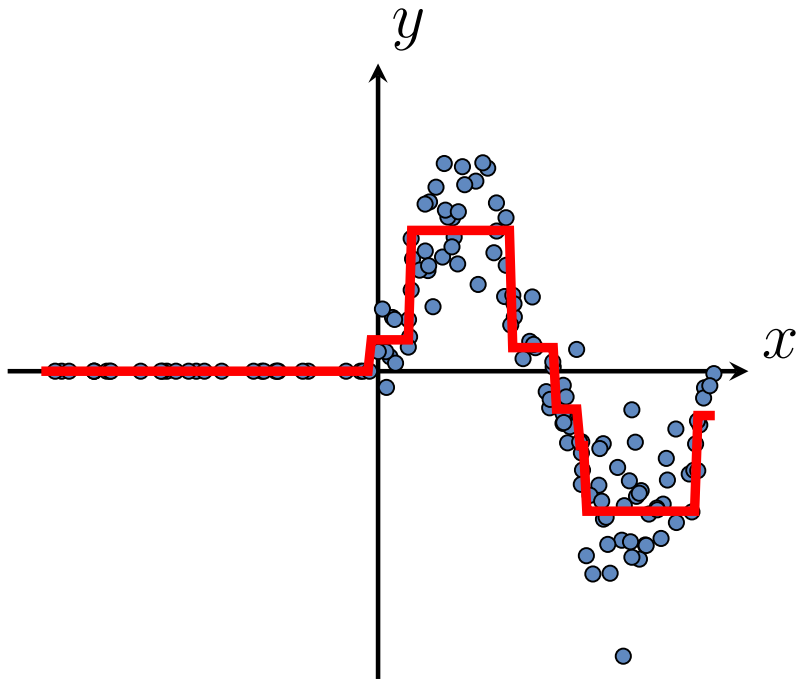


Predicted regression curve

Single Decision Tree with no regularization

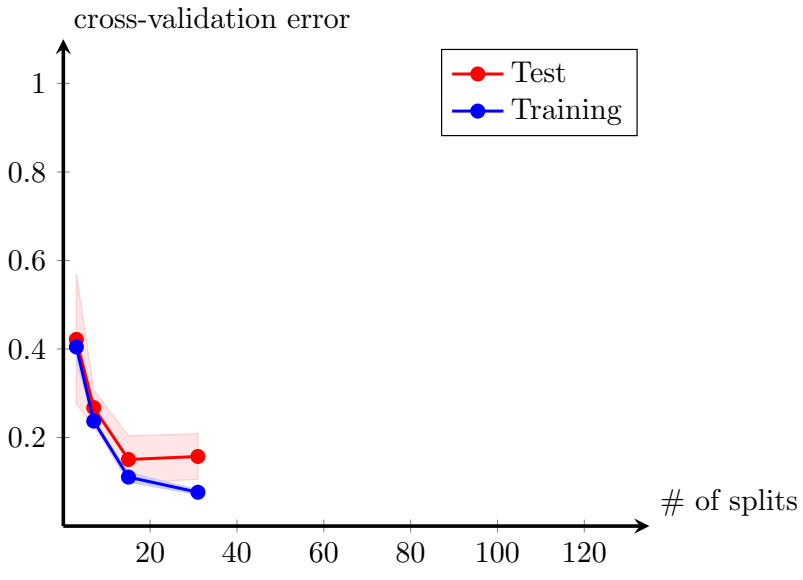


MSE scores

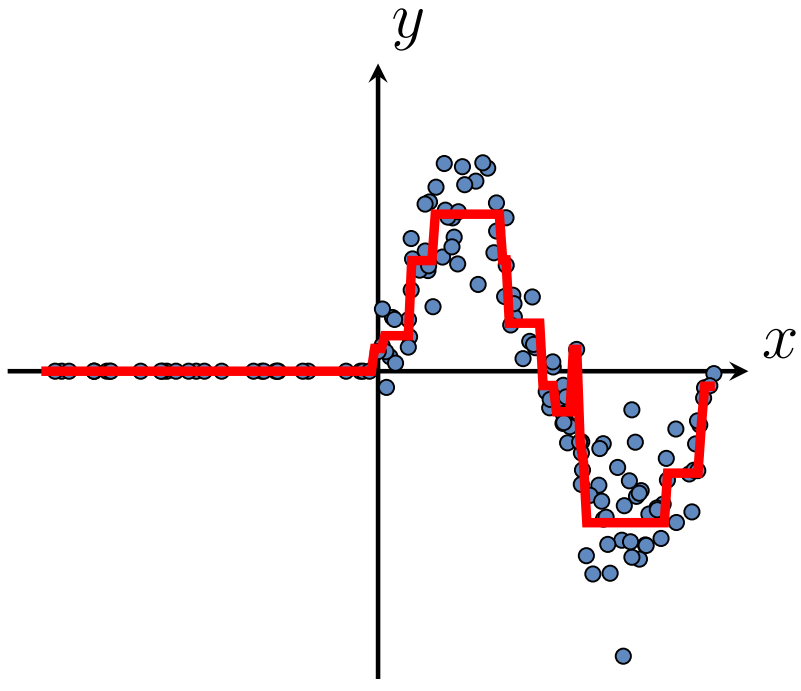


Predicted regression curve

Single Decision Tree with no regularization

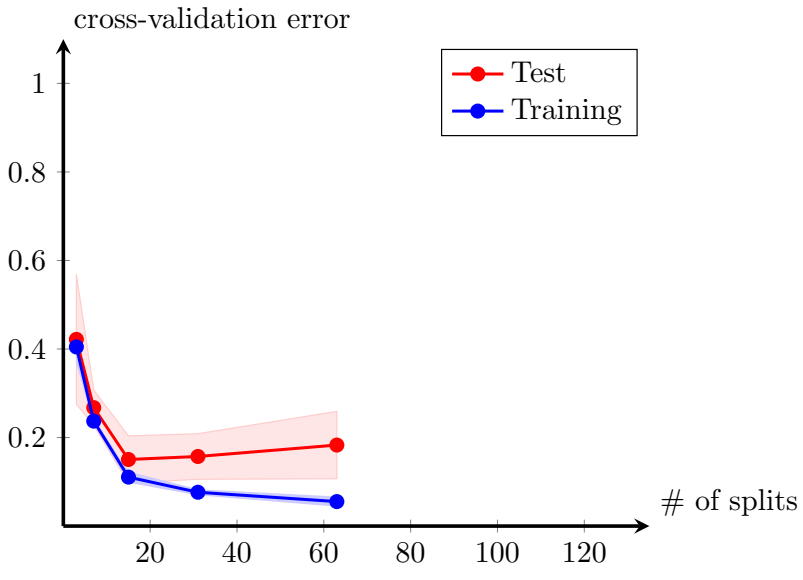


MSE scores

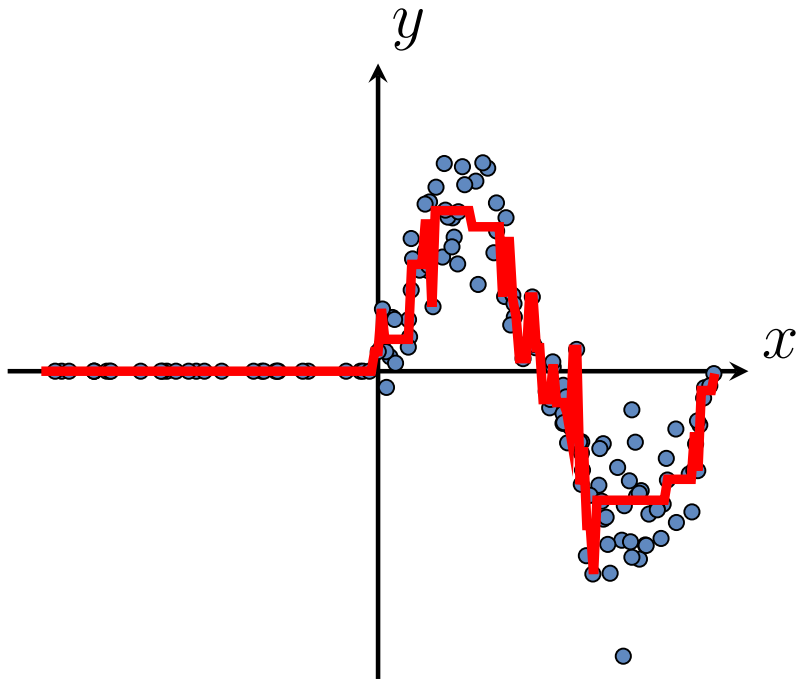


Predicted regression curve

Single Decision Tree with no regularization

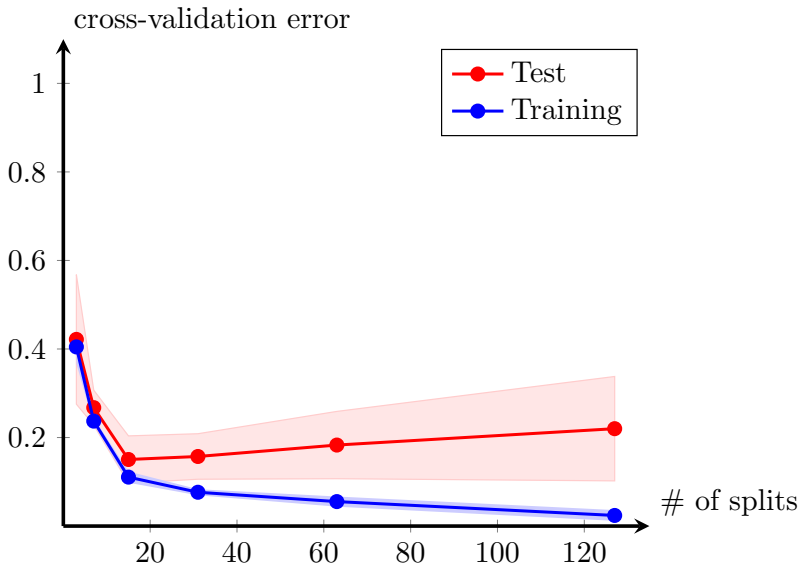


MSE scores

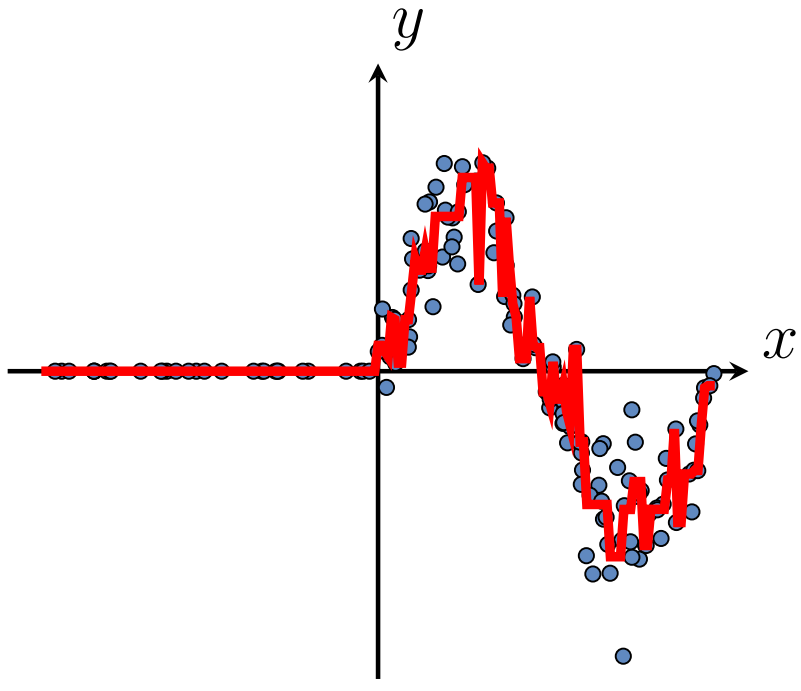


Predicted regression curve

Single Decision Tree with no regularization

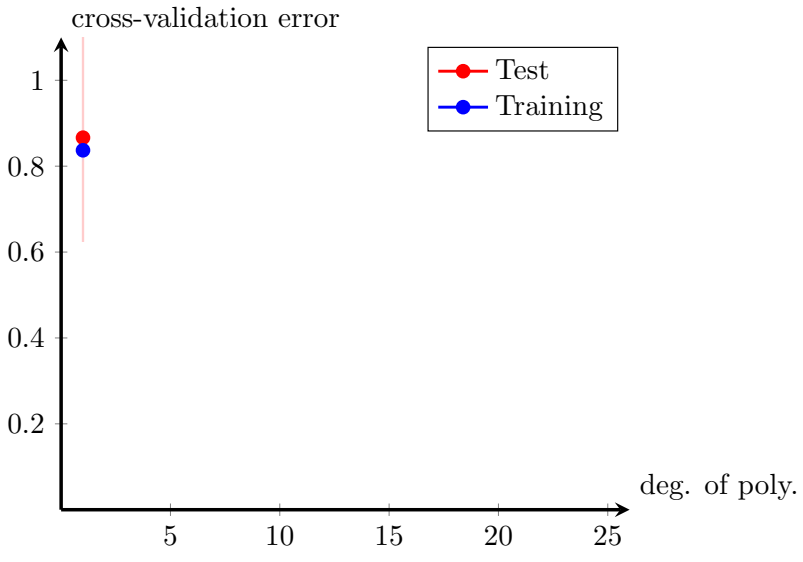


MSE scores

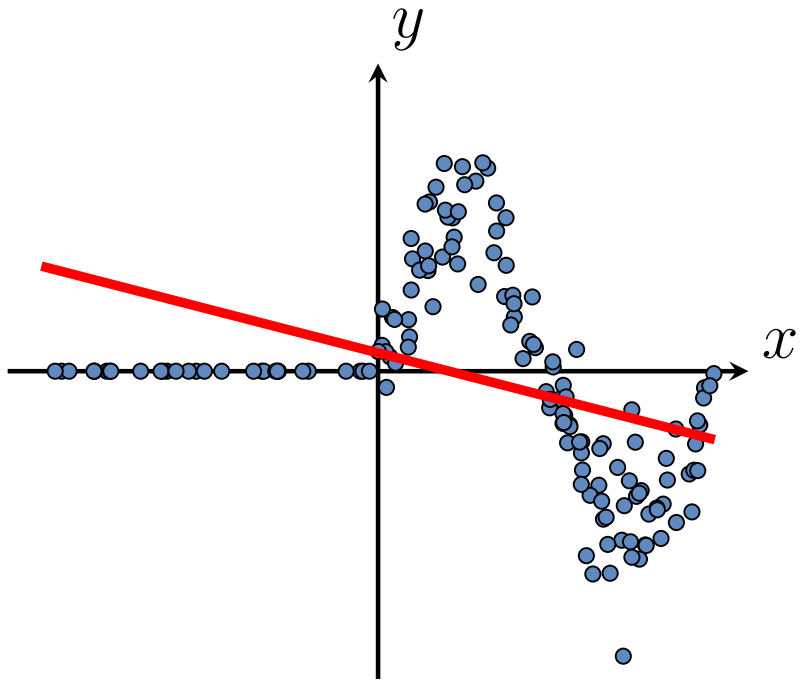


Predicted regression curve

Polynomial Regression with no regularization

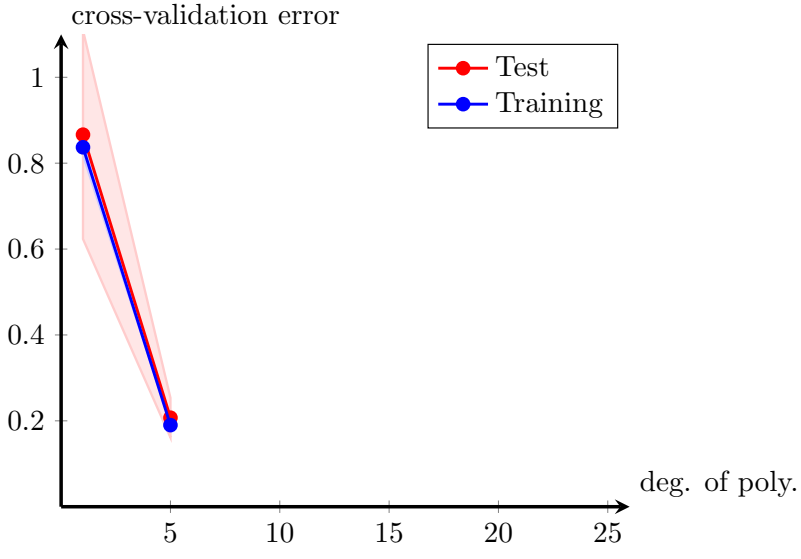


MSE scores

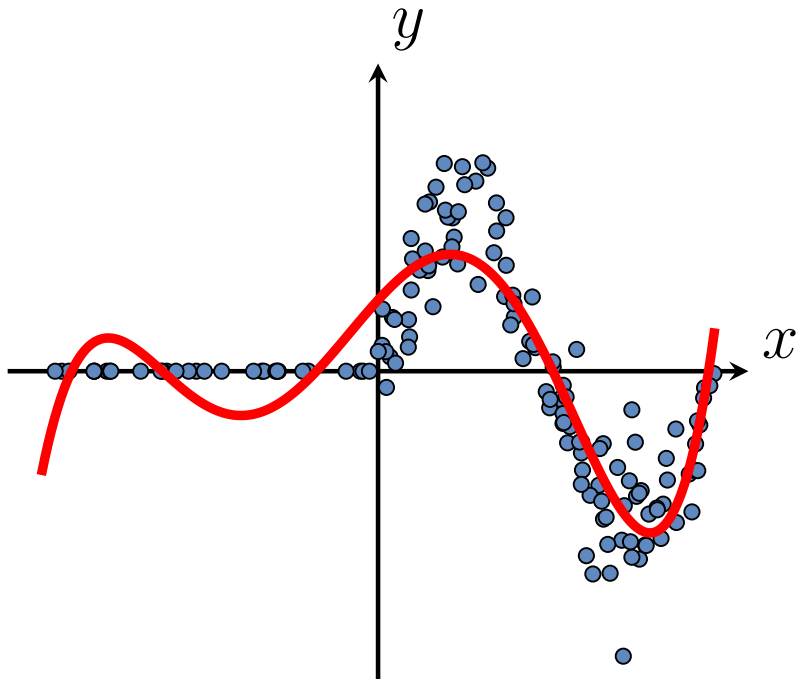


Predicted regression curve

Polynomial Regression with no regularization

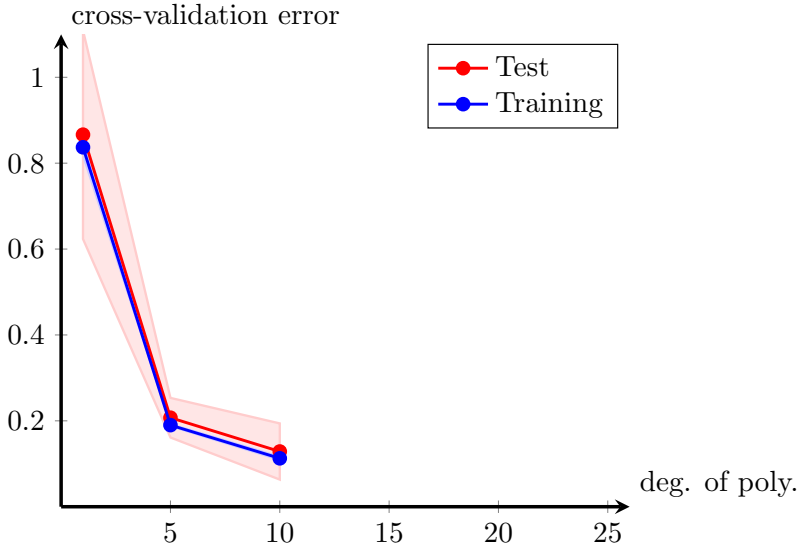


MSE scores

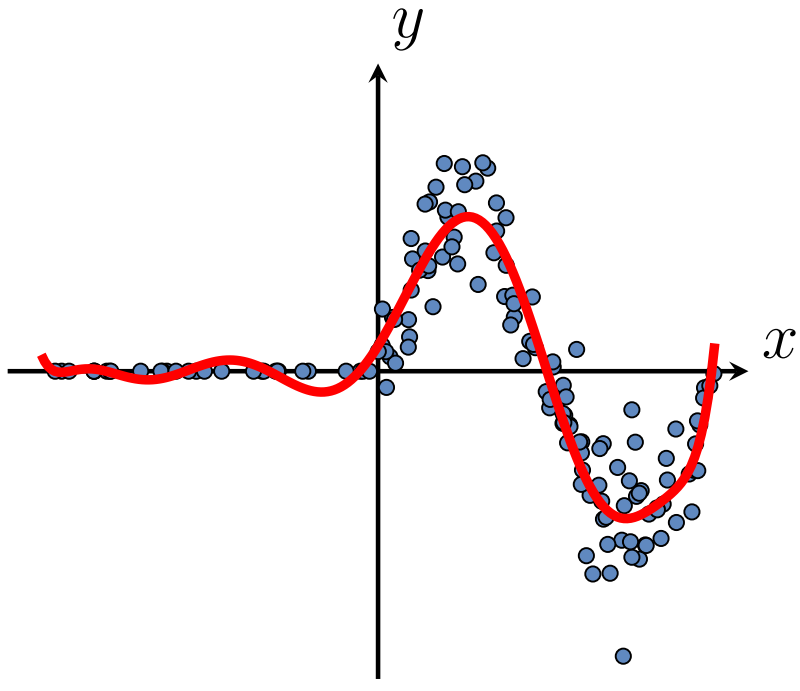


Predicted regression curve

Polynomial Regression with no regularization

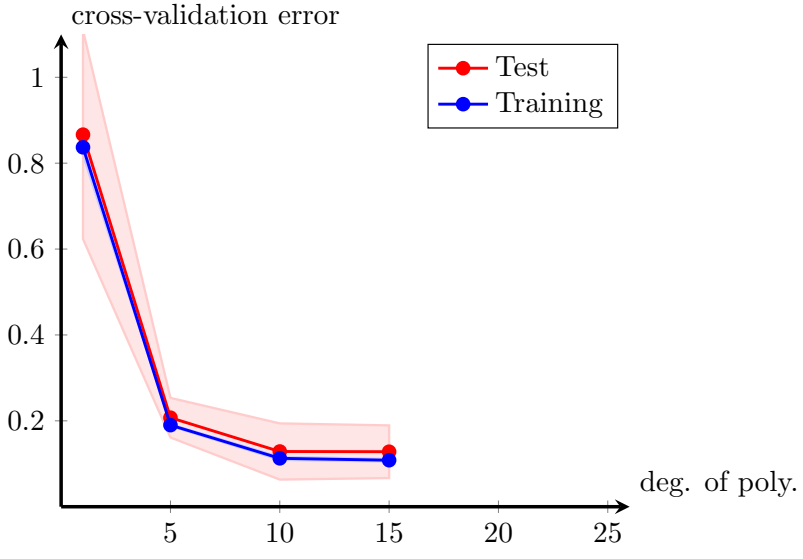


MSE scores

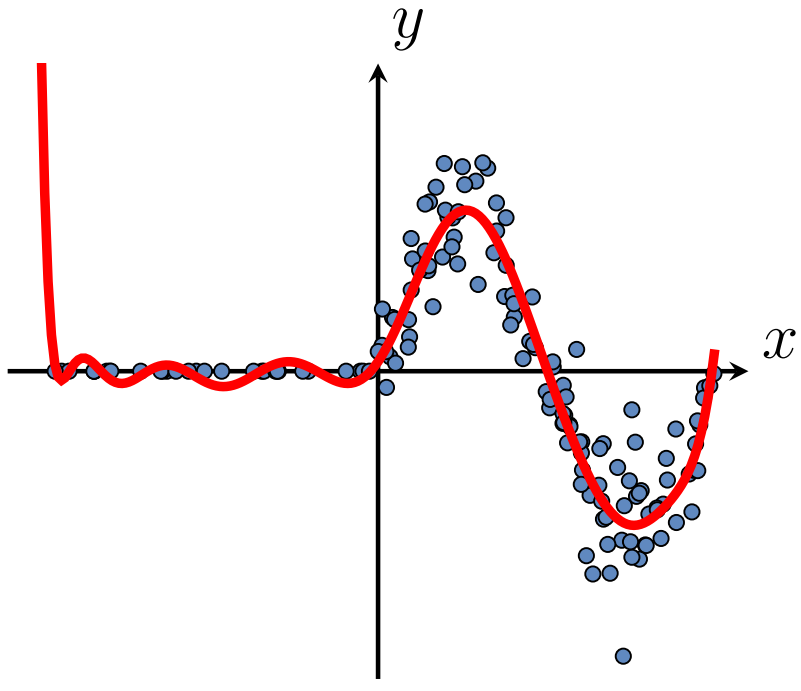


Predicted regression curve

Polynomial Regression with no regularization

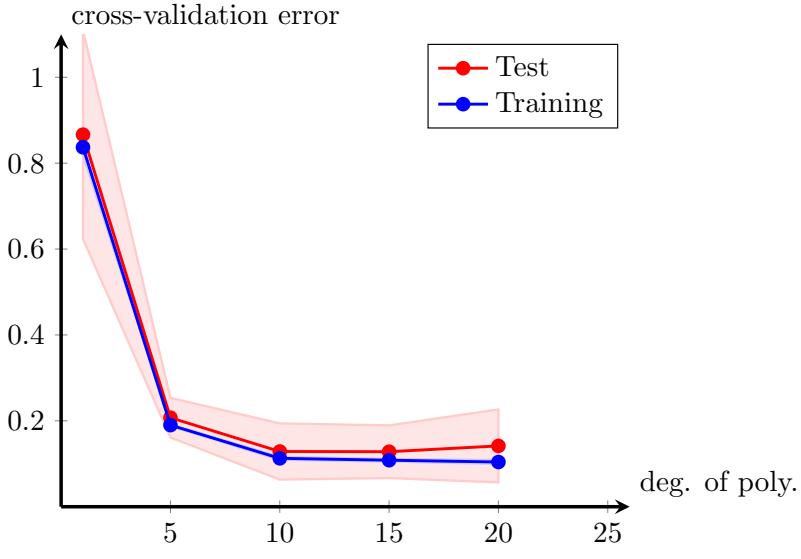


MSE scores

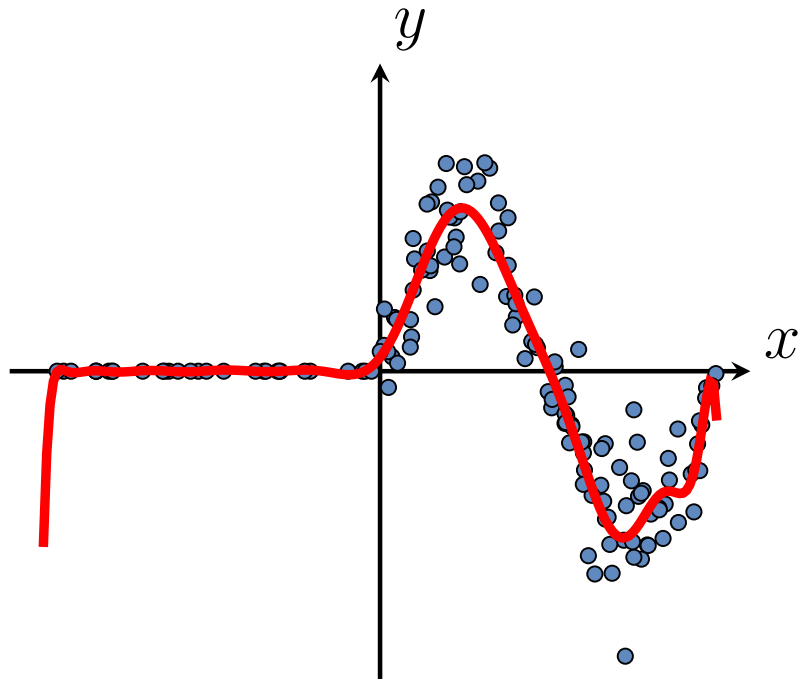


Predicted regression curve

Polynomial Regression with no regularization

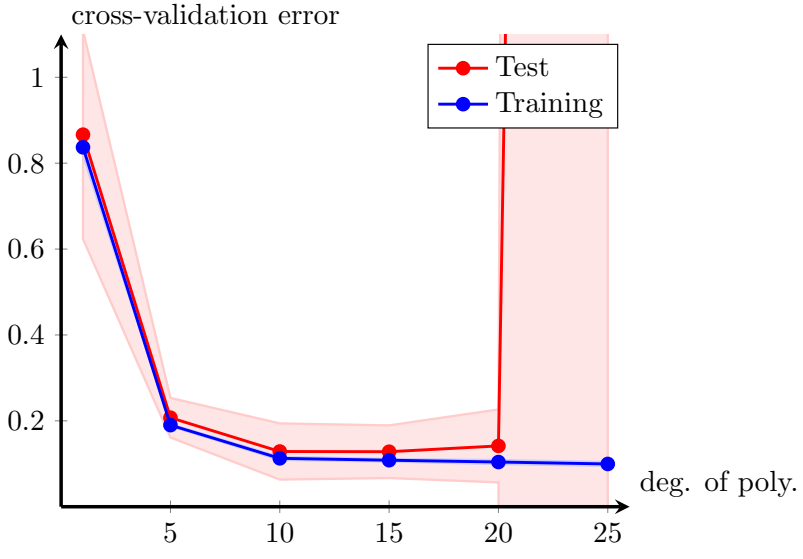


MSE scores

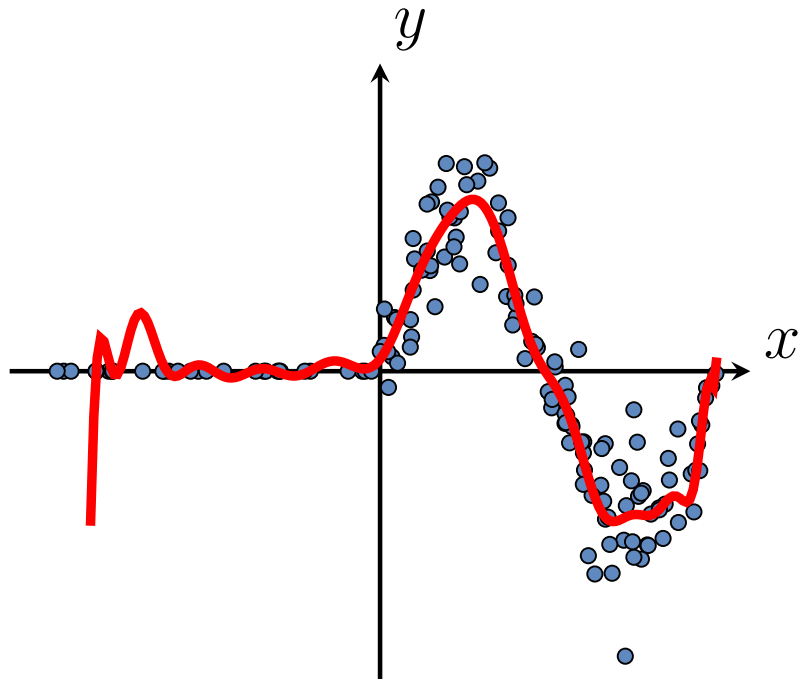


Predicted regression curve

Polynomial Regression with no regularization



MSE scores

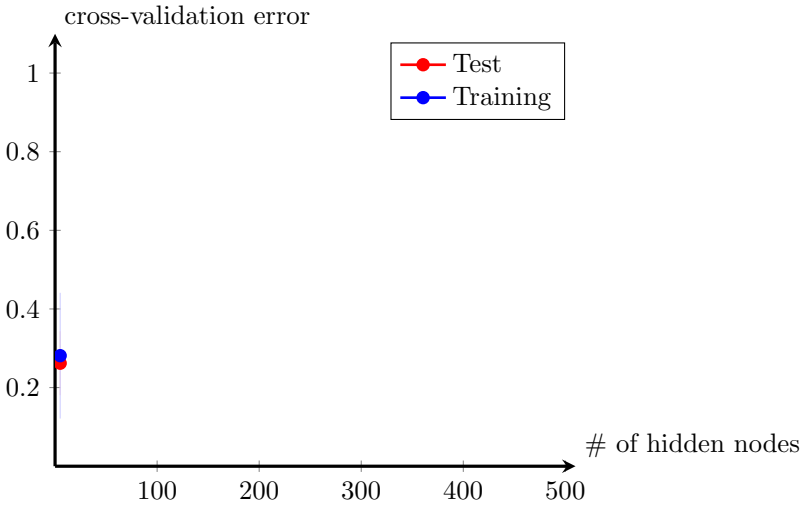


Predicted regression curve

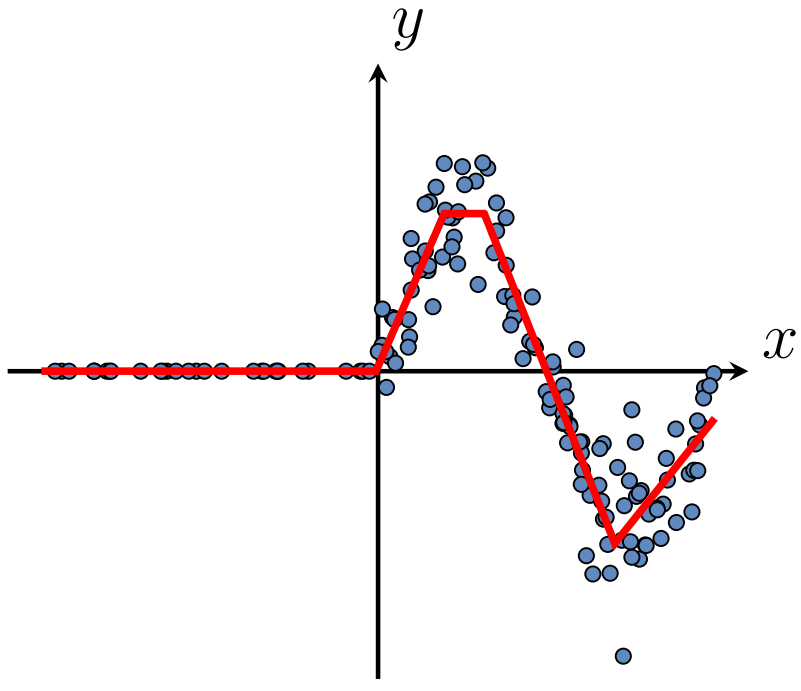
How about a MLP?

- Fit a 1-hidden layer MLP with h hidden nodes
- Training procedure:
  - He initialization
  - ReLu activation function
  - Optimizer: mini-batch GD with cyclic learning rates
  - batch size: 10

1-hidden layer MLP with no regularization

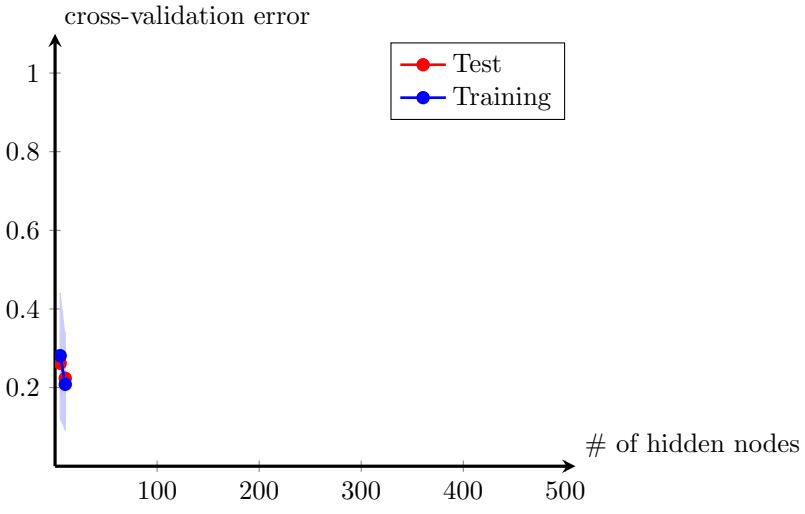


MSE scores

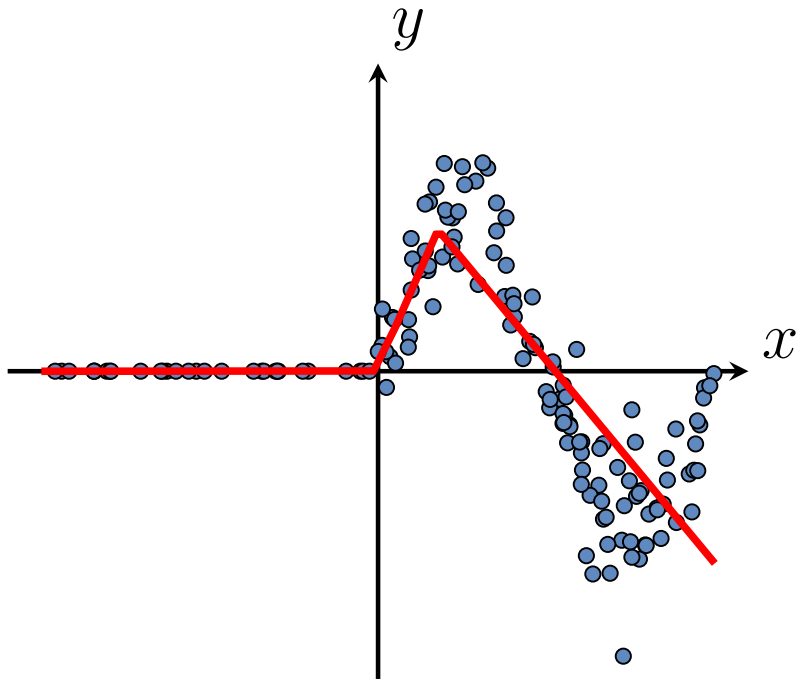


Predicted regression curve

1-hidden layer MLP with no regularization

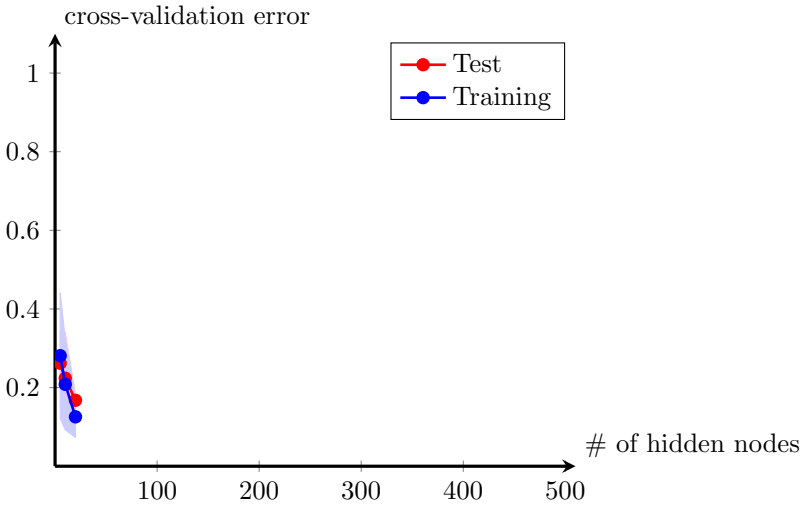


MSE scores

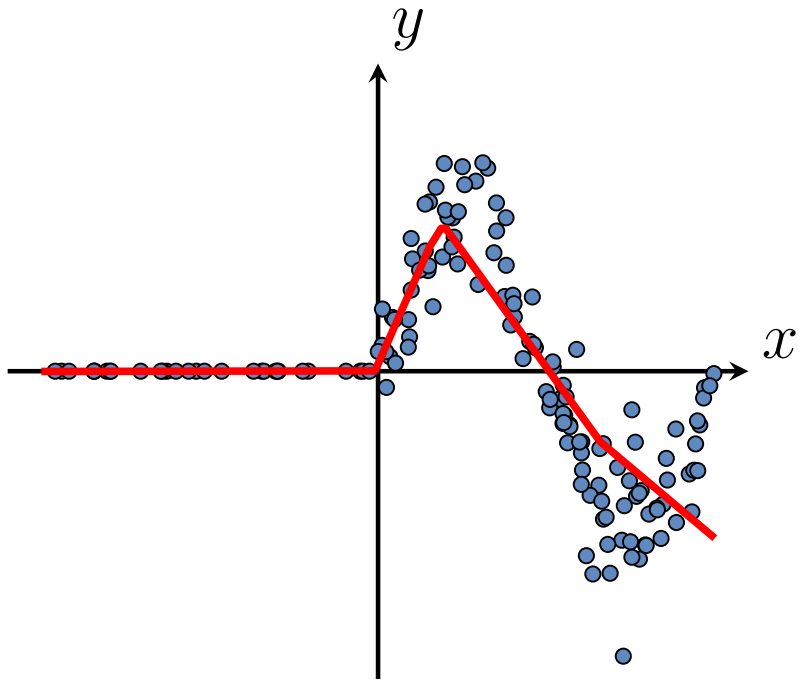


Predicted regression curve

1-hidden layer MLP with no regularization

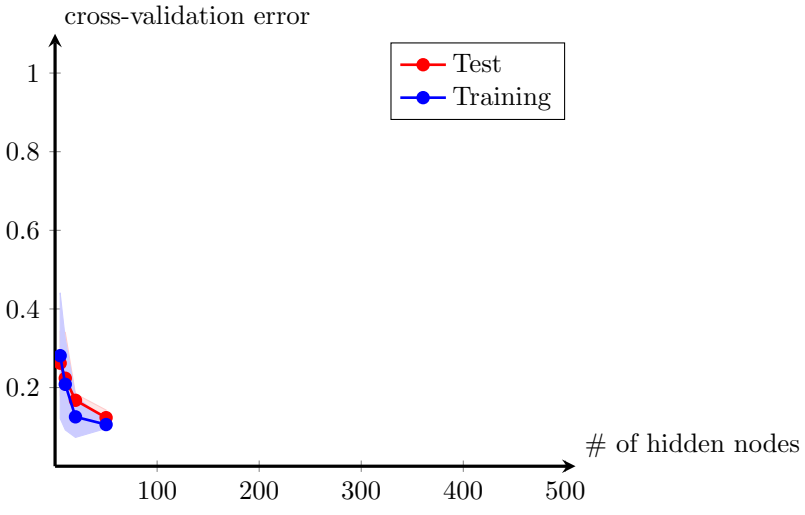


MSE scores

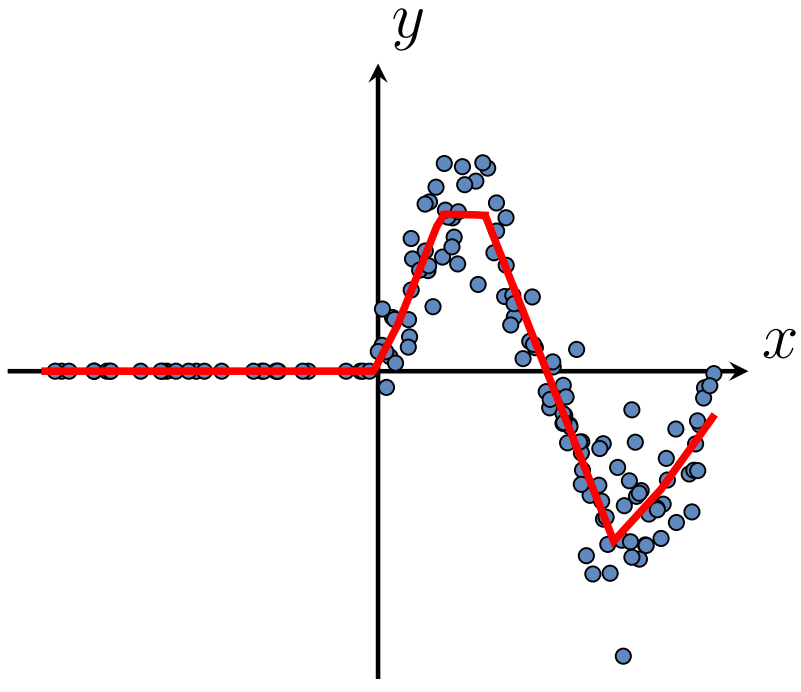


Predicted regression curve

1-hidden layer MLP with no regularization

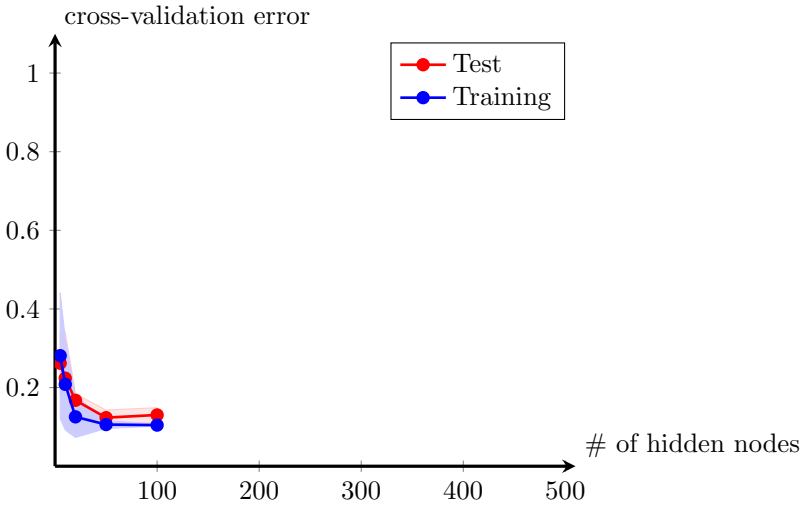


MSE scores

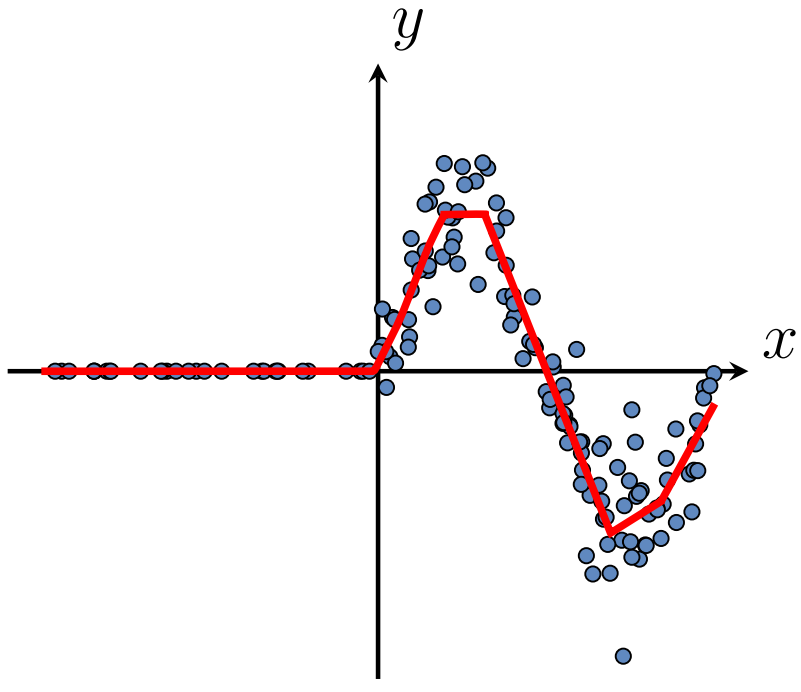


Predicted regression curve

1-hidden layer MLP with no regularization

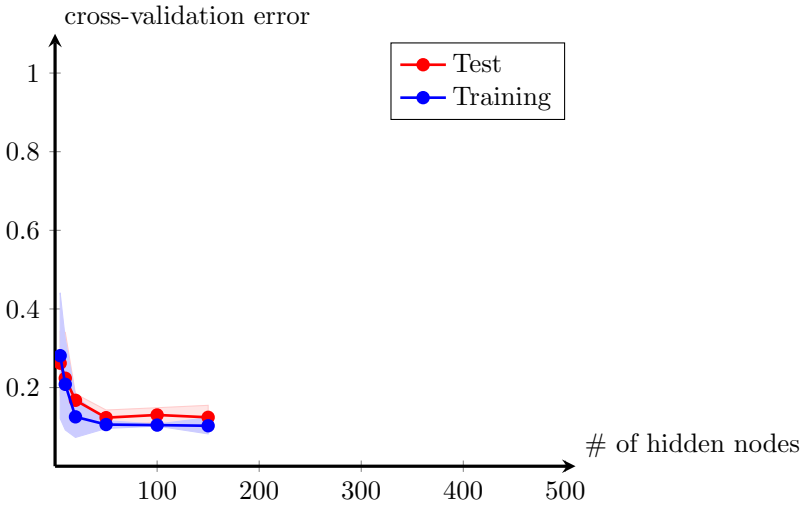


MSE scores

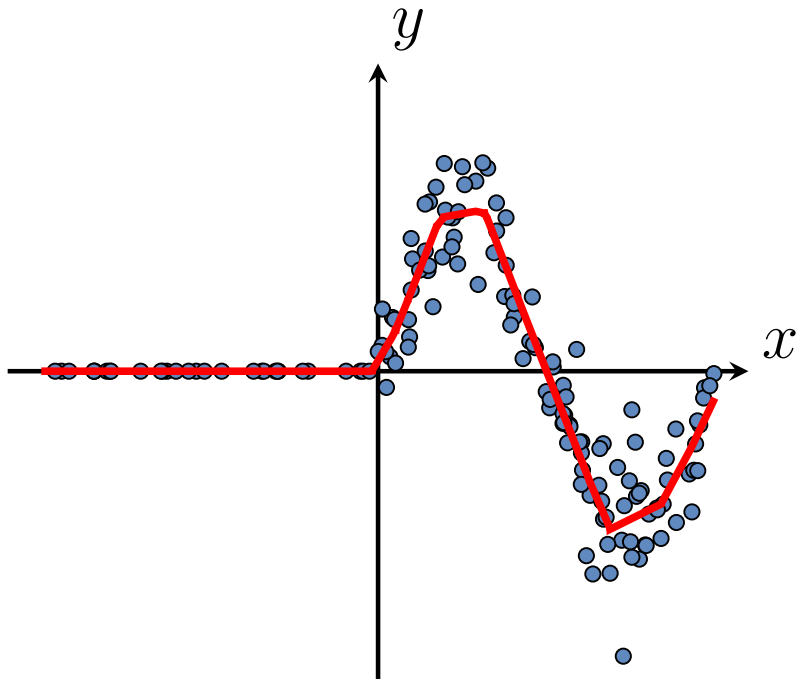


Predicted regression curve

1-hidden layer MLP with no regularization

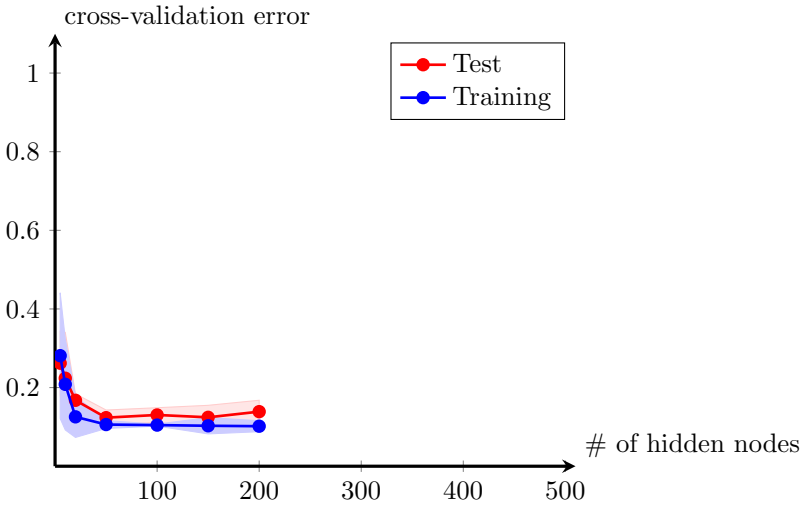


MSE scores

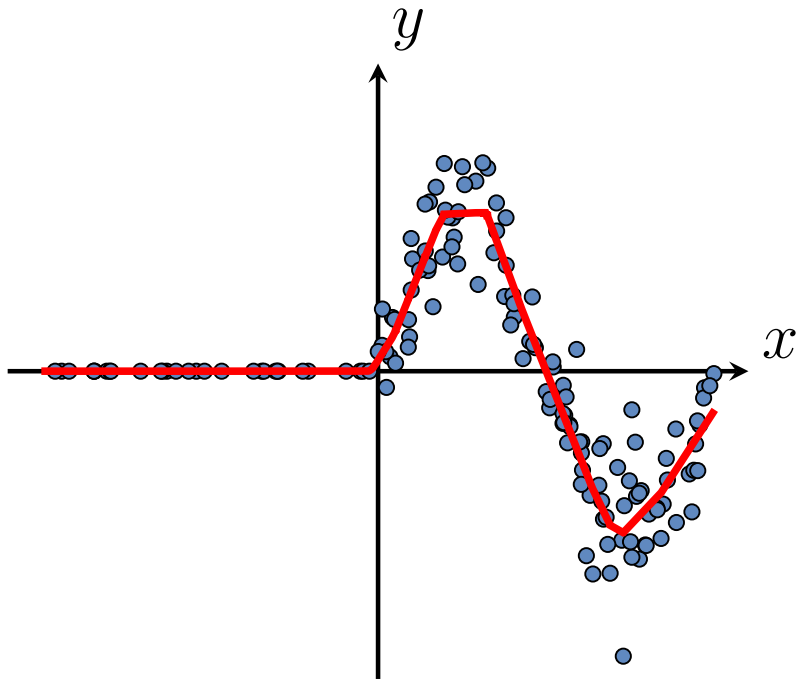


Predicted regression curve

1-hidden layer MLP with no regularization

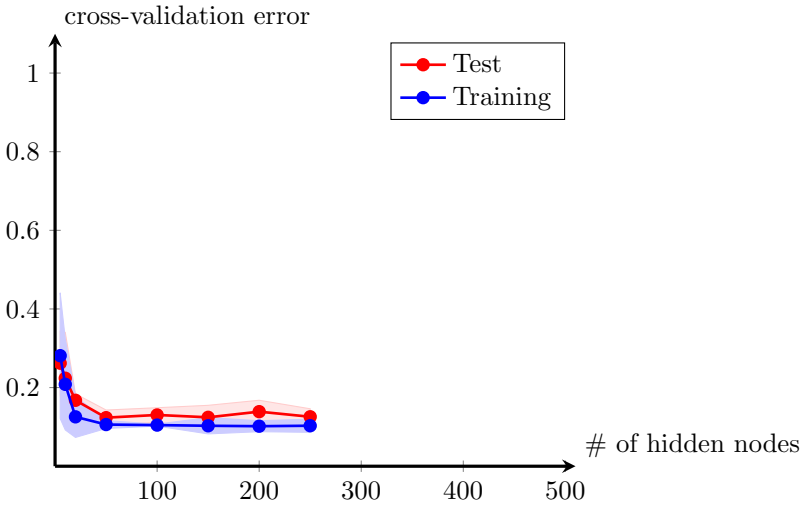


MSE scores

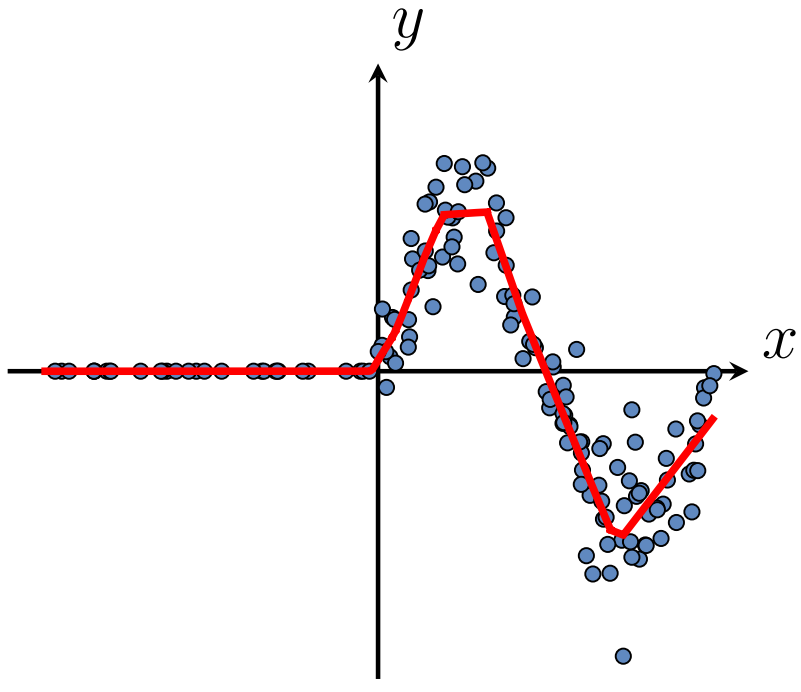


Predicted regression curve

1-hidden layer MLP with no regularization

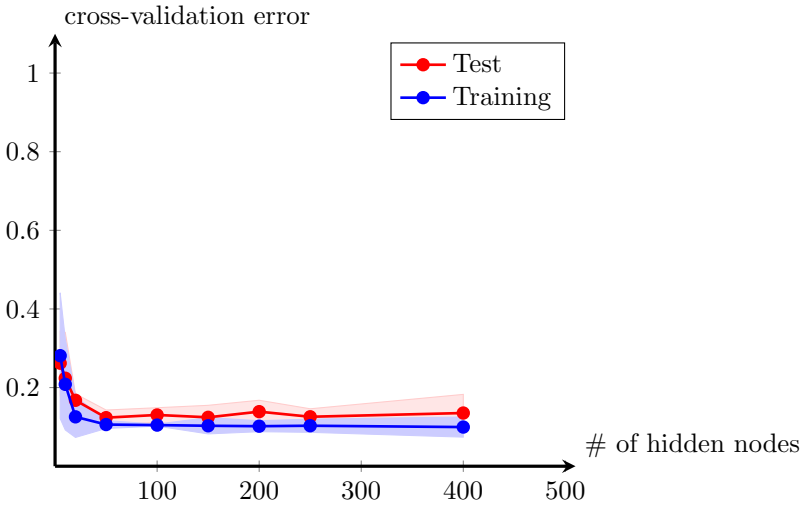


MSE scores

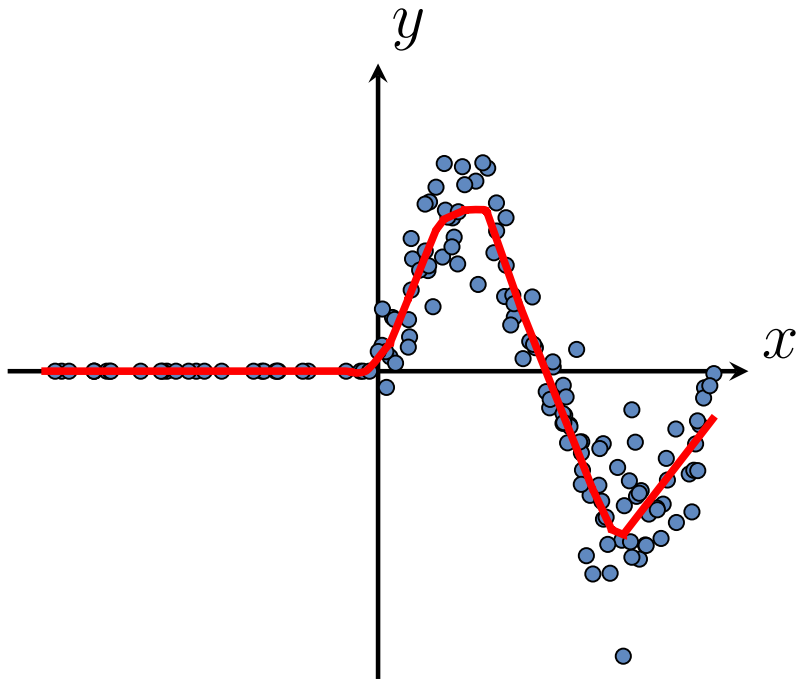


Predicted regression curve

1-hidden layer MLP with no regularization

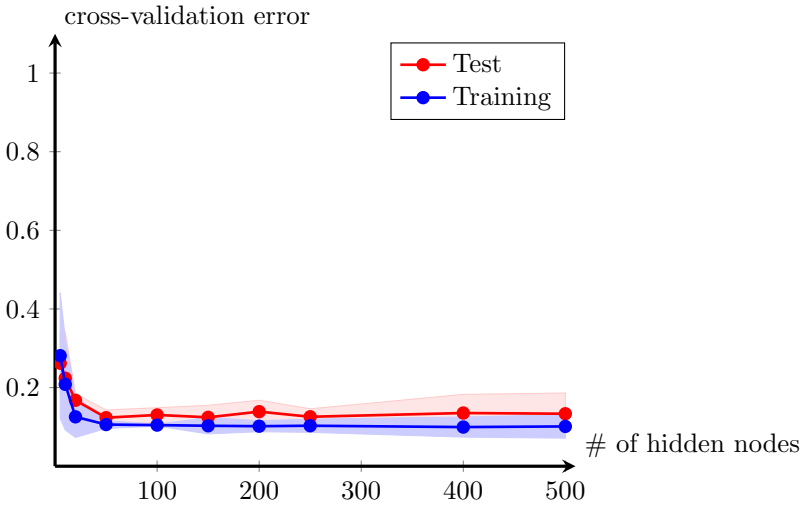


MSE scores

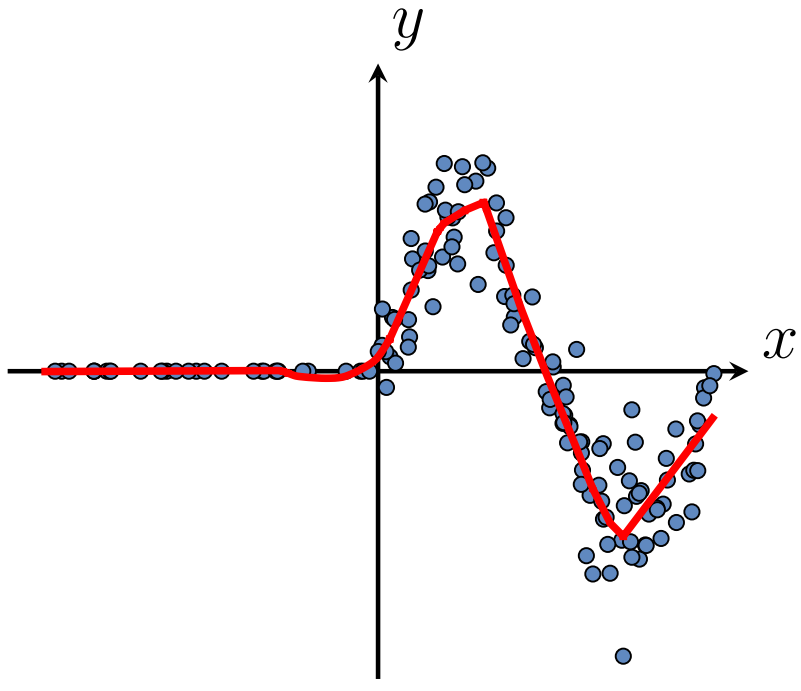


Predicted regression curve

1-hidden layer MLP with no regularization

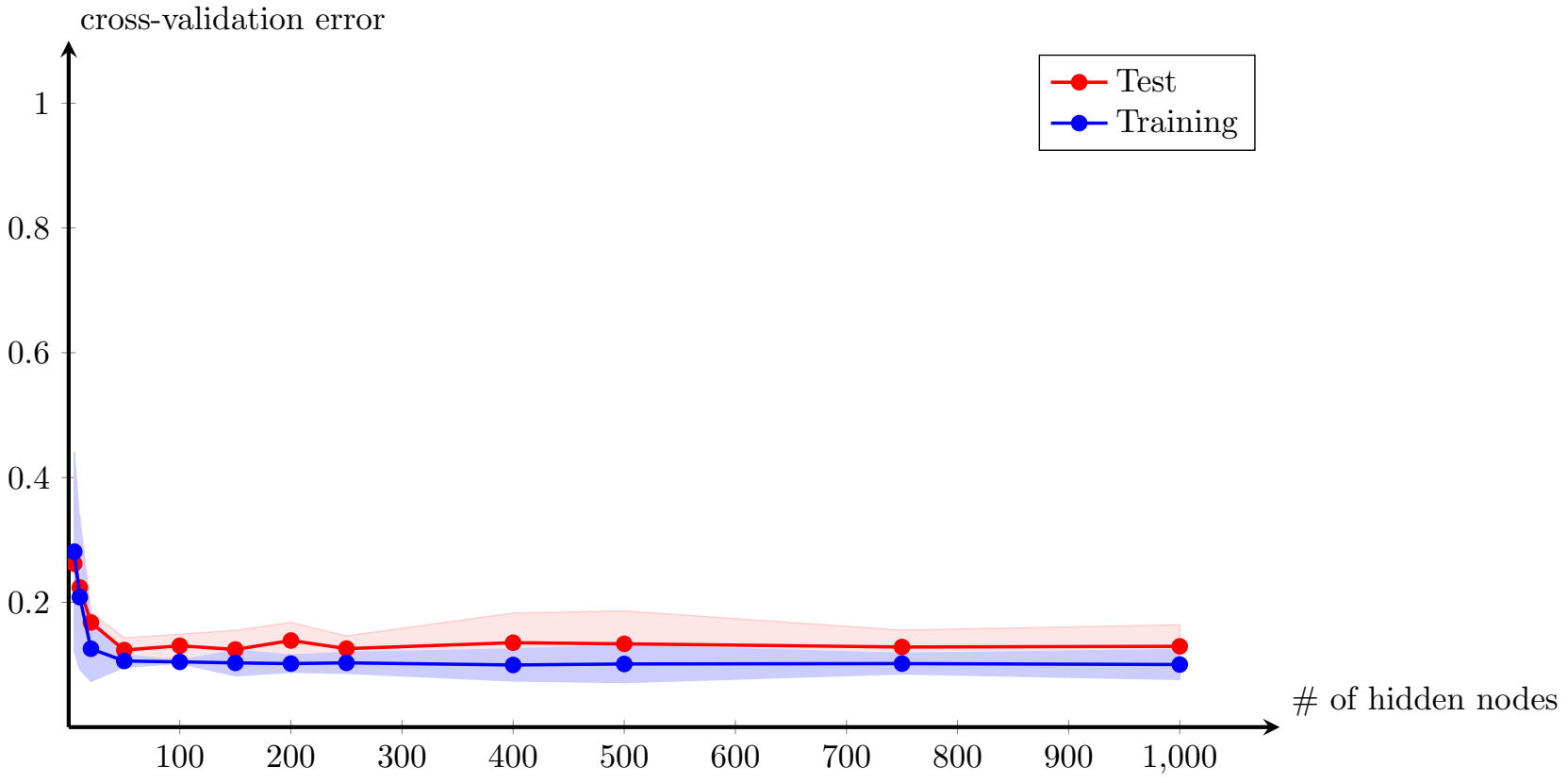


MSE scores



Predicted regression curve

1-hidden layer MLP with no regularization



Have way more parameters than data points and no over-fitting???

Non-intuitive behaviour of MLP

Adding hidden units:

- Does not lead to (more) overfitting
- Makes optimization easier
- Makes computation slower

Already known since the end of the 90s:
Overfitting in Neural Nets: Backpropagation, Conjugate Gradient, and Early Stopping,
NIPS 2001, Rich Caruana, Steve Lawrence, Lee Giles

Non-intuitive behaviour of MLP

Adding layers:

- Does not cause much more overfitting either
- Can (does) cause optimization issues

Already known since the end of the 90s:
Overfitting in Neural Nets: Backpropagation, Conjugate Gradient, and Early Stopping,
Rich Caruana, Steve Lawrence, Lee Giles, NIPS 2001,

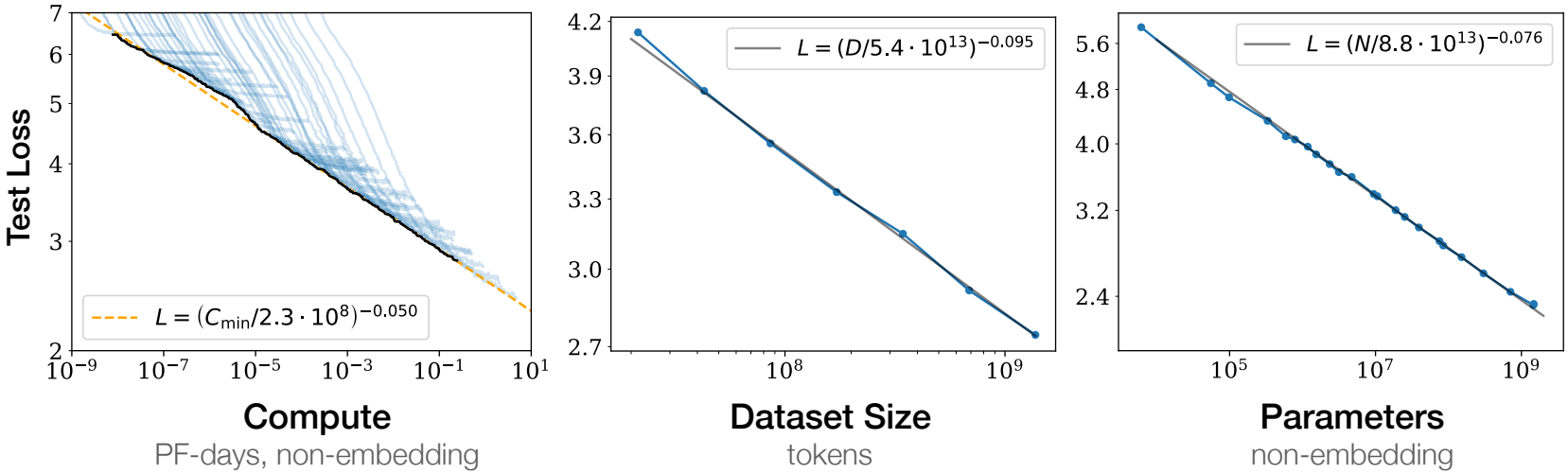
Some convincing empirical results at large scale:

Transformer networks trained as an auto-regressive predictor of language

Results and plots taken from:

Scaling Laws for Neural Language Models, J. Kaplan et al., arXiv, Jan 2020 (OpenAI)

Some interesting scaling laws for neural language machines

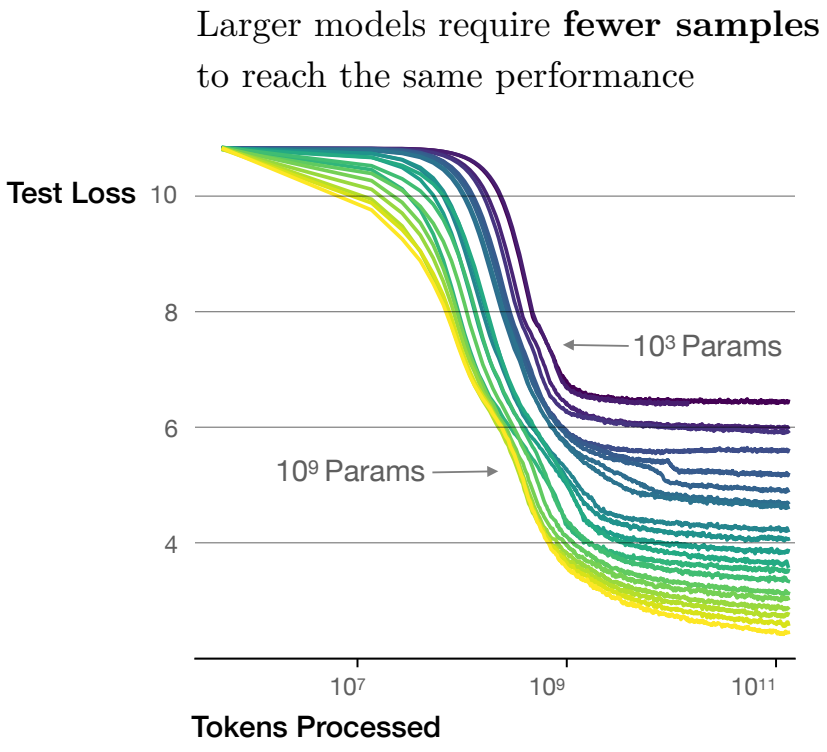


- Model performance depends strongly on
  - # of model parameters: N
  - size of dataset: D
  - amount of compute: C

  (shape of the network *depth* Vs *width* not as important).
- Performance has a power-law relationship with each of N, D and C
- Performance improves when N and D increased in tandem. Diminishing returns if either N or D fixed while the other increases.

PF-day $\equiv 10^{15} \times 24 \times 3600 = 8.64 \times 10^{19}$ floating point operations.

Larger models are more sample efficient



- Large models reach the same level of performance with fewer optimization steps and use fewer data points.

Some preliminaries about the Supervised Learning problem

Just so we know what we are talking about

- Have space of input-output pairs $(\mathbf{x}, \mathbf{y}) \in \mathcal{X} \times \mathcal{Y}$ with pdf $p_{\mathbf{X},\mathbf{Y}}(\mathbf{x}, \mathbf{y})$.

- $p_{\mathbf{Y}|\mathbf{X}}(\mathbf{y} \mid \mathbf{x})$ represents (unknown) relationship between inputs and outputs.

- **Goal**:
  Learn a prediction function f approximating this relationship

Just so we know what we are talking about

- **Loss function**:
  $l : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}^+$ measures discrepancy between predicted output $\hat{\mathbf{y}}$ and ground truth output $\mathbf{y}$.

- **Expected Risk** for a function $f : \mathcal{X} \to \mathcal{Y}$:

$$R(f) = \mathsf{E}[l(f(\mathbf{X}), \mathbf{Y})] = \int_{(\mathbf{x},\mathbf{y}) \in \mathcal{X} \times \mathcal{Y}} l(f(\mathbf{x}), \mathbf{y})\, p_{\mathbf{X},\mathbf{Y}}(\mathbf{x}, \mathbf{y})\, d\mathbf{x}\, d\mathbf{y}$$

- **Benchmark** function is $f^\star$:

$$f^\star(\mathbf{x}) = \arg\min_{\hat{\mathbf{y}}} \mathsf{E}[\, l(\hat{\mathbf{y}}, \mathbf{Y}) \mid \mathbf{X} = \mathbf{x}]$$

Just so we know what we are talking about

- $p_{\mathbf{X},\mathbf{Y}}(\mathbf{x},\mathbf{y})$ unknown but have samples $\mathcal{S} = \{(\mathbf{x}_i,\mathbf{y}_i)\}_{i=1}^n$

- **Empirical Risk** for a function $f : \mathcal{X} \to \mathcal{Y}$ is:

$$R_n(f) = \frac{1}{n}\sum_{i=1}^{n} l(f(\mathbf{x}_i),\mathbf{y}_i)$$

- **High level learning problem**
  - Choose a family $\mathcal{F}$ of candidate prediction functions.
  - Then find $f_n \in \mathcal{F}$ s.t.

$$f_n = \arg\min_{f\in\mathcal{F}} R_n(f)$$

What factors contribute to $f_n \neq f^\star$

- Define the **excess error** as

$$E\left[R(f_n) - R(f^\star)\right]$$

where expectation is taken w.r.t. the random choice of training set.

- In general $f^\star \notin \mathcal{F}$ so define

$$f^\star_{\mathcal{F}} = \arg\min_{f \in \mathcal{F}} R(f)$$

- Initial decomposition of the error:

$$E\left[R(f_n) - R(f^\star)\right] = \underbrace{E\left[R(f^\star_{\mathcal{F}}) - R(f^\star)\right]}_{\text{Approximation error}} + \underbrace{E\left[R(f_n) - R(f^\star_{\mathcal{F}})\right]}_{\text{Estimation error}}$$

What factors contribute to $f_n \neq f^\star$

Initial decomposition of the excess error:

$$E\left[R(f_n) - R(f^\star)\right] = \underbrace{E\left[R(f_{\mathcal{F}}^\star) - R(f^\star)\right]}_{\text{Approximation error}} + \underbrace{E\left[R(f_n) - R(f_{\mathcal{F}}^\star)\right]}_{\text{Estimation error}}$$

- **Approximation error**:
  Measures how closely functions in $\mathcal{F}$ can approximate $f^\star$.

- **Estimation error**:
  Measures the effect of minimizing the **empirical risk** as opposed to the **expected risk**.

What factors contribute to $f_n \neq f^\star$

Initial decomposition of the excess error:

$$E\left[R(f_n) - R(f^\star)\right] = \underbrace{E\left[R(f^\star_{\mathcal{F}}) - R(f^\star)\right]}_{\text{Approximation error}} + \underbrace{E\left[R(f_n) - R(f^\star_{\mathcal{F}})\right]}_{\text{Estimation error}}$$

- **Approximation error**:
  Measures how closely functions in $\mathcal{F}$ can approximate $f^\star$.

- **Estimation error**:
  Measures the effect of minimizing the **empirical risk** as opposed to the **expected risk**.
  It depends on
  1. number of training examples n
  2. capacity of the $f \in \mathcal{F}$

What factors contribute to $f_n \neq f^\star$

Initial decomposition of the excess error:

$$E\left[R(f_n) - R(f^\star)\right] = \underbrace{E\left[R(f_{\mathcal{F}}^\star) - R(f^\star)\right]}_{\text{Approximation error}} + \underbrace{E\left[R(f_n) - R(f_{\mathcal{F}}^\star)\right]}_{\text{Estimation error}}$$

- **Approximation error**:
  Measures how closely functions in $\mathcal{F}$ can approximate $f^\star$.

- **Estimation error**:
  Measures the effect of minimizing the **empirical risk** as opposed to the **expected risk**.

$\mathcal{F}$'s with **high capacity functions** generally $\Longrightarrow$
small approximation errors but *high estimation errors*

Optimization Error

- Exactly computing

$$\arg\min_{f \in \mathcal{F}} R_n(f)$$

  can be very costly.... or even untractable....

- In practice optimizer returns approximate solution $\tilde{f}_n$ up to some tolerance $\rho \geq 0$

$$R_n(\tilde{f}_n) < R_n(f_n) + \rho$$

- Decompose error $E\left[R(\tilde{f}_n) - R(f^\star)\right]$ as

$$\underbrace{E\left[R(f^\star_{\mathcal{F}}) - R(f^\star)\right]}_{\text{Approximation error}} + \underbrace{E\left[R(f_n) - R(f^\star_{\mathcal{F}})\right]}_{\text{Estimation error}} + \underbrace{E\left[R(\tilde{f}_n) - R(f_n)\right]}_{\text{Optimization error}}$$

- Optimization error reflects impact of approximate optimization on **generalization performance**.

Decomposition of the Error

- **Approximation error**:
  - decreases when $\mathcal{F}$ increases;
  - but typically bounded by computational constraints.
- **Estimation error**:
  - decreases when n increases;
  - can increase when $\mathcal{F}$ increases (according to VC theory).
- **Optimization error**:
  - can increase when tolerance ρ increases;
  - can increase when $\mathcal{F}$ becomes more complex.

Boils down to...

When doing supervised learning

- Have constraints on
  - maximum # of training examples and
  - computational resources
- Given these constraints
  have to make decisions about $\mathcal{F}$ and ρ to trade-off these errors:

  Approximation - *Estimation* - *Optimization*
- True for training of neural networks, however,....

Boils down to...

When doing supervised learning

- Have constraints on
  - maximum # of training examples and
  - computational resources

- Given these constraints
  have to make decisions about $\mathcal{F}$ and ρ to trade-off these errors:

  Approximation - *Estimation* - *Optimization*

- True for training of neural networks, however,....

  There are many things we don't understand.

Expressivity:

What type of functions can neural networks approximate?

Universal Function Approximation

- Let σ be a non-constant, bounded, and monotonically-increasing continuous function.
- For any $f \in C([0,1]^d)$ and $\epsilon > 0$, there exists $h \in \mathbb{N}$, and constants $v_i, b_i \in \mathbb{R}$ and real vectors $\mathbf{w}_i \in \mathbb{R}^d$ for $i = 1, \ldots, h$ s.t.

$$\left| \sum_{i=1}^{h} v_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i) - f(\mathbf{x}) \right| < \epsilon$$

- That is: neural networks are dense in $C([0,1]^d)$

This still holds for any compact subset of $\mathbb{R}^d$ and if σ is the ReLU function (Sonoda & Murata, 2015).

Approximation Capabilities of Multilayer Feedforward Networks, K. Hornik, 1991

Problem solved?

The UFA theorems do NOT tell us:

- The number h of hidden units is small enough to have the network fit in RAM.
- The network's optimal parameters can be found in finite time by
  - minimizing the Empirical Risk with SGD and
  - the usual random initialization schemes.

Approximation with ReLU nets

- Denote ReLU with

$$[x]_+ = \max(x, 0)$$

- Define function $r : \mathbb{R} \times \mathbb{R}^3 \to \mathbb{R}$ (with $\epsilon = 1e - 7$)

$$\begin{aligned} r(x; a, b, c) = \frac{c}{\epsilon} \big(& [x - a]_+ \\ & - [x - (a + \epsilon)]_+ \\ & - [x - b]_+ \\ & + [x - (b + \epsilon)]_+ \big) \end{aligned}$$

- Let $a = 0, b = 1, c = 1.3$.

- Sketch $r(x; 0, 1, 1.3)$ for $x \in [-3, 3]$.

Quora: Is a single layered ReLU network still a universal approximator?, Conner Davis

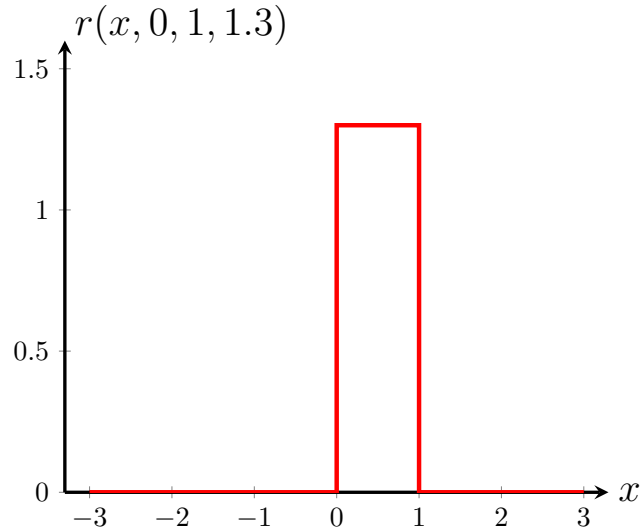
Approximation with ReLU nets

- Denote ReLU with

$$[x]_+ = \max(x, 0)$$

- Define function $r : \mathbb{R} \times \mathbb{R}^3 \to \mathbb{R}$ (with $\epsilon = 1e - 7$)

$$r(x; a, b, c) = \frac{c}{\epsilon} \left([x - a]_+ - [x - (a + \epsilon)]_+ - [x - b]_+ + [x - (b + \epsilon)]_+ \right)$$



- Let $a = 0, b = 1, c = 1.3$.

- Sketch $r(x; 0, 1, 1.3)$ for $x \in [-3, 3]$.

Quora: Is a single layered ReLU network still a universal approximator?, Conner Davis

Approximation with ReLU nets

- Denote ReLU with

$$[x]_+ = \max(x, 0)$$

- Define function $r : \mathbb{R} \times \mathbb{R}^3 \to \mathbb{R}$ (with $\epsilon = 1e - 7$)

$$r(x; a, b, c) = \frac{c}{\epsilon} \left([x - a]_+ - [x - (a + \epsilon)]_+ - [x - b]_+ + [x - (b + \epsilon)]_+ \right)$$

- Sketch for $x \in [-3, 3]$:

$$r(x; -1, 0, 0.4) + r(x; 0, 1, 1.3) + r(x; 1, 2, 0.8)$$

Quora: Is a single layered ReLU network still a universal approximator?, Conner Davis

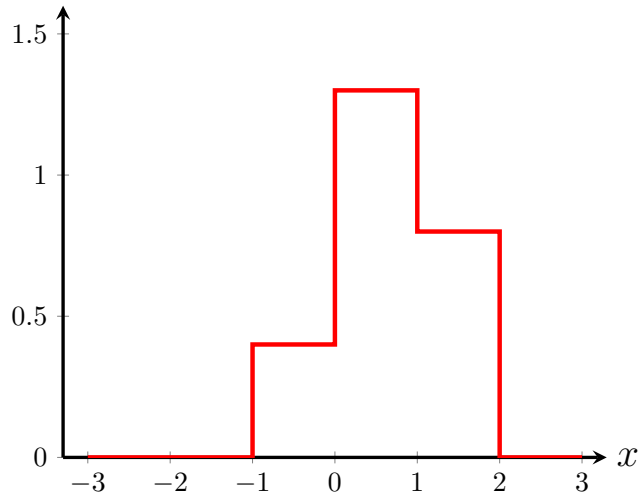
Approximation with ReLU nets

- Denote ReLU with

$$[x]_+ = \max(x, 0)$$

- Define function $r : \mathbb{R} \times \mathbb{R}^3 \to \mathbb{R}$ (with $\epsilon = 1e - 7$)

$$r(x; a, b, c) = \frac{c}{\epsilon} \left([x - a]_+ - [x - (a + \epsilon)]_+ - [x - b]_+ + [x - (b + \epsilon)]_+ \right)$$



- Sketch for $x \in [-3, 3]$:

$$r(x; -1, 0, 0.4) + r(x; 0, 1, 1.3) + r(x; 1, 2, 0.8)$$

Quora: Is a single layered ReLU network still a universal approximator?, Conner Davis

Properties of functions generated by ReLU networks

Previous Example:

- Function r corresponds to the 1-hidden layer network:

$$W_2 \left[W_1 x + \mathbf{b}_1 \right]_+$$

with

$$W_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, \quad \mathbf{b}_1 = - \begin{pmatrix} a \\ a + \epsilon \\ b \\ b + \epsilon \end{pmatrix}$$

$$W_2 = \frac{c}{\epsilon} \begin{pmatrix} 1 & -1 & -1 & 1 \end{pmatrix}$$

- Resulting 1d function is continuous piecewise linear.

In general

- Do ReLU networks define continuous piecewise linear functions?
- If yes, how are the linear regions defined?

Consider a simple 1-hidden layer network

A small 1-hidden layer network

- input dim: $d = 2$, # of hidden nodes: $h = 2$, output dim: $m = 2$

- Our neural network function f has the form

$$\mathbf{x}_1 = \max(W_1\mathbf{x} + \mathbf{b}_1, 0)$$
$$\mathbf{y} = W_2\mathbf{x} + \mathbf{b}_2$$

  where

  - $\mathbf{x} \in [-1, 1]^2$,
  - $\mathbf{x}_1, \mathbf{y} \in \mathbb{R}^2$,
  - $W_1, W_2 \in \mathbb{R}^{2\times 2}$ and $\mathbf{b}_1, \mathbf{b}_2 \in \mathbb{R}^2$

- $f(\mathbf{x}) = \mathbf{y}$ defines a classification function $g : \mathbb{R}^2 \to \{1, 2\}$ as

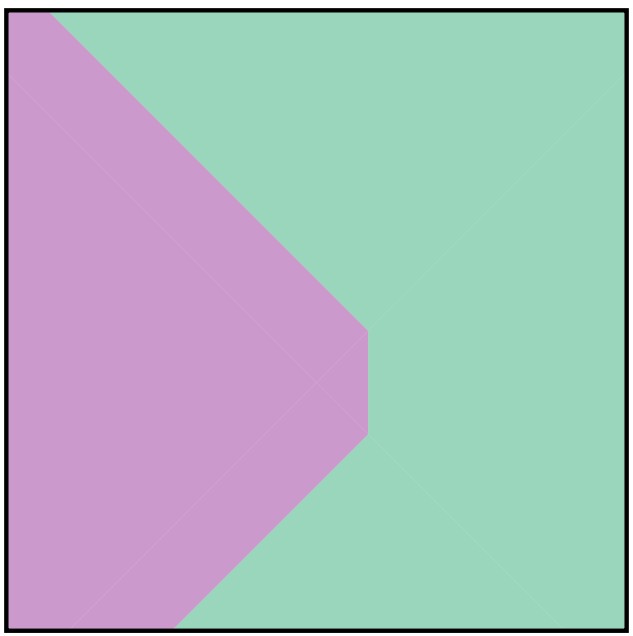
$$g(\mathbf{x}) = \begin{cases} 1 & \text{if } y_1 \geq y_2 \\ 2 & \text{if } y_1 < y_2 \end{cases}$$

Example: 1-hidden layer network

- Consider this parameter setting.

$$W_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \qquad \mathbf{b}_1 = \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} -0.1 & -0.5 \\ 0.5 & 0.1 \end{pmatrix}, \qquad \mathbf{b}_2 = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$$



Decision regions on $[-1, 1]^2$ defined by this network

$y_1 \geq y_2$, $y_1 < y_2$

What happens when we apply Layer 1

- Apply Layer 1 affine function:

$$\mathbf{v} = W_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix} = \begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$$

First layer defines hyper-planes in input space

- Apply Layer 1 affine function:

$$\mathbf{v} = W_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix} = \begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$$

- Focus on row 1 of $\mathbf{v}$: $\begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$

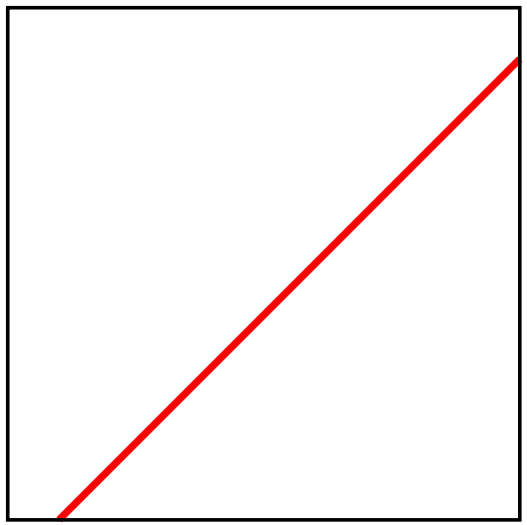
First layer defines hyper-planes in input space

- Apply Layer 1 affine function:

$$\mathbf{v} = W_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix} = \begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$$

- Focus on row 1 of $\mathbf{v}$: $\begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$

- Setting $v_x = 0$ defines hyperplane in the input space $[-1, 1]^2$:



$$x - y - 0.2 = 0$$

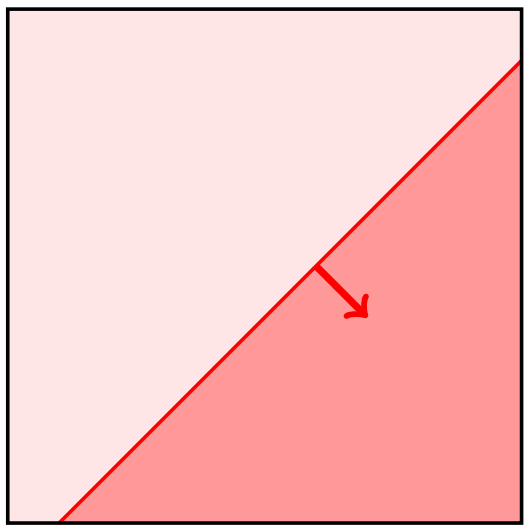
First layer defines hyper-planes in input space

- Apply Layer 1 affine function:

$$\mathbf{v} = W_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix} = \begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$$

- Focus on row 1 of $\mathbf{v}$: $\begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$

- This hyperplane defines two halfspaces $v_x \geq 0$ and $v_x < 0$:



$x - y - 0.2 \geq 0, \quad x - y - 0.2 < 0$

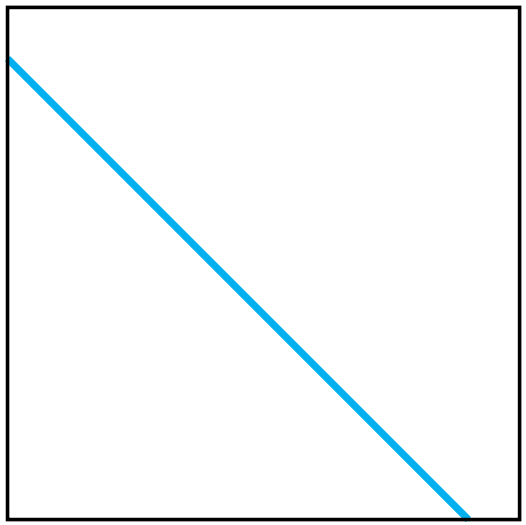
First layer defines hyper-planes in input space

- Apply Layer 1 affine function:

$$\mathbf{v} = W_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix} = \begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$$

- Focus on row 2 of $\mathbf{v}$: $\begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$

- Setting $v_y = 0$ defines hyperplane in original input space $[-1, 1]^2$:



$x + y + 0.2 = 0$

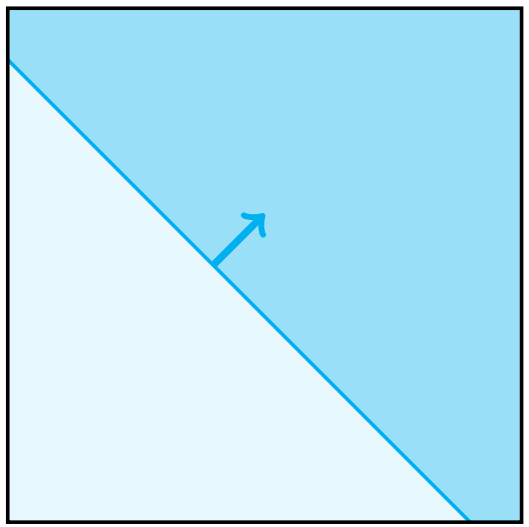
First layer defines hyper-planes in input space

- Apply Layer 1 affine function:

$$\mathbf{v} = W_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix} = \begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$$

- Focus on row 2 of $\mathbf{v}$: $\begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$

- This hyperplane defines two halfspaces $v_y \geq 0$ and $v_y < 0$:



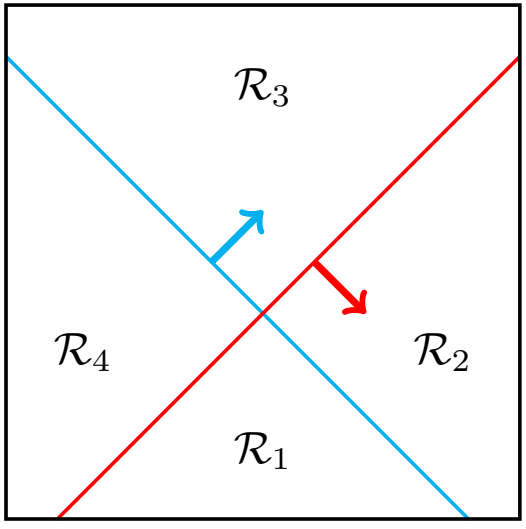
$x + y + 0.2 \geq 0, \quad x + y + 0.2 < 0$

Layer 1 hyperplanes partition input space

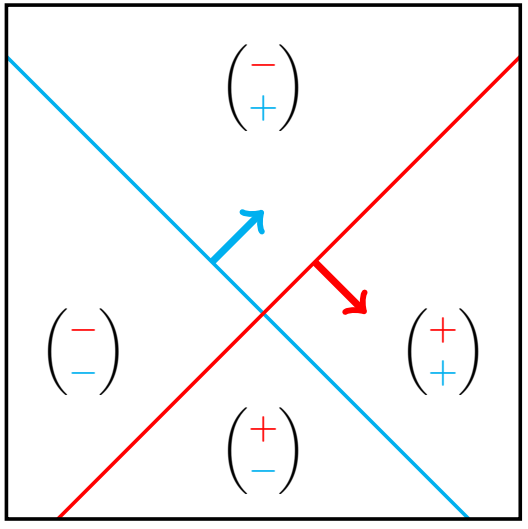
- Apply Layer 1 affine function:

$$\mathbf{v} = W_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix} = \begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$$

- Hyperplanes $v_x = 0$, $v_y = 0$ split input space $[-1, 1]^2$ into 4 regions:



4 regions



Each region: Qualitative values of $\mathbf{v}$

Layer 1 hyperplanes partition input space

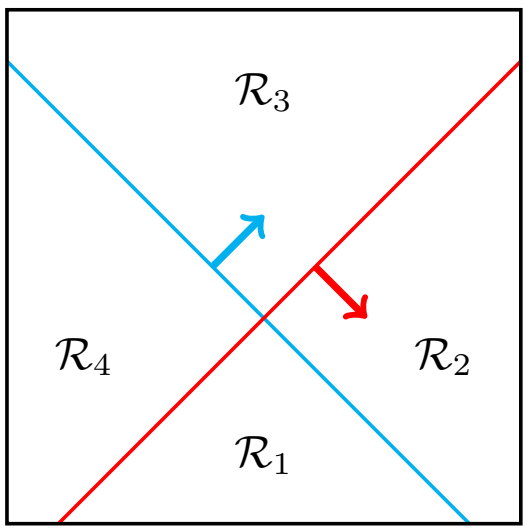
- Apply Layer 1 affine function:

$$\mathbf{v} = W_1\mathbf{x} + \mathbf{b}_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} -0.2 \\ 0.2 \end{pmatrix} = \begin{pmatrix} x - y - 0.2 \\ x + y + 0.2 \end{pmatrix}$$

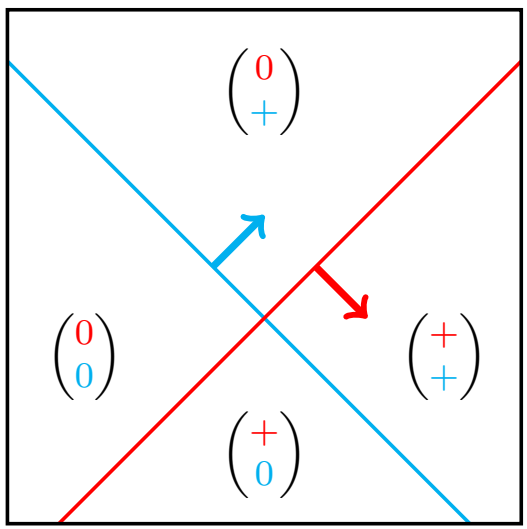
- Apply Layer 1 ReLu activation function:

$$\mathbf{x}_1 = \max(\mathbf{v}, 0) = \begin{pmatrix} \max(x - y - 0.2, 0) \\ \max(x + y + 0.2, 0) \end{pmatrix}$$

- $\mathcal{R}_i$ defined by indices of non-negative entries after the first ReLu:

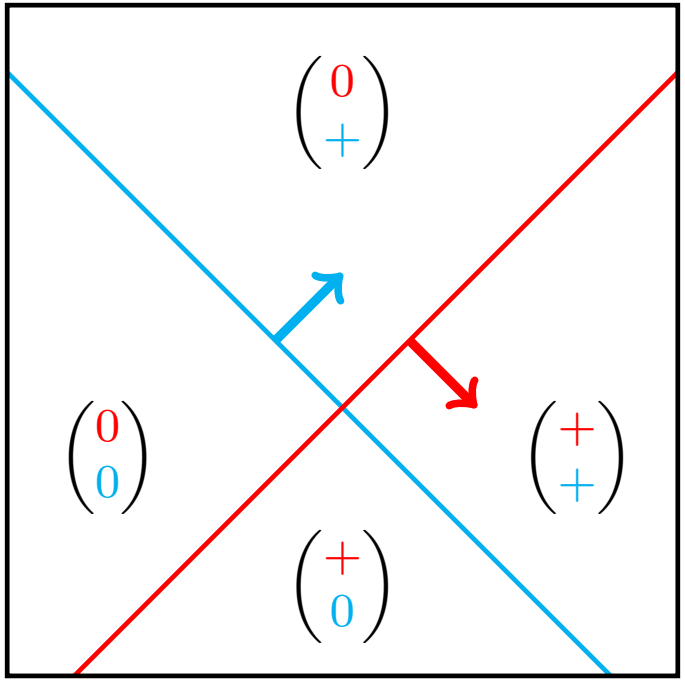


4 regions



Each region: Qualitative values of $\mathbf{x}_1$

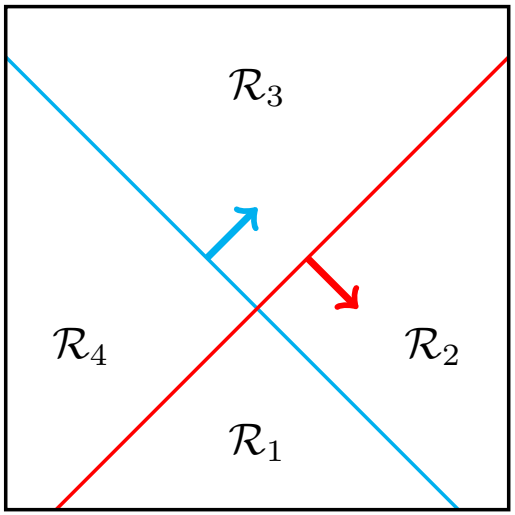
Activity Patterns



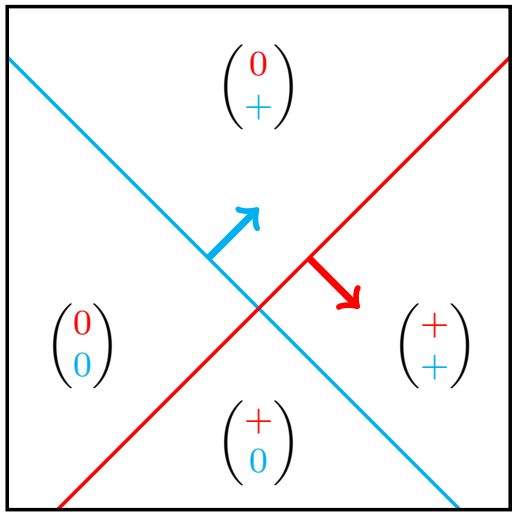
Activity Patterns of Layer 1

- Each column vector represents **Activity Pattern** of Layer 1.

Important observation



4 regions



Activity Patterns of Layer 1

- Each $\mathcal{R}_i$ is a **convex polytope** (assuming bounded inputs).

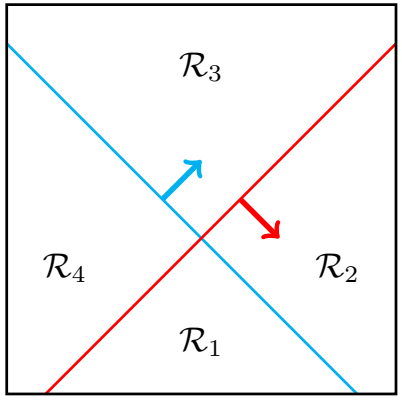
- **Example**: $\mathcal{R}_1$ is defined as (using Matlab notation)

$$W_1(1,:)\,\mathbf{x} + \mathbf{b}_1(1) > 0$$
$$W_1(2,:)\,\mathbf{x} + \mathbf{b}_1(2) \leq 0$$
$$-1 \leq x \leq 1$$
$$-1 \leq y \leq 1$$

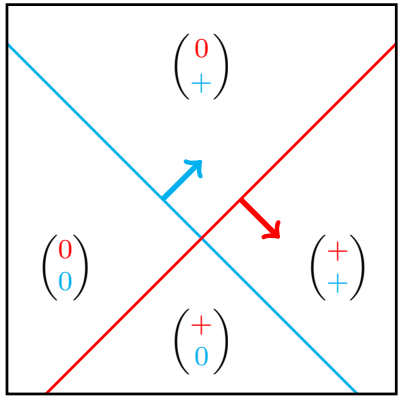
Apply Layer 2

- Apply 2nd layer affine transformation:

$$\mathbf{y} = W_2\mathbf{x}_1 + \mathbf{b}_2 = \begin{pmatrix} -0.1 & -0.5 \\ 0.5 & 0.1 \end{pmatrix} \mathbf{x}_1 + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$$



4 regions

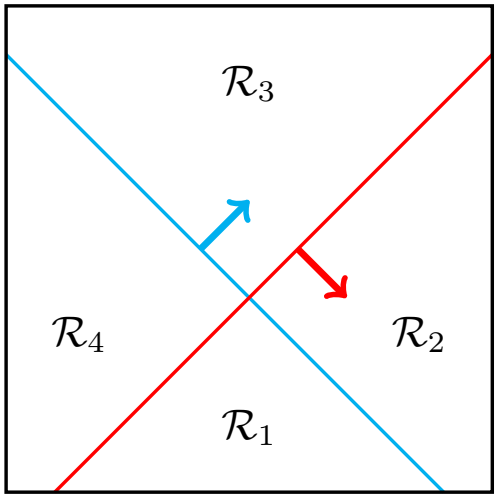


Activity Patterns of Layer 1

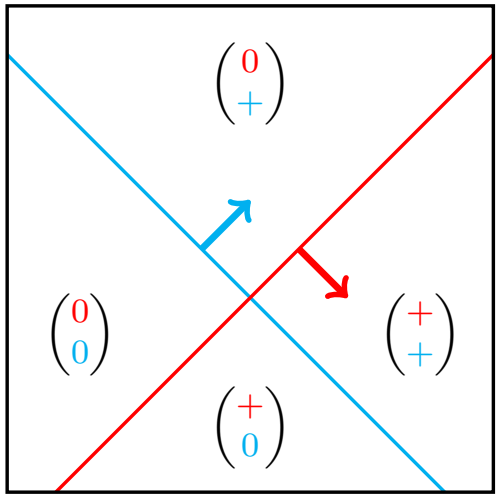
- For $\mathbf{x} \in \mathcal{R}_1$:

$$\mathbf{y} = \begin{pmatrix} -0.1 & -0.5 \\ 0.5 & 0.1 \end{pmatrix} \begin{pmatrix} x_1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} -0.1 \\ 0.5 \end{pmatrix} x_1 + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$$

Example: Network function is affine in each region



4 regions



Activity Patterns of Layer 1

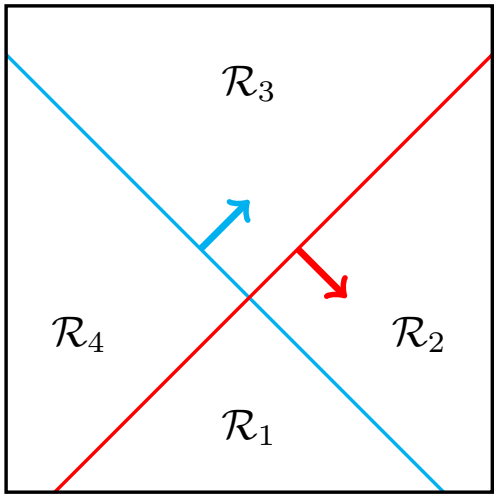
- Apply 2nd layer affine transformation: $\mathbf{y} = W_2\mathbf{x}_1 + \mathbf{b}_2$
- Thus $f(\mathbf{x})$, for $\mathbf{x} \in \mathcal{R}_1$, is the affine function:

$$W_2 \begin{pmatrix}+\\0\end{pmatrix} + \mathbf{b}_2 = W_2(:,1)\,\mathbf{x}_1(1) + \mathbf{b}_2 \quad \leftarrow \text{using Matlab notation}$$

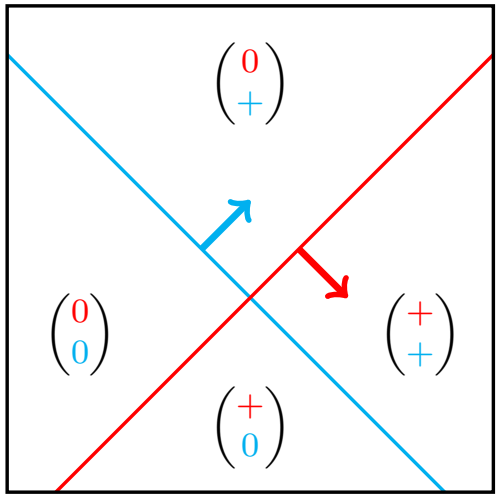
$$= W_2(:,1)\,\left(W_1(1,:)\,\mathbf{x} + \mathbf{b}_1(1)\right) + \mathbf{b}_2$$

$$= \underbrace{W_2(:,1)\,W_1(1,:)}_{W_{\mathcal{R}_1}}\mathbf{x} + \underbrace{W_2(:,1)\,\mathbf{b}_1(1) + \mathbf{b}_2}_{\mathbf{b}_{\mathcal{R}_1}} = W_{\mathcal{R}_1}\,\mathbf{x} + \mathbf{b}_{\mathcal{R}_1}$$

Example: Network function is affine in each region



4 regions



Activity Patterns of Layer 1

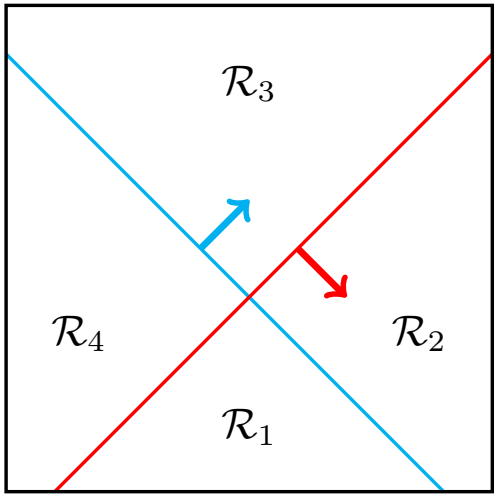
- Apply 2nd layer affine transformation: $\mathbf{y} = W_2\mathbf{x}_1 + \mathbf{b}_2$
- Similarly $f(\mathbf{x})$ for $\mathbf{x} \in \mathcal{R}_3$ is the affine function:

$$W_2 \begin{pmatrix}0\\+\end{pmatrix} + \mathbf{b}_2 = W_2(:,2)\,\mathbf{x}_1(2) + \mathbf{b}_2 \quad \leftarrow \text{using Matlab notation}$$

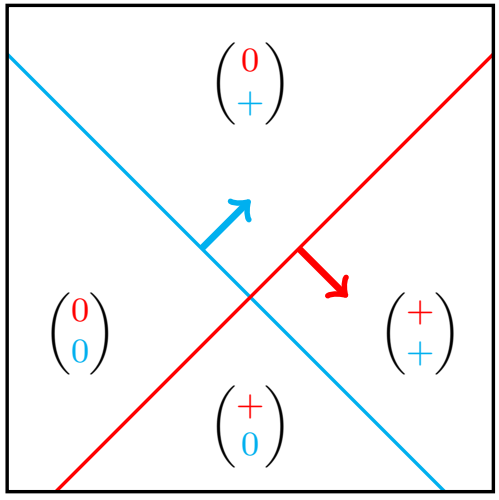
$$= W_2(:,2)\,\left(W_1(2,:)\,\mathbf{x} + \mathbf{b}_1(2)\right) + \mathbf{b}_2$$

$$= \underbrace{W_2(:,2)\,W_1(2,:)}_{W_{\mathcal{R}_3}}\mathbf{x} + \underbrace{W_2(:,2)\,\mathbf{b}_1(2) + \mathbf{b}_2}_{\mathbf{b}_{\mathcal{R}_3}} = W_{\mathcal{R}_3}\,\mathbf{x} + \mathbf{b}_{\mathcal{R}_3}$$

Example: Network function is affine in each region



4 regions



Activity Patterns of Layer 1

- Apply 2nd layer affine transformation: $\mathbf{y} = W_2\mathbf{x}_1 + \mathbf{b}_2$

- For $\mathbf{x} \in \mathcal{R}_i$ have affine function from input to output

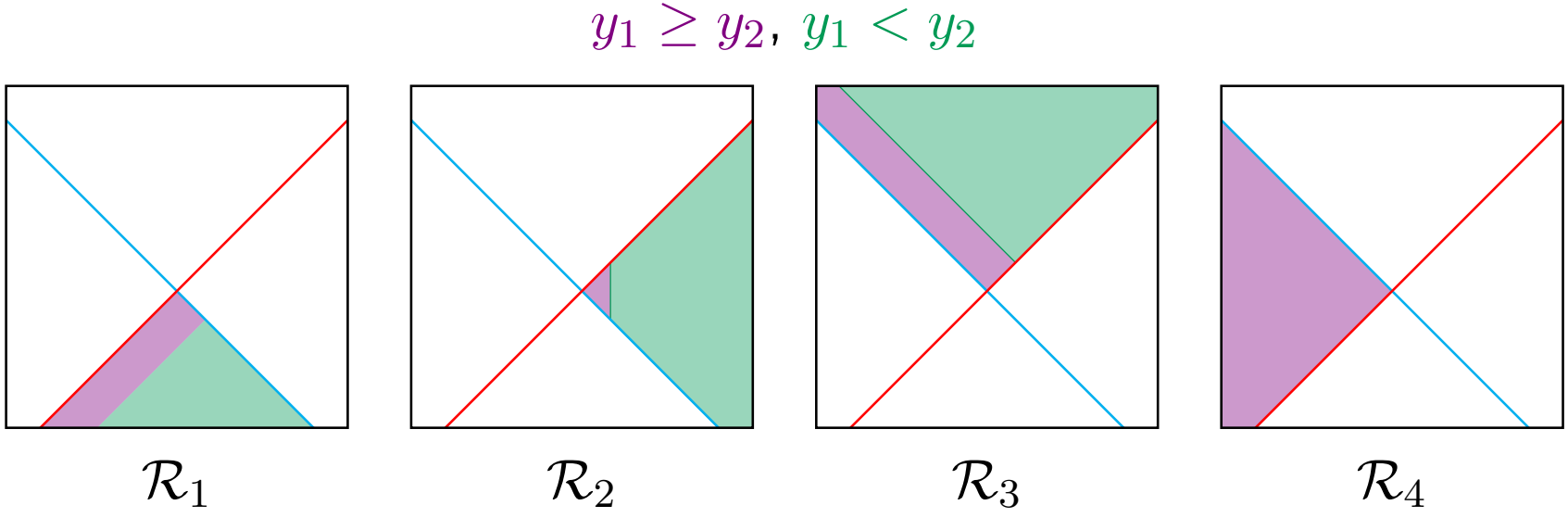
$$f(\mathbf{x}) = \mathbf{y} = W_{\mathcal{R}_i}\,\mathbf{x} + \mathbf{b}_{\mathcal{R}_i}$$

where, given $\mathbf{p}_i$ - indices of positive entries in $\mathbf{x}_1$ - have

$$W_{\mathcal{R}_i} = W_2(:, \mathbf{p}_i)\, W_1(\mathbf{p}_i, :)$$
$$\mathbf{b}_{\mathcal{R}_i} = W_2(:, \mathbf{p}_i)\, \mathbf{b}_1(\mathbf{p}_i) + \mathbf{b}_2$$

Final decision regions of the simple network

- Use $\mathbf{y} = f(\mathbf{x})$ to define a 2-class classifier.



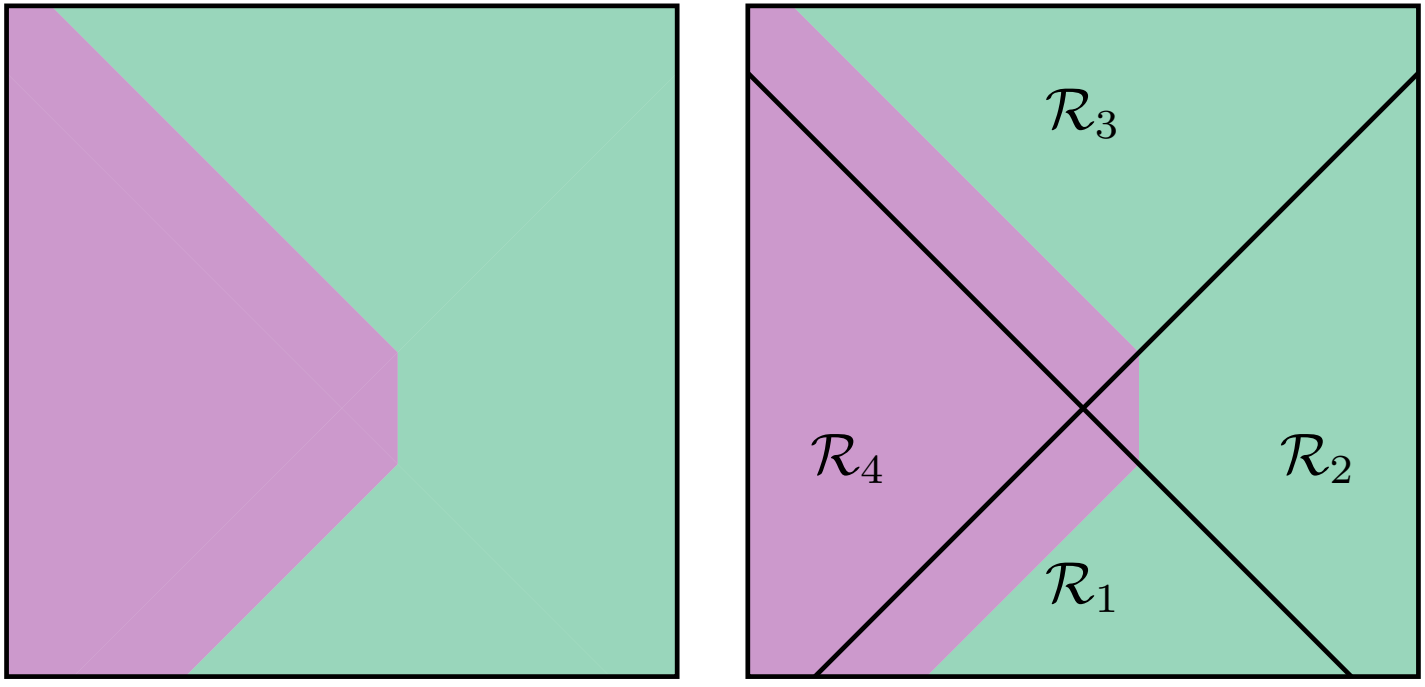
Final decision regions for each activity pattern

Final decision regions of the simple network

- Use $\mathbf{y} = f(\mathbf{x})$ to define a 2-class classifier.

Final decision regions for input in $[-1, 1]^2$

$y_1 \geq y_2$, $y_1 < y_2$



Important to remember...

1. Input space partitioned into different regions based on Layer 1 activity patterns.

2. Each region is a convex polytope (assuming bounded inputs).

3. In each region the non-linear neural network function corresponds to an affine function.

4. Our simple network function corresponds to a **continuous piecewise affine function**.

What about a more generic 1-hidden layer network?

Arbitrary # inputs, outputs and hidden nodes?

- Dimension of input, output and hidden nodes: d, m and h.

- This remains true from simple example:
  1. Input space partitioned into different regions based on Layer 1 activity patterns.
  2. Each region is a convex polytope (assuming bounded inputs).
  3. In each region the non-linear neural network function corresponds to an affine function.
  4. The network function corresponds to a **continuous piecewise affine function**.

- What differs?
  1. The number of possible activity patterns.....

Arbitrary # inputs, outputs and hidden nodes?

- Dimension of input, output and hidden nodes: d, m and h.

- This remains true from simple example:
  1. Input space partitioned into different regions based on Layer 1 activity patterns.
  2. Each region is a convex polytope (assuming bounded inputs).
  3. In each region the non-linear neural network function corresponds to an affine function.
  4. The network function corresponds to a **continuous piecewise affine function**.

- What differs?
  1. The number of possible activity patterns.....

Efficient Oscillations with Composition

- Define the function $g : \mathbb{R} \to \mathbb{R}$

$$g(x) = \left[[2x]_+ - [4x - 2]_+ \right]_+$$

- Sketch

$$g(x)$$

for $x \in [-.3, 1.3]$

Benefits of depth in neural networks, Matus Telgarsky, 2016

Efficient Oscillations with Composition

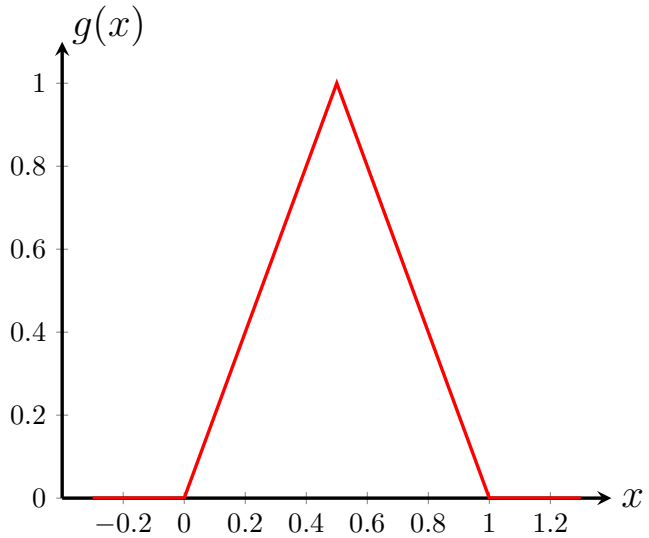
- Define the function $g : \mathbb{R} \to \mathbb{R}$

$$g(x) = \left[[2x]_+ - [4x - 2]_+ \right]_+$$

- Sketch

$$g(x)$$

for $x \in [-.3, 1.3]$



Benefits of depth in neural networks, Matus Telgarsky, 2016

Efficient Oscillations with Composition

- Define the function $g : \mathbb{R} \to \mathbb{R}$

$$g(x) = \left[[2x]_+ - [4x - 2]_+ \right]_+$$

- Sketch

$$g(g(x))$$

for $x \in [-.3, 1.3]$

Benefits of depth in neural networks, Matus Telgarsky, 2016

Efficient Oscillations with Composition

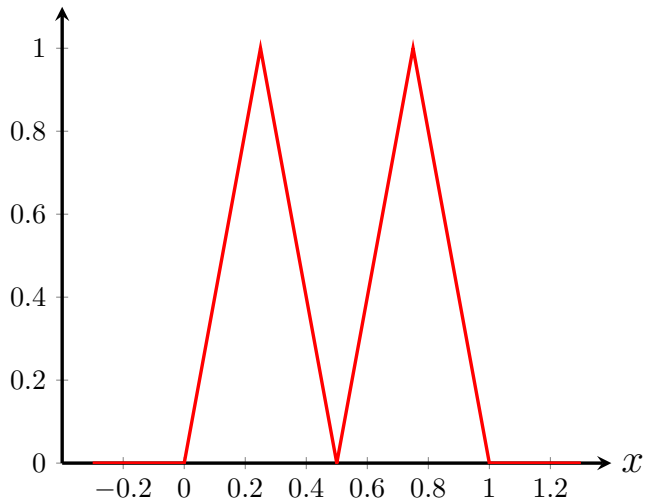
- Define the function $g : \mathbb{R} \to \mathbb{R}$

$$g(x) = \left[[2x]_+ - [4x - 2]_+\right]_+$$

- Sketch

$$g(g(x))$$

for $x \in [-.3, 1.3]$



Benefits of depth in neural networks, Matus Telgarsky, 2016

Efficient Oscillations with Composition

- Define the function $g : \mathbb{R} \to \mathbb{R}$

$$g(x) = \left[[2x]_+ - [4x - 2]_+ \right]_+$$

- Sketch

$$g(g(g(x)))$$

for $x \in [-.3, 1.3]$

Benefits of depth in neural networks, Matus Telgarsky, 2016

Efficient Oscillations with Composition

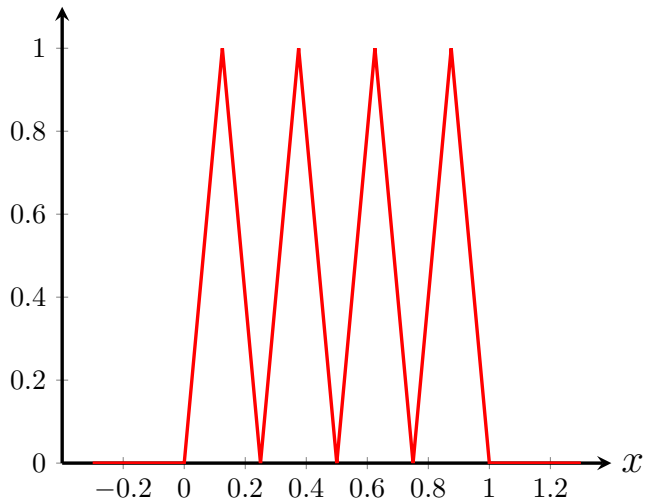
- Define the function $g : \mathbb{R} \to \mathbb{R}$

$$g(x) = \left[[2x]_+ - [4x - 2]_+\right]_+$$

- Sketch

$$g(g(g(x)))$$

for $x \in [-.3, 1.3]$



1 more layer $\implies$ twice as many oscillations

Benefits of depth in neural networks, Matus Telgarsky, 2016

Efficient Oscillations with Composition

- Define the function $g : \mathbb{R} \to \mathbb{R}$

$$g(x) = \left[[2x]_+ - [4x - 2]_+\right]_+$$

- Sketch

$$g(g(g(g(x))))$$

for $x \in [-.3, 1.3]$

Benefits of depth in neural networks, Matus Telgarsky, 2016

Efficient Oscillations with Composition

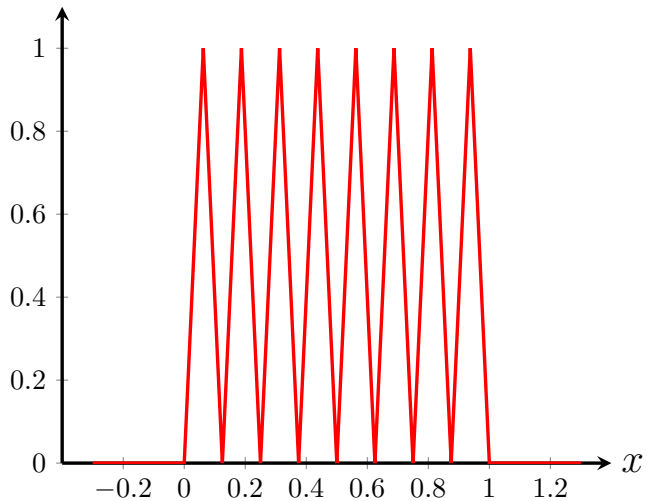
- Define the function $g : \mathbb{R} \to \mathbb{R}$

$$g(x) = \left[[2x]_+ - [4x - 2]_+\right]_+$$

- Sketch

$$g(g(g(g(x))))$$

for $x \in [-.3, 1.3]$



1 more layer $\implies$ twice as many oscillations

Benefits of depth in neural networks, Matus Telgarsky, 2016

Deep ReLU networks can generate lots of regions efficiently

In this example:

- Function g corresponded to the 2-hidden layer network:

$$g(x) = \left[W_2 \left[W_1 x + \mathbf{b}_1\right]_+\right]_+$$

with

$$W_1 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 \\ -2 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 1 & -1 \end{pmatrix}$$

- As g is recursively applied double the number of linear regions.

- Recursive much more efficient parameter-wise than defining the eight regions as

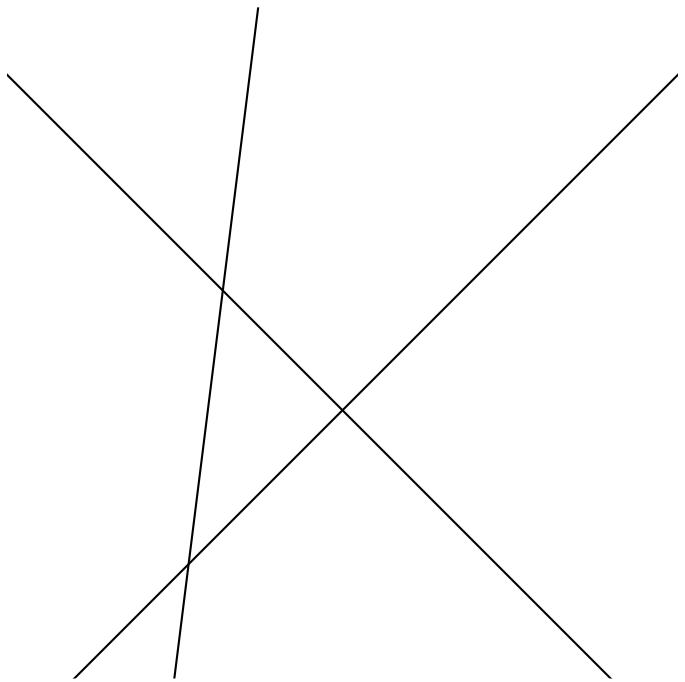
$$g_1(x) + g_2(x) + \ldots + g_8(x)$$

What about for higher dimesional inputs and generic networks?

Number of regions for 1-hidden layer network

Theorem by Zaslavsky (1975).
k hyper-planes (in general position) in d dimensional space:

$$\#\text{ of regions created} = \sum_{j=0}^{d} \binom{k}{j}$$

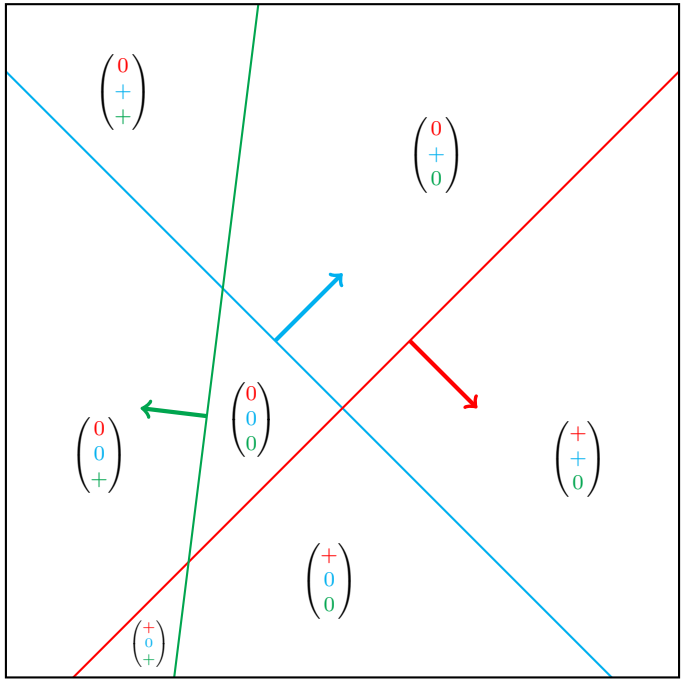


$d = 2$, $k = 3$, # of regions is 7

Number of regions for 1-hidden layer network

Apply Theorem to our 1-layer network with h hidden nodes (tight bound)

$$\#\text{ of activity patterns} \leq \sum_{j=0}^{d} \binom{h}{j}$$

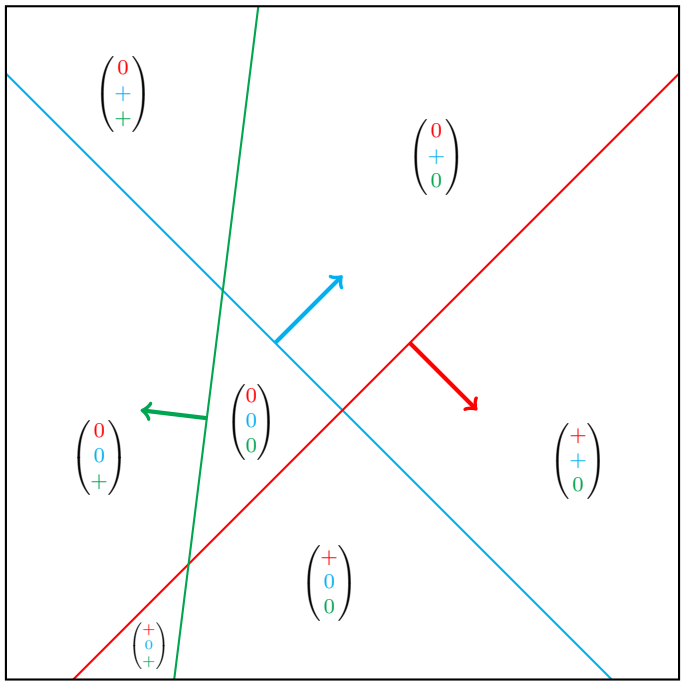


$d = 2$, $h = 3$, **# of activity patterns is 7**

Number of regions for 1-hidden layer network

Apply Theorem to our 1-layer network with h hidden nodes (tight bound)

$$\#\text{ of activity patterns} \leq \sum_{j=0}^{d} \binom{h}{j}$$

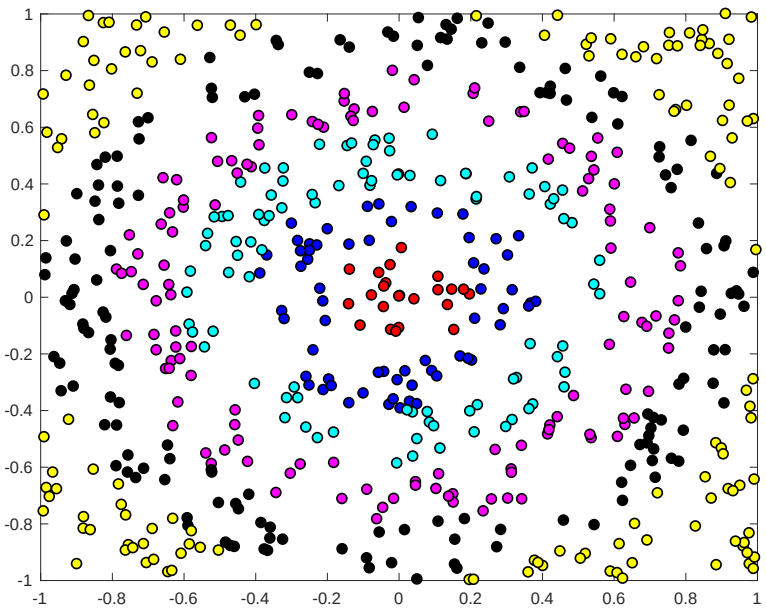


$d = 2$, $h = 3$, **# of activity patterns is 7**

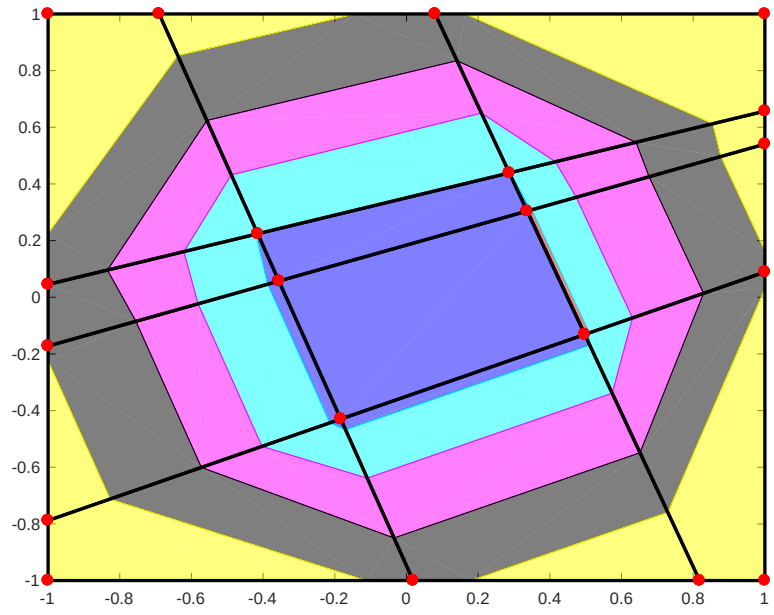
- **Note**: Activity pattern $(+ \quad + \quad +)^T$ is not possible for this network.
- As h increases set of possible activity patterns $\rightarrow .5h^2$ not 2^h.

1-hidden layer network: example of learned functions

- $d = 2, h = 5$ and $m = 6$
- Random initialization.
- Mini-batch gradient descent training.



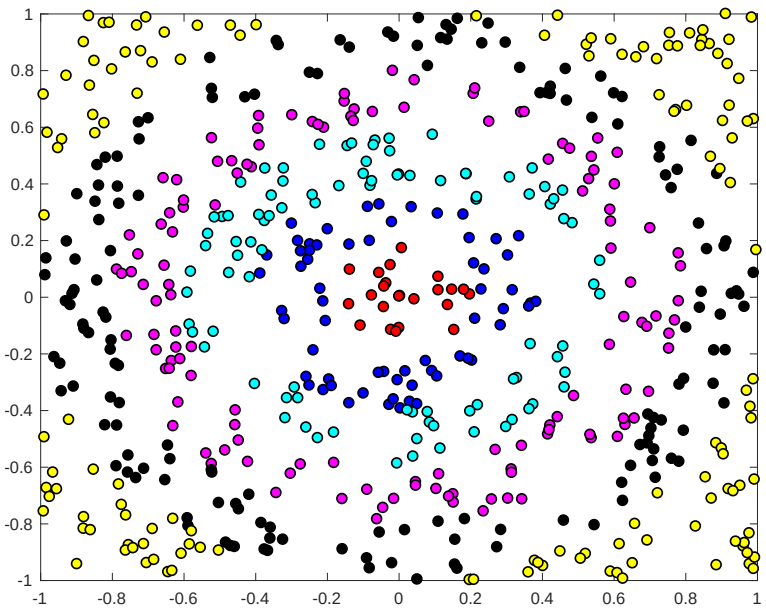
Training Data



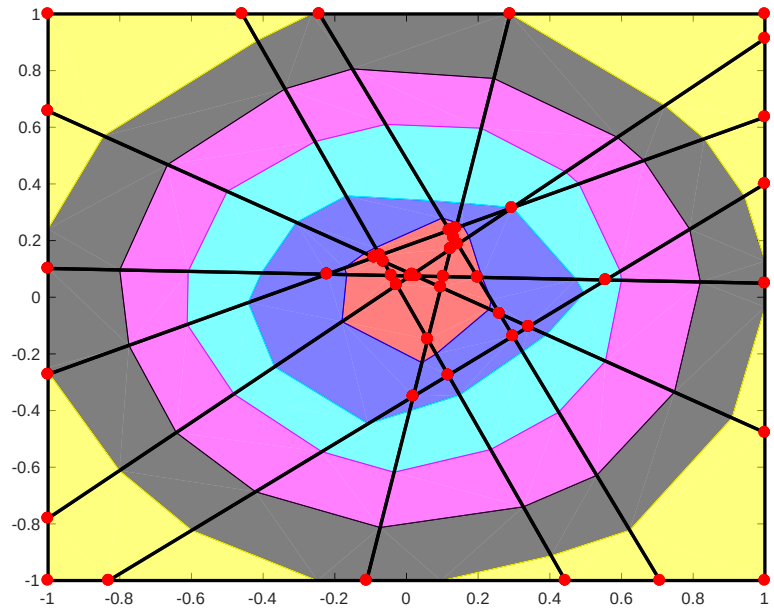
Decision surface for learnt network

1-hidden layer network: example of learned functions

- $d = 2, h = 10$ and $m = 6$
- Random initialization.
- Mini-batch gradient descent training.



Training Data



Decision surface for learnt network

What about a $(L-1)$-hidden layer network?

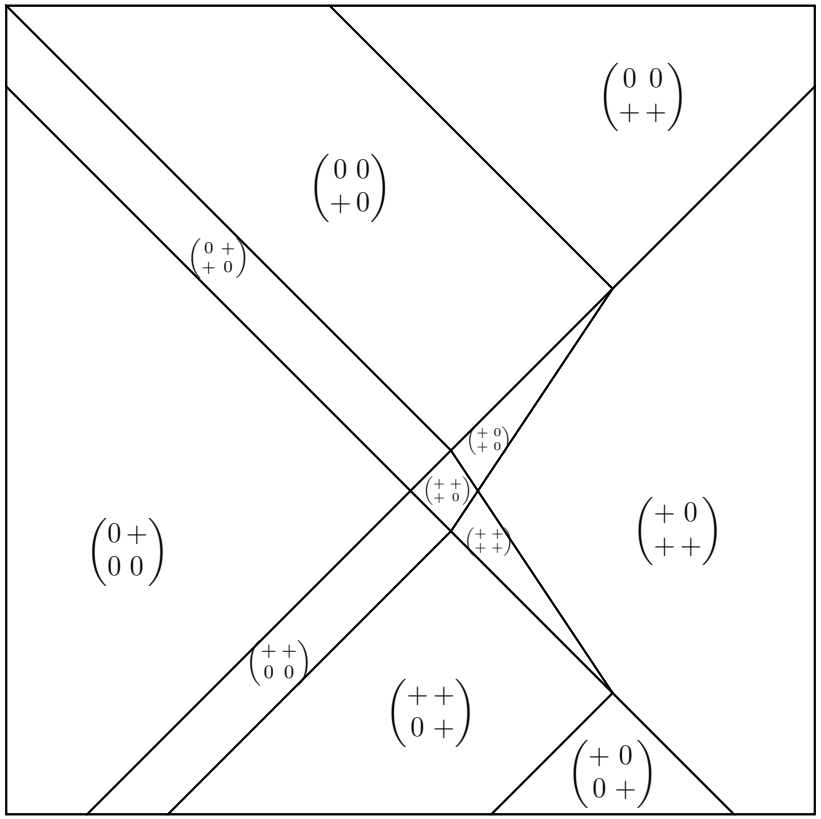
Same qualitative results as before

- Assume $L-1$ hidden layers each with dimension h.

- These results hold
  1. Input space partitioned into different regions based on Layer 1, Layer 2, ..., Layer $(L-1)$ activity patterns.
  2. Each region is a convex polytope (assuming bounded inputs).
  3. In each region the non-linear neural network function corresponds to an affine function.
  4. The network function corresponds to a **continuous piecewise affine function**.

Activity patterns for > 1 hidden layer

Each column is the activity pattern at a layer.

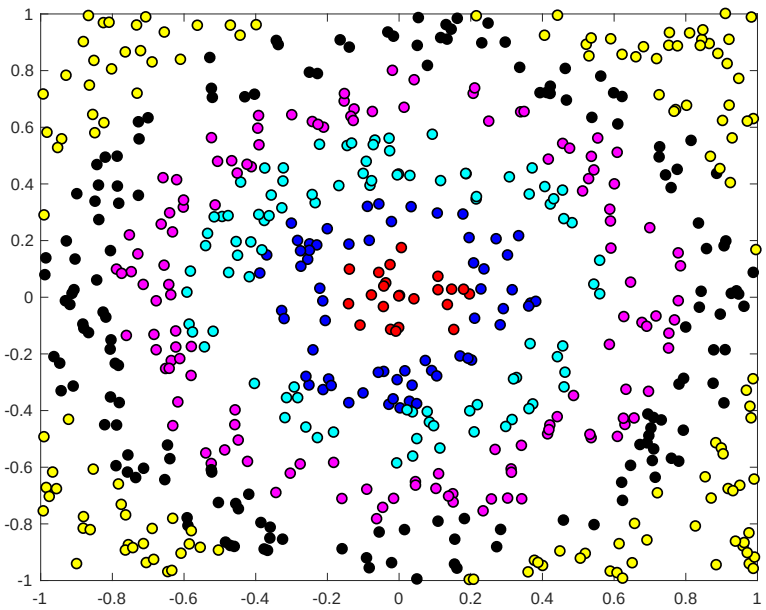


$d = 2, h = 2, L = 3$, # of regions is 11

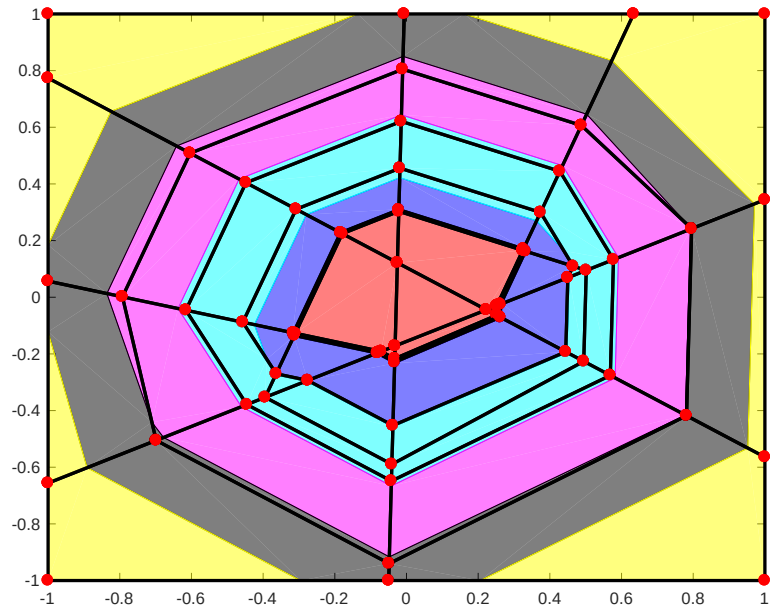
- Notes on the previous slides
  1. To define each **affine function** need positive indices at each layer.
  2. Convex polytope describing *activity region* defined by $h(L-1)$ linear constraints.

Multi-layer network: example of learned function

- $d = 2, h = 3,\ L = 5$ and $m = 6$
- Random initialization.
- Mini-batch gradient descent training.



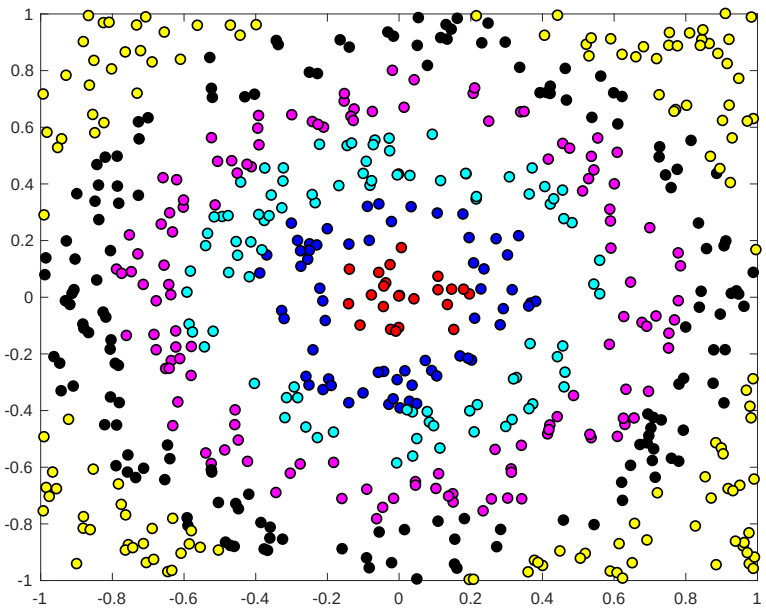
Training Data



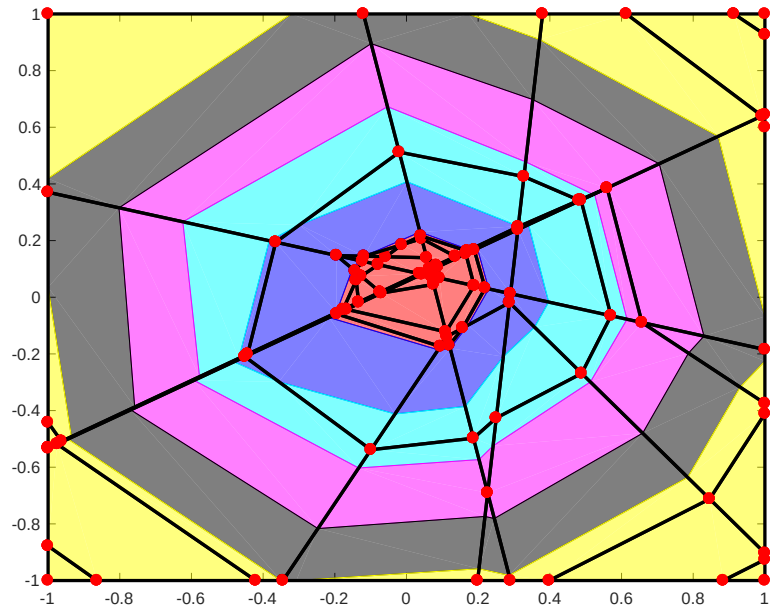
Decision surface for learnt network

Multi-layer network: example of learned function

- $d = 2, h = 4,\ L = 5$ and $m = 6$
- Random initialization.
- Mini-batch gradient descent training.



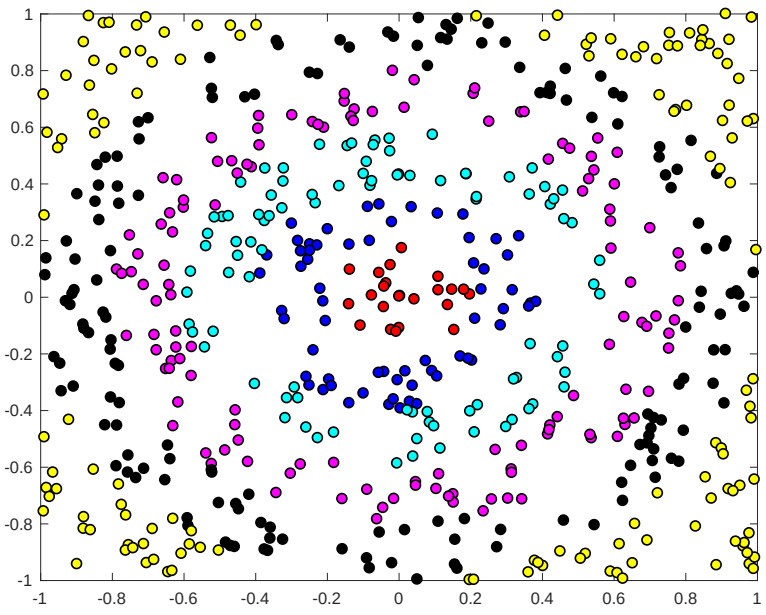
Training Data



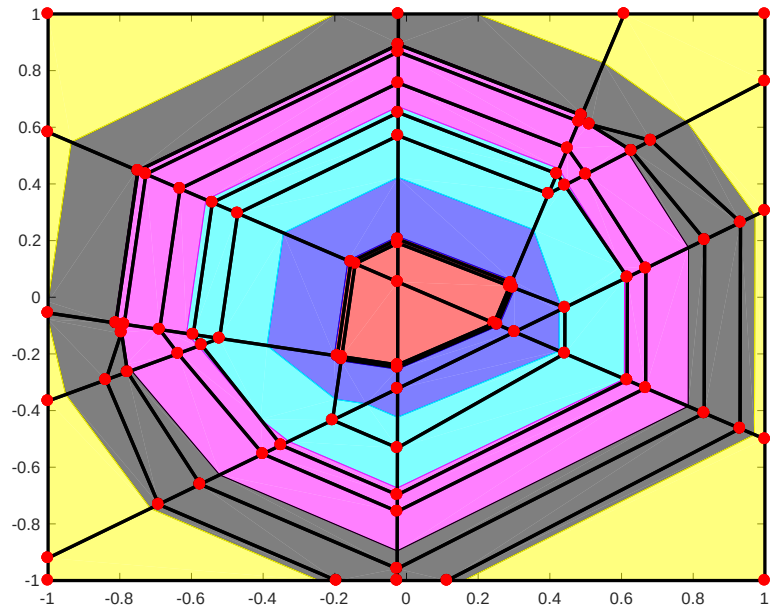
Decision surface for learnt network

Multi-layer network: example of learned function

- $d = 2, h = 3,\ L = 6$ and $m = 6$
- Random initialization.
- Mini-batch gradient descent training.



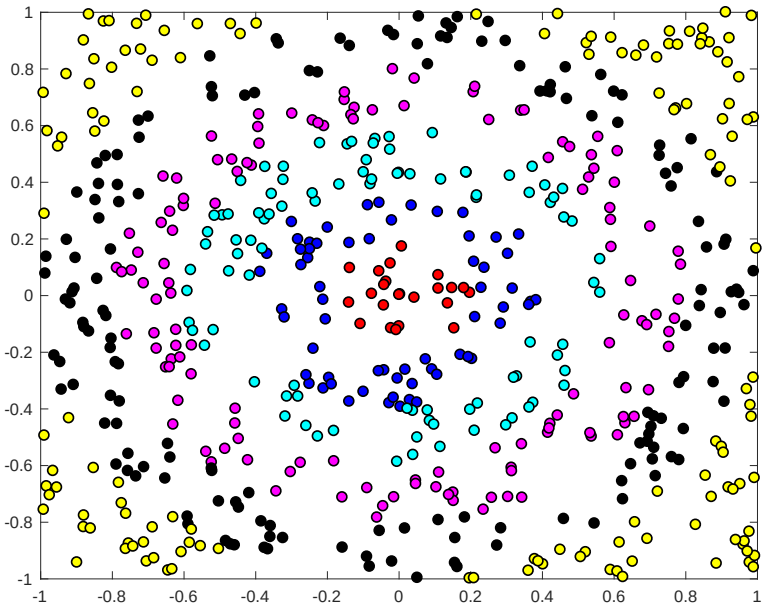
Training Data



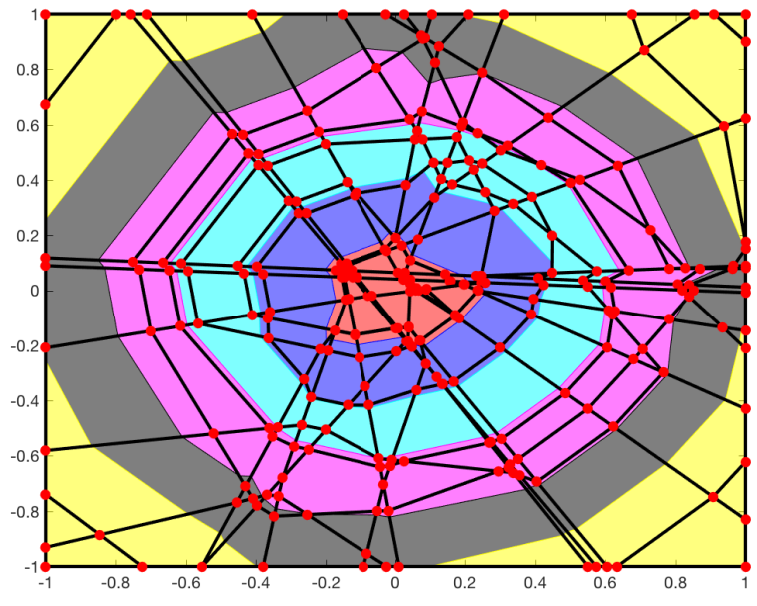
Decision surface for learnt network

Multi-layer network: example of learned function

- $d = 2, h = 4,\ L = 6$ and $m = 6$
- Random initialization.
- Mini-batch gradient descent training.



Training Data



Decision surface for learnt network

Number of possible activity patterns??

- Active area of research
  - Arora, R., Basu, A., Mianjy, P., and Mukherjee, A. Understanding deep neural networks with rectified linear units. In ICLR, 2018.
  - Montufar, G. Notes on the number of linear regions of deep neural networks. In SampTA, 2017.
  - Montufar, G., Pascanu, R., Cho, K., and Bengio, Y. On the number of linear regions of deep neural networks. In NIPS, 2014.
  - Serra, T., Tjandraatmadja, C., Ramalingam, S. Bounding and Counting Linear Regions of Deep Neural Networks, ICML, 2018
  - Rahaman, N., Baratin, A., Arpit D., Draxler, F., Lin, M., Hamprecht, F.A., Bengio, Y., Courville, A. On the Spectral Bias of Neural Networks, arXiv, October 2018
- Let K_{MAX} be the maximum number of activity patterns.
- Upper and lower bounds exist for K_{MAX}
- One easy bound to state and that is fairly tight

$$\left\lfloor \frac{h}{d} \right\rfloor^{(L-2)d} \sum_{j=0}^{d} \binom{h}{j} \leq K_{\text{MAX}}$$

Consequences of lower bound on K_{MAX}

- K_{MAX} grows exponentially in L but only polynomial in h
- $\Longrightarrow$ to increase the capacity of network much more efficient to mainly increase L as opposed to mainly increase h.
- Theoretical argument as opposed to empirical argument for deep networks over shallow networks.

Depth & Parametric Cost

Benefits of depth in neural networks, Matus Telgarsky, 2016:

There exist deep ReLU networks which can only be approximated by shallower networks if they have exponentially as many nodes.

For a fixed param budget, deeper is better

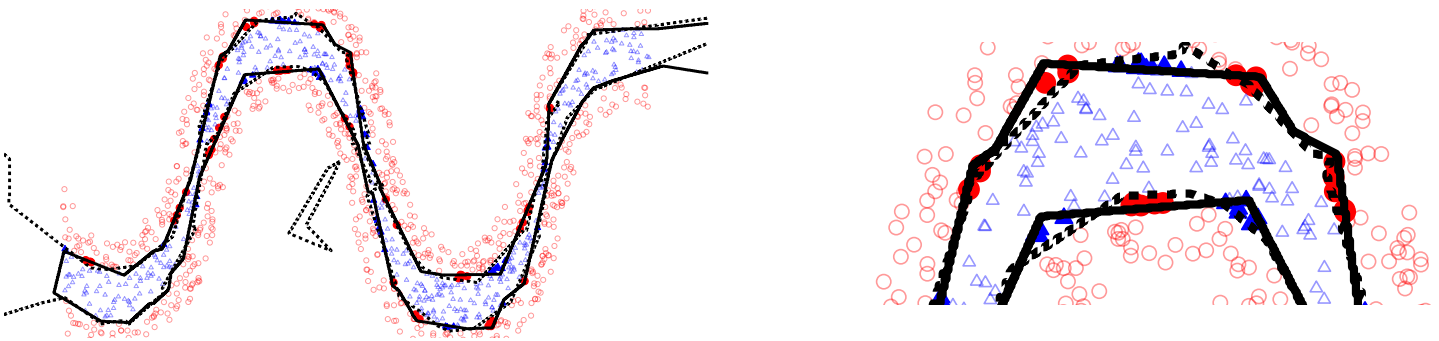
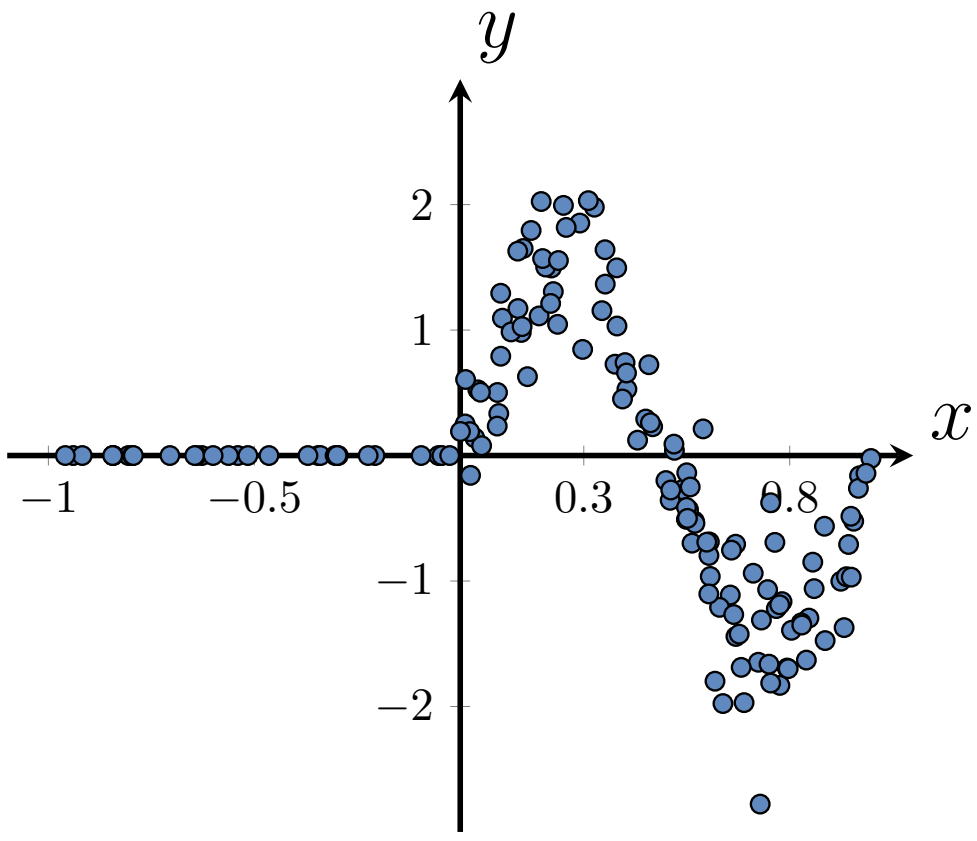


Figure 1: Binary classification using a shallow model with 20 hidden units (solid line) and a deep model with two layers of 10 units each (dashed line). The right panel shows a close-up of the left panel. Filled markers indicate errors made by the shallow model.

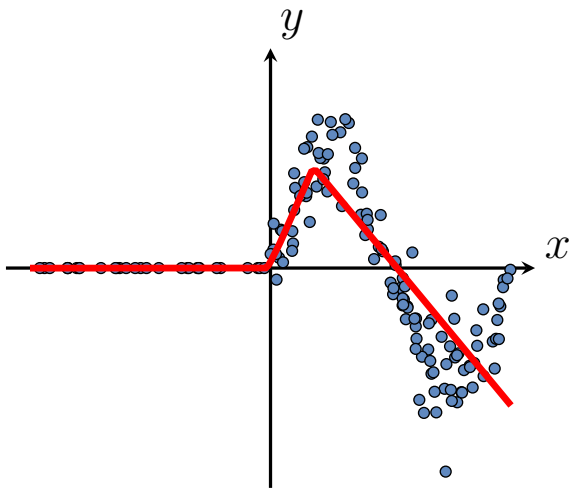
On the Number of Linear Regions of Deep Neural Networks, G. Montúfar, R. Pascanu, K. Cho, Y. Bengio, 2014.

But for typical trained networks: "# of regions" does not grow as expected with depth..

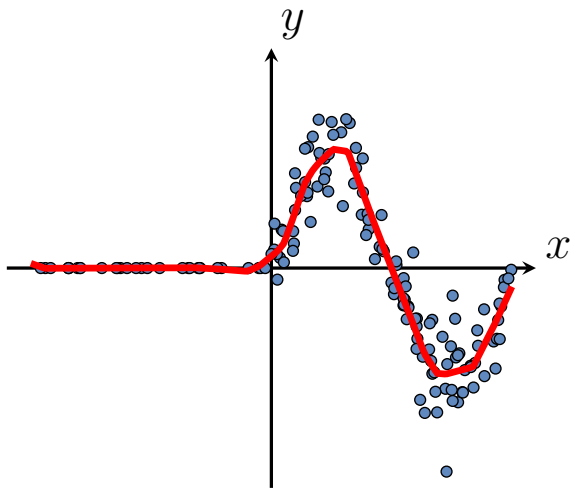
Consider this simple regression problem



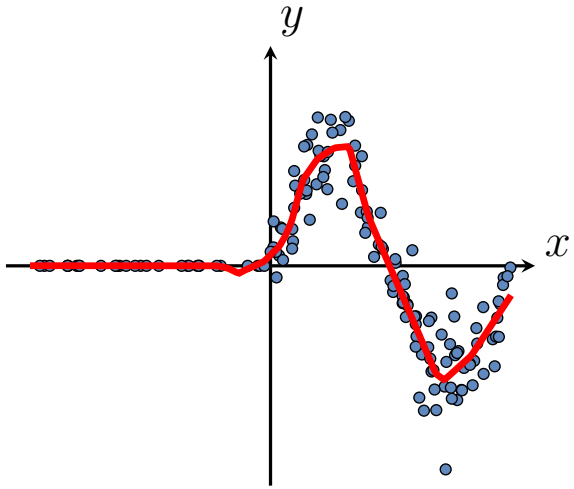
Fit MLPs with 50 hidden nodes



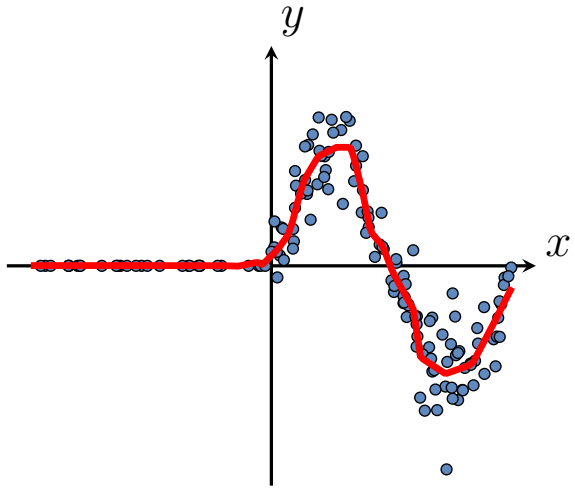
[50]



[25, 25]

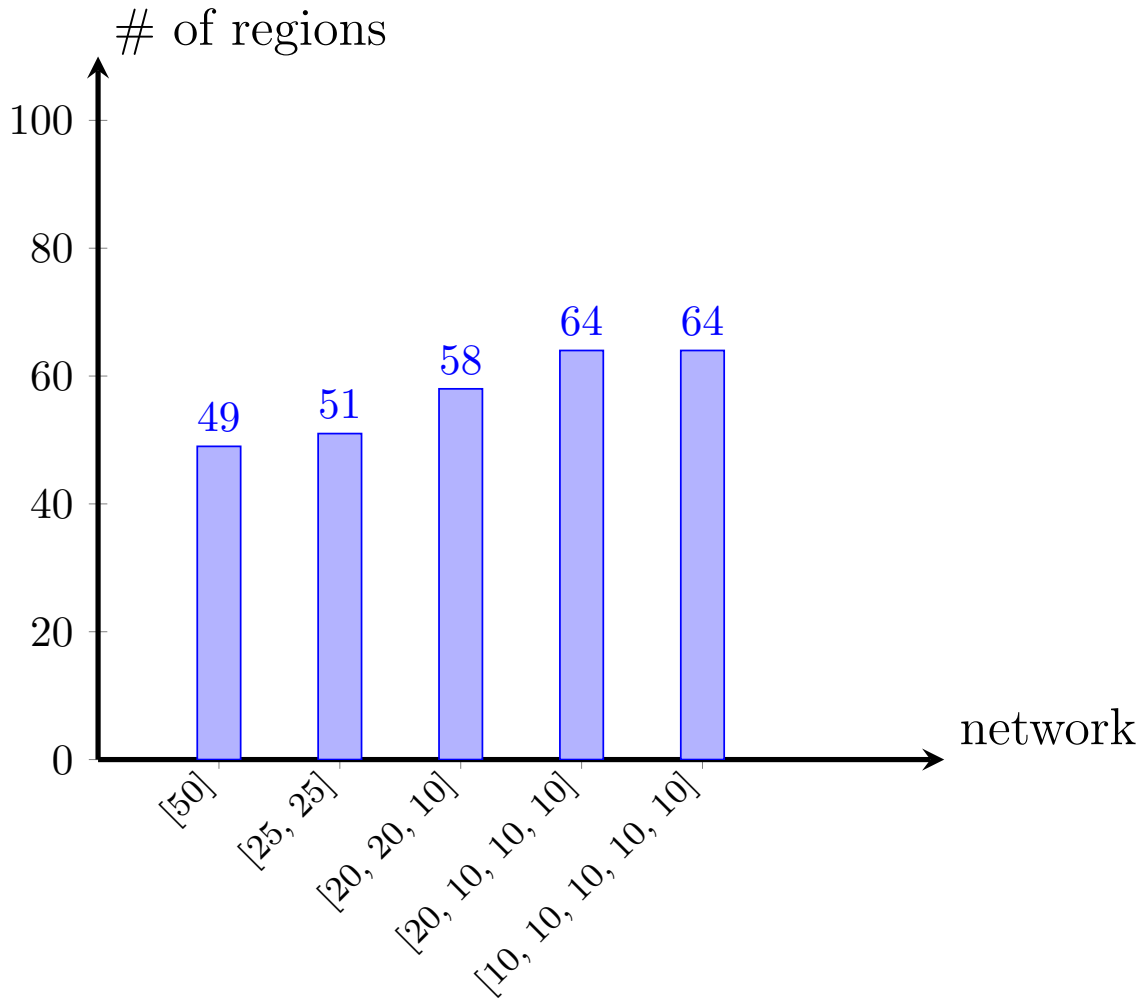


[20,20,10]

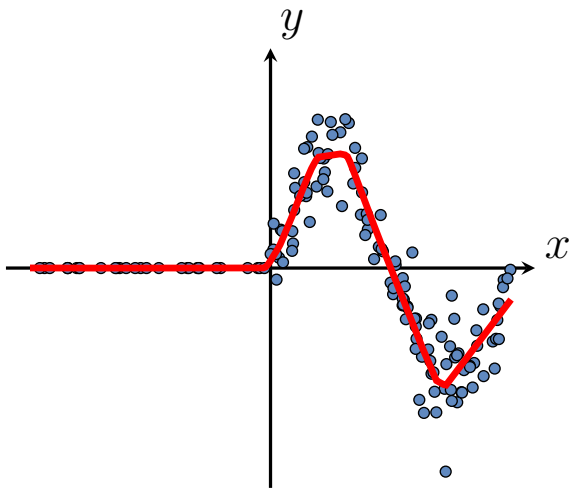


[10, 10, 10, 10, 10]

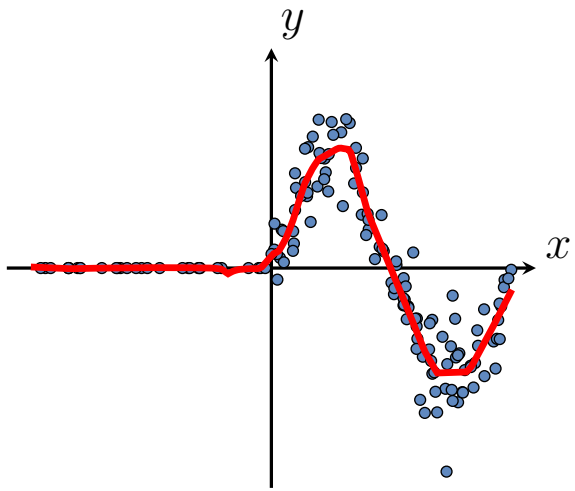
#number of regions as depth varies



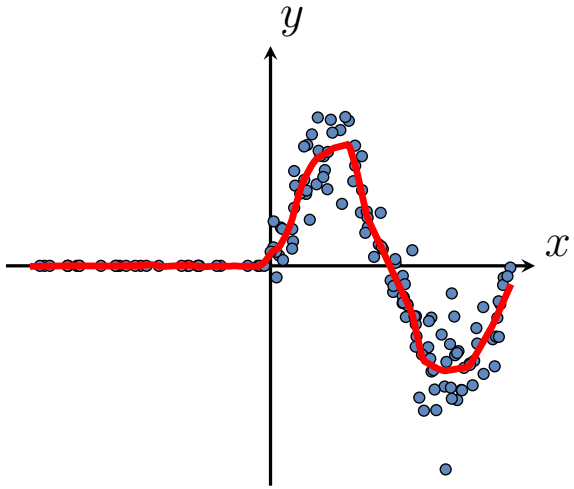
Fit MLPs with 300 hidden nodes



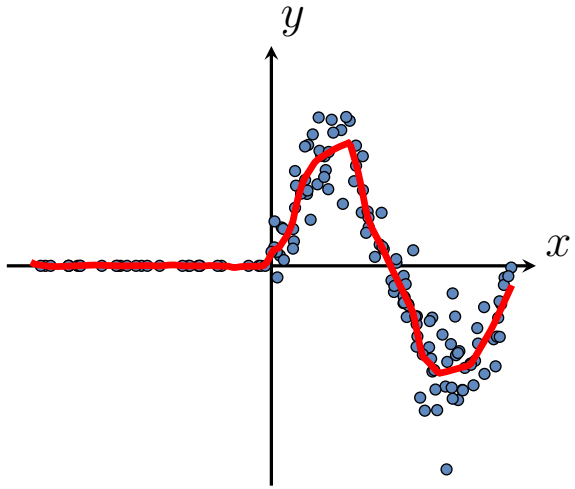
[300]



[200, 100]

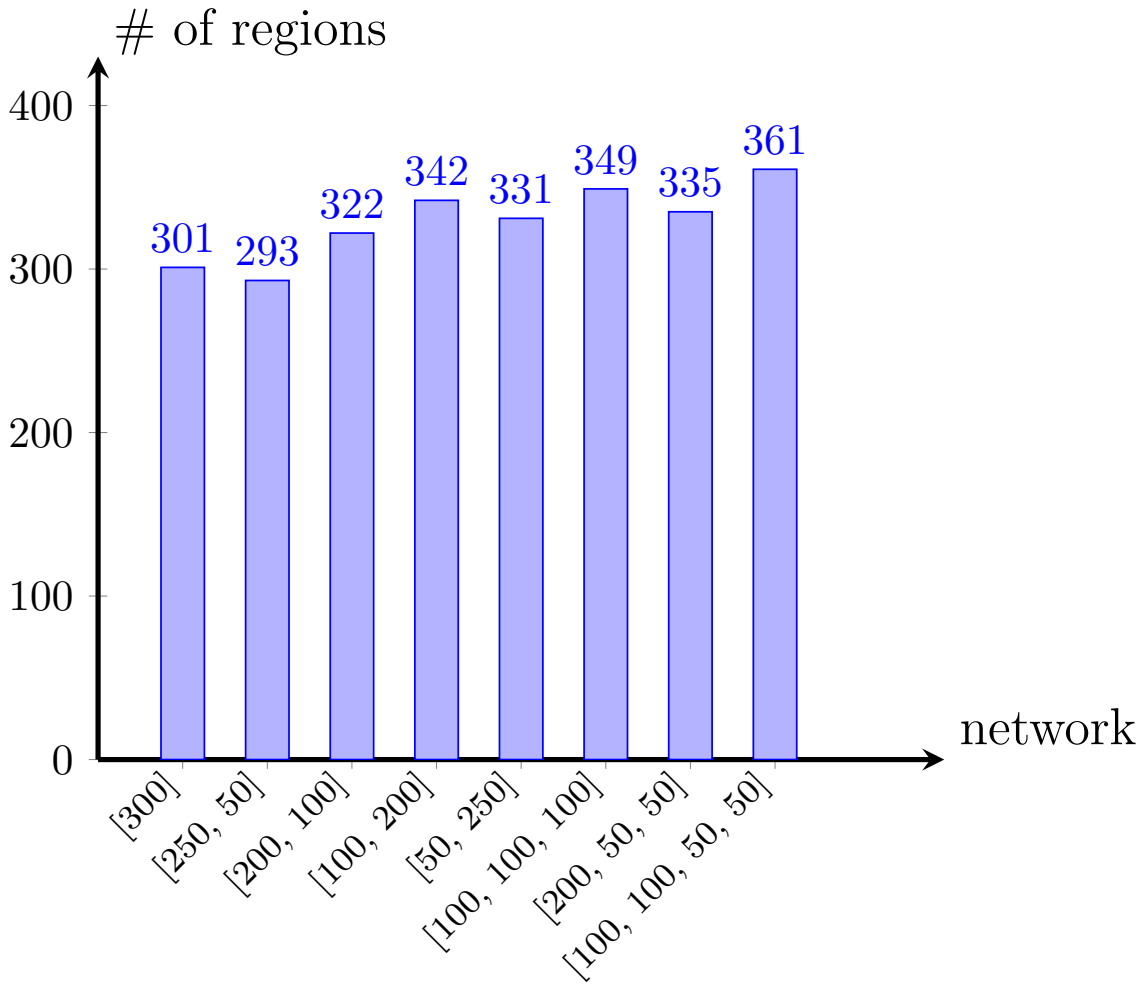


[100,100,100]



[100, 50, 50, 50, 50]

#number of regions as depth varies



Hanin & Rolnick work

Deeper networks do not learn significantly more complex functions than shallow networks.

if complexity is measured by # of linear regions.

- Give evidence (theoretical and empirical) that for ReLU networks
  "*# of activation patterns*" $\propto \{\text{total \# of neurons}\}^d$

Is the "# of regions" a red herring...

- **Lots of experimental evidence**: deep Convolutional Networks needed to learn complicated mappings between inputs and outputs.
- What is the theoretical justification for this????
- Looks like *# of regions* of a trained classifier is not the prime key factor for explaining this necessity.
- Perhaps not a case of "*maxing out*" the representational power but the inherent redundancy of the deep network's capacity makes it easier to train.
- Or "*maxing out*" involves learning very strange functions.

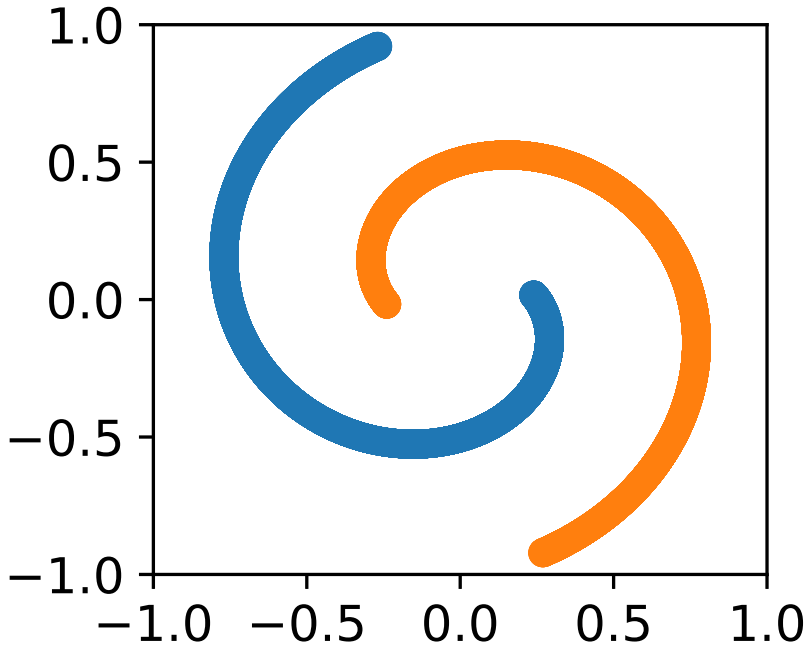
But "# of regions" is affected by training procedures.

Following slides review of results from

EMPIRICAL STUDIES ON THE PROPERTIES OF LINEAR REGIONS IN DEEP NEURAL NETWORKS

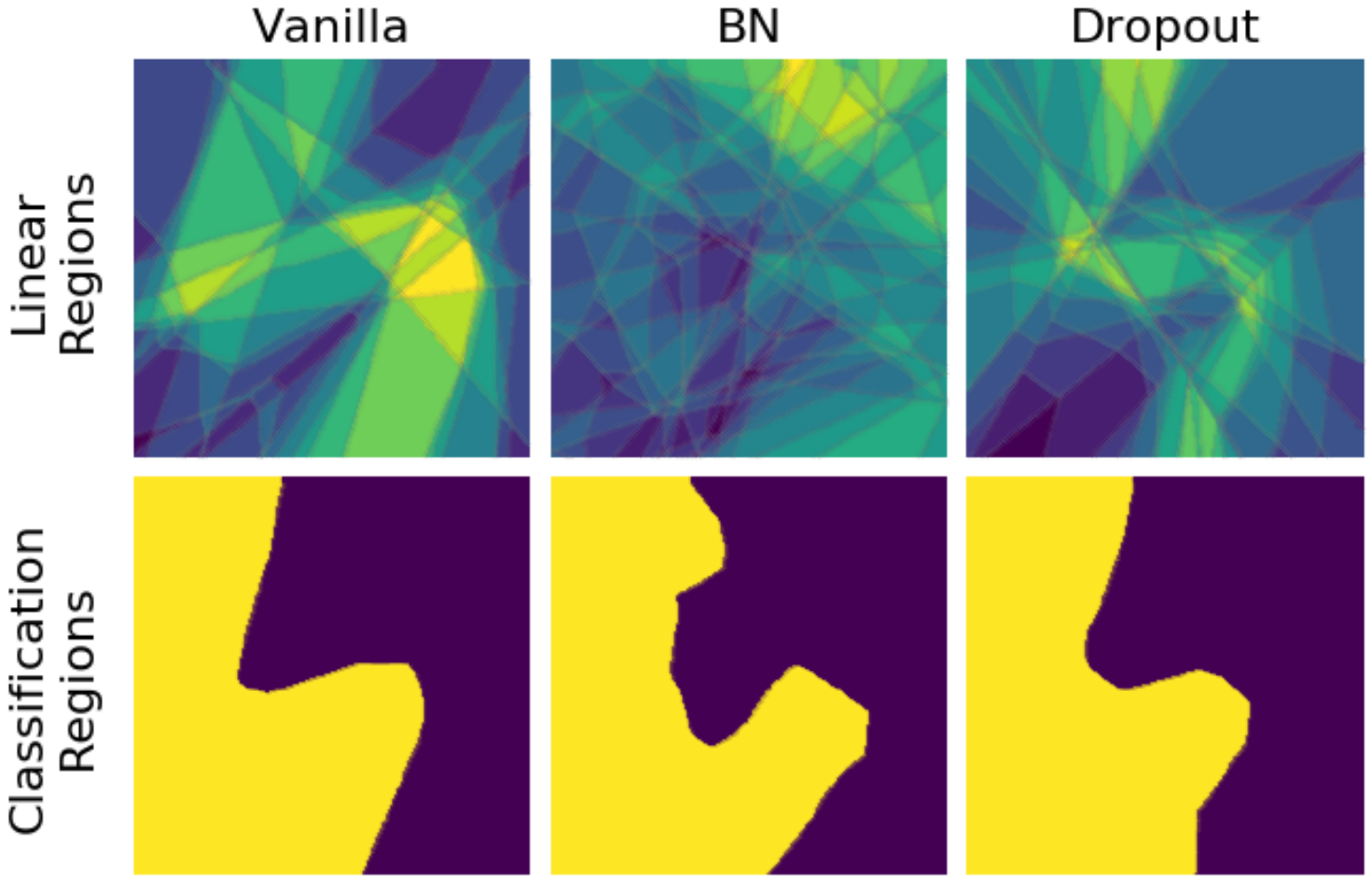
X. Zhang and D. Wu, ICLR 2020

Consider this toy example

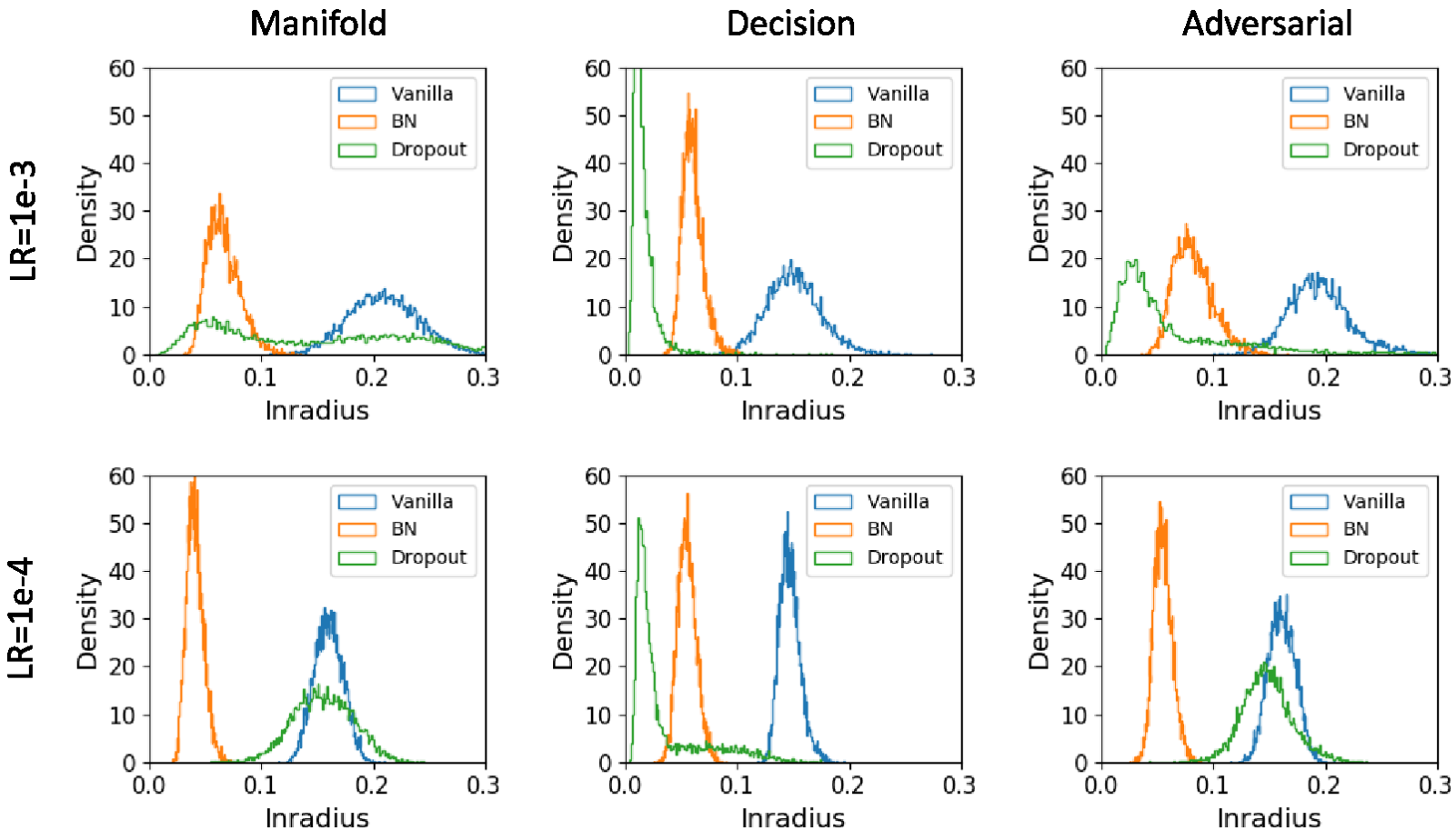


- Binary classification problem.
- Fit a 3-hidden layer network each with 10 nodes.

Linear and classication regions with different training



Some results on MNIST



- Fit a 3-hidden layer network each with 1,024 nodes.
- Histograms of radius of inscribed sphere of different linear regions.
- **Manifold** - the linear regions of the test points

General empirical findings

- BN and dropout $\implies$ trained network partitions the input space into many more linear regions than vanilla training.
- Linear regions for the BN model are more uniform in size than those from dropout.
- Many transition boundaries of the dropout NN share similar norm directions, and concentrate around the decision boundaries.

Optimization for Deep Networks

Assume most of you know the practical stuff

To summarize optimization process

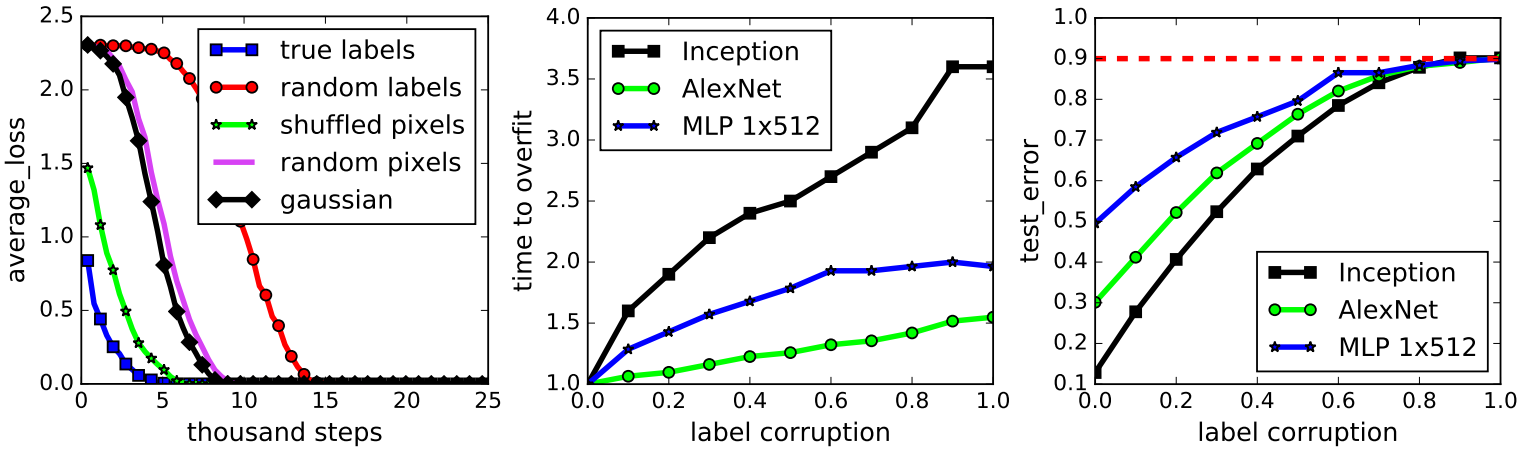
- Use variants of mini-batch gradient descent.
- No global minimum convergence guarantee
- But feed-forward networks are often "easy" to train:
  - Different random seeds yield similar final train losses.
  - When wide enough, training error can often reach zero: no bad local minima.
  - SGD (with momentum) seem to be able to evade saddle structures quite easily.

Must take account of conditioning issues

- Vanishing gradients:
  - Deep FF Network: smaller gradient components in first layers.
- Optimal learning rate should be higher for some gradient components:
  - Bad conditioning of the Hessian matrix.
- Can fix the Conditioning Issues by:
  - Fixing the optimizer - AdaGrad, Adam, etc,
  - Fixing the architecture - skip connections, BN, etc...
  - Careful initialization

Generalization

Deep Nets Generalize Better than They Should



(a) learning curves (b) convergence slowdown (c) generalization error growth

Figure 1: Fitting random labels and random pixels on CIFAR10. (a) shows the training loss of various experiment settings decaying with the training steps. (b) shows the relative convergence time with different label corruption ratio. (c) shows the test error (also the generalization error since training error is 0) under different label corruptions.

Understanding deep learning requires rethinking generalization, C. Zhang et al., 2016.

The Mystery of Generalization

What makes deep networks generalize so well?

- Intrinsic to the piecewise linear nature of the ReLU networks?
- Deep compositional architectures = strong inductive prior?
- SGD with
  - large step sizes,
  - small mini-batches and
  - early stopping?
- Gradient Descent on over-parametrized network from small random init?
- A combination of all of the above?

The Mystery of Generalization

What makes deep networks generalize so well?

- Intrinsic to the piecewise linear nature of the ReLU networks?
- Deep compositional architectures = strong inductive prior?
- SGD with
  - large step sizes,
  - small mini-batches and
  - early stopping?
- Gradient Descent on over-parametrized network from small random init?
- A combination of all of the above?

The Mystery of Generalization

What makes deep networks generalize so well?

- Intrinsic to the piecewise linear nature of the ReLU networks?
- Deep compositional architectures = strong inductive prior?
- SGD with
  - large step sizes,
  - small mini-batches and
  - early stopping?
- Gradient Descent on over-parametrized network from small random init?
- A combination of all of the above?

The Mystery of Generalization

What makes deep networks generalize so well?

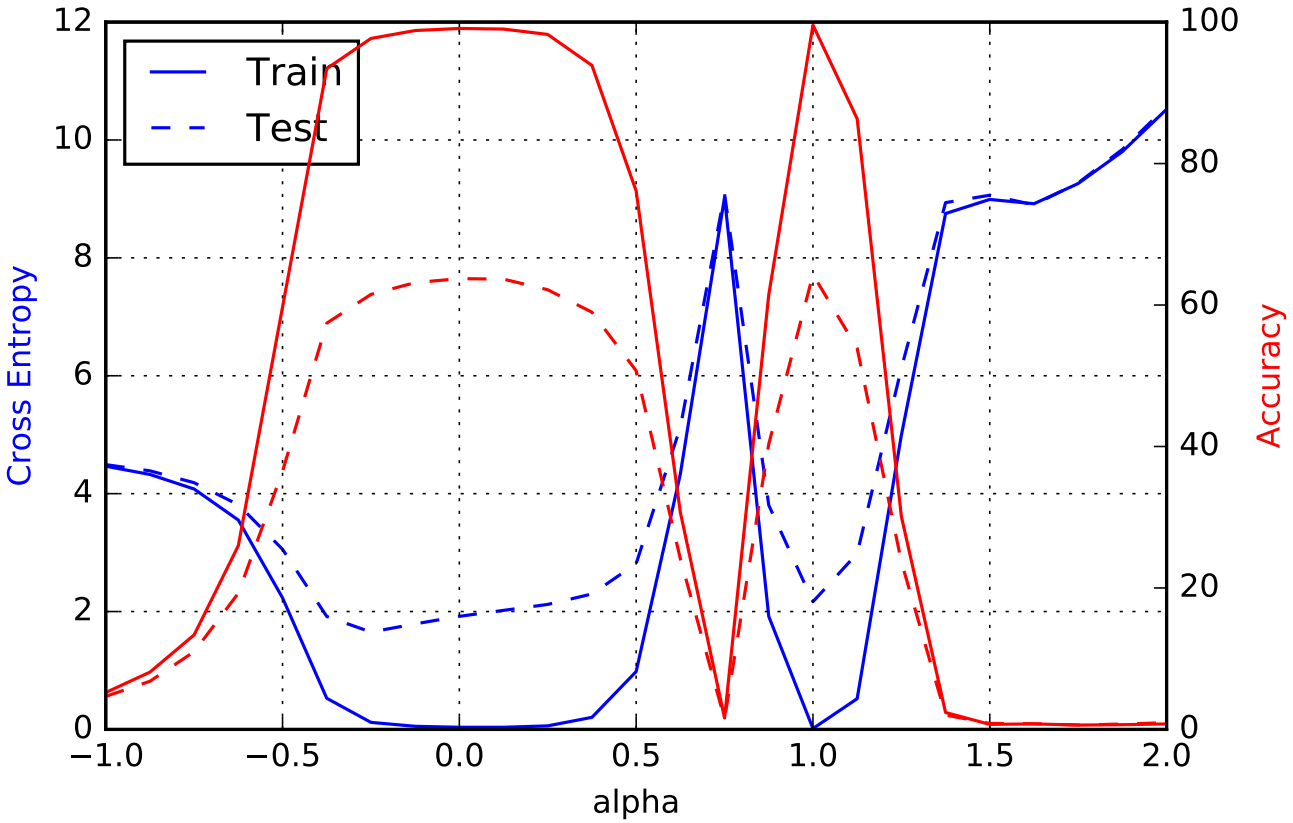
- Intrinsic to the piecewise linear nature of the ReLU networks?
- Deep compositional architectures = strong inductive prior?
- SGD with
  - large step sizes,
  - small mini-batches and
  - early stopping?
- Gradient Descent on over-parametrized network from small random init?
- A combination of all of the above?

The Mystery of Generalization

What makes deep networks generalize so well?

- Intrinsic to the piecewise linear nature of the ReLU networks?
- Deep compositional architectures = strong inductive prior?
- SGD with
  - large step sizes,
  - small mini-batches and
  - early stopping?
- Gradient Descent on over-parametrized network from small random init?
- A combination of all of the above?

Small Batch Vs Large Batch



Curves generated by a deep ConvNet trained & tested on Cifar-100

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima, N. S. Keskar et al., 2016

Interplay of SGD, learning rate, shape of loss landscape, ...

- Typical model capacity is enough to represent bad global minima.
- But even with large batches, GD avoids those pathological global minima.
- Small-batch / large learning rate models generalize a bit better but not the main effect.
- Measuring wideness, sharpness of minima is tricky because of reparametrization.
- Good global minimizers have a wide attractor basin for GD

Source: Towards Understanding Generalization of Deep Learning: Perspective of Loss Landscapes by Lei Wu, Zhanxing Zhu, Weinan E

Interesting recent papers in these areas

- "Sharpness-Aware Minimization for Efficiently Improving Generalization", by Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. ICLR, 2021.

  Defines loss that encourages finding minima with uniform low loss in a neighbourhood.

- "Exponential Learning Rate Schedules for Deep Learning", by Zhiyuan Li and Sanjeev Arora. ICLR, 2020.

  For commonly used values of the paremeters, every net produced by "Weight Decay + Constant LR + BN + Momentum" can also be produced (in function space) via "ExpLR + BN + Momentum"

  (Check out http://www.offconvex.org/2020/04/24/ExpLR1/)

Traditional statistical theory any help?

Unfortunately:

Traditional generalization approaches of bounding VC-dimension and Rademacher complexity not so useful at the moment.

An existing bound on VC dimension

For a ReLU network with

- n_p total parameters and
- L layers

then there exist constants c and C s.t.

$$c \cdot n_p \, L \, \log(n_p/L) \leq \text{VC dimension} \leq C \cdot n_p \, L \log(n_p)$$

Nearly-tight VC-dimension and pseudo-dimension bounds for piecewise linear neural networks, Peter L. Bartlett, Nick Harvey, Chris Liaw, Abbas Mehrabian, arXiv 2017

An existing bound on VC dimension

For a ReLU network with

- n_p total parameters and
- L layers

then there exist constants c and C s.t.

$$c \cdot n_p \, L \, \log(n_p/L) \leq \text{VC dimension} \leq C \cdot n_p \, L \log(n_p)$$

- Very loose bound.
- Also empirically shown large enough network can fit any random labellings of a finite number of inputs $\implies$ VC dimension is not useful to explain generalization ability.

Nearly-tight VC-dimension and pseudo-dimension bounds for piecewise linear neural networks, Peter L. Bartlett, Nick Harvey, Chris Liaw, Abbas Mehrabian, arXiv 2017

Example bound on Rademacher Complexity

- For a network with L layers and weight matrices $W_1, \ldots, W_L$
- Generalization error scales (with high probability) as

$$\mathcal{O}\left(\frac{B\,2^L\,\prod_{j=1}^{L} M_F(j)}{\sqrt{N}}\right)$$

where

- N is the number of training examples
- Norm of each input in bounded by B
- Frobenius norm of each weight matrix is bounded as

$$\|W_j\|_F \leq M_F(j) \qquad \text{for } j = 1, \ldots, L$$

Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. *Norm-based capacity control in neural networks*. In Conference on Learning Theory, 2015.

Example bound on Rademacher Complexity

- For a network with L layers and weight matrices $W_1, \ldots, W_L$
- Generalization error scales (with high probability) as

$$\mathcal{O}\left(\frac{B\,2^L\,\prod_{j=1}^{L} M_F(j)}{\sqrt{N}}\right)$$

where

- N is the number of training examples
- Norm of each input in bounded by B
- Frobenius norm of each weight matrix is bounded as

$$\|W_j\|_F \leq M_F(j) \qquad \text{for } j = 1, \ldots, L$$

Have exponential dependence on depth of network!

Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. *Norm-based capacity control in neural networks*. In Conference on Learning Theory, 2015.

Toward more prescriptive measures:
Measures of network capacity that correlate with generalization ability.
Involve: lipschitz / margin

Norm-based capacity measures

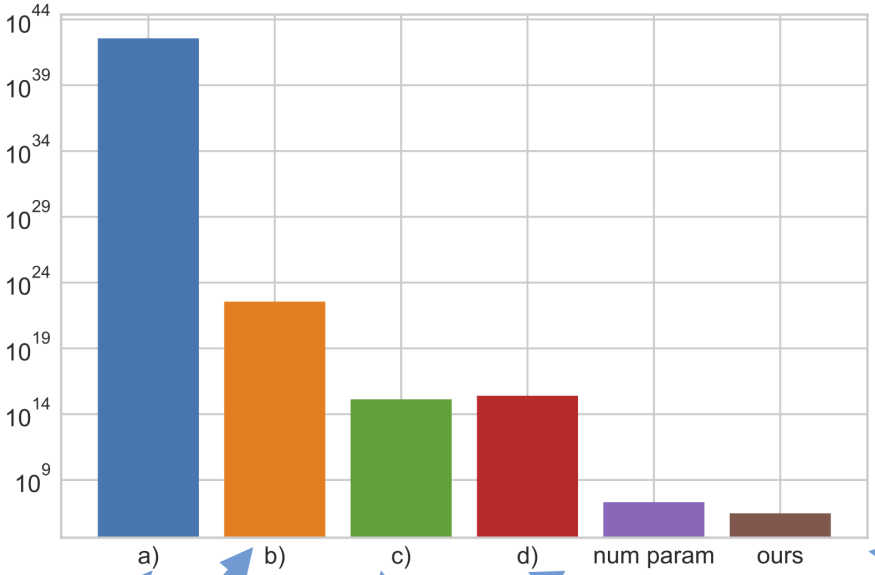
Capacity Type	**Capacity Order**	**Inntroduced**
$l_{1,\infty}$-norm	$\frac{1}{\gamma^2_{\text{marg}}} \prod_{l=1}^{L} \|W_l\|_{1,\infty}$	Bartlett & Mendelson, 2002
Frobenius	$\frac{1}{\gamma^2_{\text{marg}}} \prod_{l=1}^{L} \|W_l\|_F^2$	Neyshabur et al., 2015
spectral-$l_{1,2}$	$\frac{1}{\gamma^2_{\text{marg}}} \prod_{l=1}^{L} \sigma^2_{\max}(W_l) \sum_{i=1}^{L} \frac{\|W_i\|_{2,1}^2}{\|W_i\|_2^2}$	Barlett et al., 2017
spectral-frobenius	$\frac{1}{\gamma^2_{\text{marg}}} \prod_{l=1}^{L} \sigma^2_{\max}(W_l) \sum_{i=1}^{L} d_i \frac{\|W_i\|_F^2}{\|W_i\|_2^2}$	Neyshabur et al., 2018

where γ_{marg} is the smallest value of γ s.t. a fixed small percentage of the training examples have a margin smaller than γ.

Norm-based capacity measures

- But these quantitative estimates are fairly vacuous!
- orders of magnitude $>$ # of actual parameters.

VGG19
(19 layers)



[Bartlett-Mendelson'02]

[Neyshabur et al '17]

[Bartlett et al NIPS17],

[Neyshabur et al ICLR18]

[A., Ge, Neyshabur, Zhang'18]

Compression Approach

Basic Idea:

complexity of network = compressed size - remaining structure.

Source: Non-Vacuous generalization bounds at the ImageNet scale: A PAC-Bayesian compression approach by Wenda Zhou, Victor Veitch, Morgane Austern, Ryan P. Adams, Peter Orbanz, ICLR, 2019

Compression Approach

- Take a deep net $\mathcal{N}$ with n_p trainable parameters
- Compress $\mathcal{N}$ to another network $\tilde{\mathcal{N}}$ that has
  - many fewer parameters $\tilde{n}_p$ than $\mathcal{N}$ **and**
  - training error $\approx$ training error as $\mathcal{N}$.
- Basic theorem says:
  if $\tilde{n}_p <$ number of training samples then the compressed network $\tilde{\mathcal{N}}$ will generalize well.

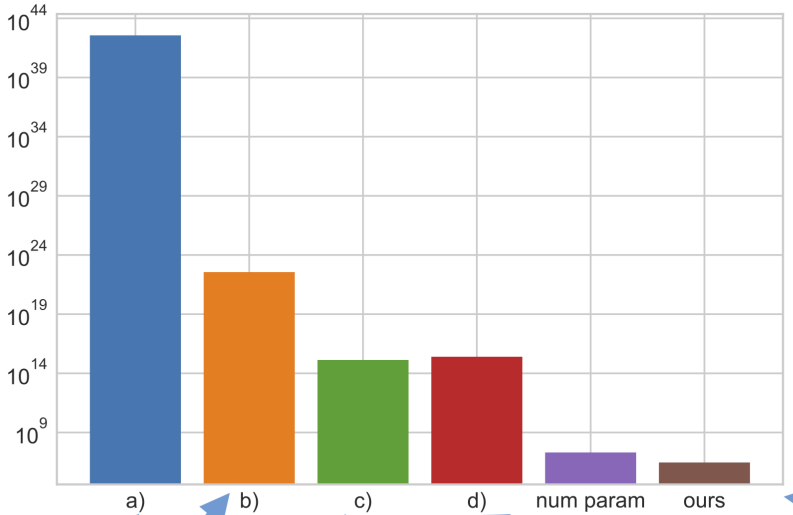
Source Sanjeev Arora: Proving generalization of deep nets via compression

Compression Approach

- Basic theorem says:
  if $\tilde{n}_p <$ number of training samples then the compressed network $\tilde{\mathcal{N}}$ will generalize well.

- Using measures from this result can put tighter bound on capacity of network.

VGG19
(19 layers)



[Bartlett-Mendelson'02]

[Neyshabur et al '17]

[Bartlett et al NIPS17],

[Neyshabur et al ICLR18]

[A., Ge, Neyshabur, Zhang'18]

Source: Sanjeev Arora, Proving generalization of deep nets via compression

Compression approach promising but..

"At the same time, I suspect that compressibility is only one part of the generalization mystery, and that we are still missing some big idea. I don't see how to use the above ideas to demonstrate that the effective number of parameters in VGG19 is as low as 50k, as seems to be the case. I suspect doing so will force us to understand the structure of the data (in this case, real-life images) which the above analysis mostly ignores. The only property of data used is that the deep net aligns itself better with data than with noise."

Sanjeev Arora

Source: Sanjeev Arora, Proving generalization of deep nets via compression

Neural Tangent Kernel:
another approach to analyse neural networks

Consider full batch gradient descent

- Let $f(\mathcal{X}, \theta_t) \equiv$ logit output of the entire training dataset at time iteration t, stacked into a vector and α the learning rate:

$$\begin{aligned} f(\mathcal{X}, \theta_{t+1}) &\approx f(\mathcal{X}, \theta_t) + \nabla_\theta f(\mathcal{X}, \theta)|_{\theta=\theta_t} \ (\theta_{t+1} - \theta_t) \\ &= f(\mathcal{X}, \theta_t) + \nabla_\theta f(\mathcal{X}, \theta)|_{\theta=\theta_t} \left(-\alpha \ \nabla_\theta L \left(f(\mathcal{X}, \theta), \mathcal{Y} \right)|_{\theta=\theta_t} \right) \\ &= f(\mathcal{X}, \theta_t) - \alpha \ \nabla_\theta f(\mathcal{X}, \theta)|_{\theta=\theta_t} \ \nabla_\theta f(\mathcal{X}, \theta)|_{\theta=\theta_t}^T \ \nabla_{f(\mathcal{X}, \theta_t)} L \end{aligned}$$

- Let $\mathbf{z}_t = f(\mathcal{X}, \theta_t)$ and $J_t = \nabla_\theta f(\mathcal{X}, \theta)|_{\theta=\theta_t}$

- Using the more streamlined notation

$$\mathbf{z}_{t+1} - \mathbf{z}_t \approx -\alpha J_t J_t^T \nabla_{f(\mathcal{X}, \theta_t)} L$$

- The matrix $K_t = J_t J_t^T$ is a Kernel matrix of the time varying kernel $k_t(\cdot, \cdot)$ evaluated on the training data

$$k_t(\mathbf{x}, \mathbf{x}') = \nabla_\theta f(\mathbf{x}, \theta_t)^T \ \nabla_\theta f(\mathbf{x}', \theta_t)$$

- Remember above model is only approximate...

Continuous time limit of gradient descent

- Continuous time GD $\implies$ evolution of logits $\mathbf{z}_t$ can be written as:

$$\frac{d\mathbf{z}_t}{dt} = -\alpha J_t J_t^T \nabla_{f(\mathcal{X},\theta_t)} L = -\alpha K_t \nabla_{f(\mathcal{X},\theta_t)} L$$

- Continuous time GD $\implies$ evolution of parameters θ:

$$\frac{d\theta_t}{dt} = -\alpha J_t^T \nabla_{f(\mathcal{X},\theta_t)} L$$

Neural Tangent Kernel

Jacot et al., 2018. "Neural tangent kernel: Convergence and generalization in neural networks"

- As the width of the network $\to \infty$, K_t approaches a well-defined limit we denote as K the **neural tangent kernel** (NTK).
- As the width increases, the distance in weight space required to fit the training set goes to 0
- In the limit, J_t, and therefore, K_t, are constant
- Dynamics of gradient flow for a MSE loss is a linear ODE

$$\frac{d\, f(\mathcal{X}, \theta_t)}{dt} = -\alpha K (f(\mathcal{X}, \theta_t) - \mathcal{Y})$$

- With solution

$$f_t(\mathcal{X}) = \mathcal{Y} + \exp\left(-\alpha K t\right)\left(f_0(\mathcal{X}) - \mathcal{Y}\right)$$

Neural Tangent Kernel

Lee et al., 2019. "Wide networks of any depth evolve as linear models under gradient descent"

- For wide but finite networks, J changes slowly enough during training.

  $\implies$ network well approximated by its first-order Taylor approximation around θ_0

  (Network behaves as linear model where the features are columns of J)

- Requires wider networks than normal, but not ridiculously so, a smaller learning rate, and full-batch GD training.

- **Still a gap between linearized training and SOTA, so probably neural nets are more than just linear random feature models.**

Neural Tangent Kernel

Lee et al., 2019. "Wide networks of any depth evolve as linear models under gradient descent"

- For wide but finite networks, J changes slowly enough during training.

  $\implies$ network well approximated by its first-order Taylor approximation around θ_0

  (Network behaves as linear model where the features are columns of J)

- Requires wider networks than normal, but not ridiculously so, a smaller learning rate, and full-batch GD training.

- **Still a gap between linearized training and SOTA, so probably neural nets are more than just linear random feature models.**

Neural Tangent Kernel

- These ideas lead to provable bounds for wide but finite networks.
  - A big challenge is proving the Jacobian changes slowly enough with high probability.
- Check out papers below for guarantees of convergence when training over-parametrized neural networks and also work on generalization.
  - Du et al., 2019. "Gradient descent provably optimizes over-parameterized neural networks"
  - Arora et al., 2019. "Fine-grained analysis of optimization and generalization for overparameterized two-layer neural networks"

Conclusions

- DL optimization is non-convex but bad local minima and saddle structures are rarely a problem (on common DL tasks).
- A stronger optimizer is not necessarily a stronger learner.
- Neural Networks are over-parametrized but can still generalize (optimization in fact often becomes easier).
- (Stochastic) Gradient Descent has a built-in implicit regularizer and seems to find "good" global minima
- Compressible / low Lipschitz networks generalize better.
- Variance in gradient can help with generalization but can hurt final convergence.